

Componentes/Item/STDFindItem.prw

1091 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "TBICONN.CH"
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "STDFindItem.CH"
6
7 Static aStartProds //Array contendo os produtos em memória
8
9 //-----
10 /*{Protheus.doc} STDFindItem
11 Busca Item na tabela de produtos
12
13 @param   cCodeReceived      Codigo recebido/digitado
14 @return  aRet               Array com informações do Item
15 @obs
16 @sample
17 */
18 //-----
19 Function STDFindItem( cCodeReceived, lPAFECECF, lHomolPaf )
20 Local      aArea              := GetArea()           // Guarda area atual
21 Local      lFindItem          := .F.                // Encontrou Item?
22 Local      aRet               := {}                 // Retorno
23 Local      nTamProd           := TamSx3("B1_COD")[1] // Tamanho codigo produto
24 Local      nTamCodBar         := TamSx3("B1_CODBAR")[1] // Tamanho codigo de barra
25 Local      cLOCPAD            := ""
26 Local      cORIGEM            := ""
27 Local      cCODISS            := ""
28 Local      cTS                := ""
29
30 Default    cCodeReceived      := ""                 // Codigo recebido/digitado
31 Default    lPAFECECF          := StbIsPaf()
32 Default    lHomolPaf          := StbHomolPaf()
33
34 //Inicializa variaveis de retorno
35 aRet := STDIniRet()
36
37 /*
38 Procura na SB1 pelo Indice 1 ( B1_FILIAL + B1_COD )
39 */
40 DbSelectArea("SB1")
41 SB1->(DbSetOrder(1)) //B1_FILIAL+B1_COD
42 If DbSeek( xFilial("SB1") + Padr(cCodeReceived,nTamProd))
43     lFindItem := .T.
44 EndIf
45
46 /*
47 Se nao achou o Item
48 Procura na SB1 pelo Indice 5 ( B1_FILIAL + B1_CODBAR )
49 */
50 If (!lFindItem)
51     DbSelectArea("SB1")
52     DbSetOrder(5) //B1_FILIAL+B1_CODBAR

```

```

53   If DbSeek ( xFilial("SB1") + Padr(cCodeReceived,nTamCodBar) )
54     lFindItem := .T.
55   EndIf
56 EndIf
57
58 /*
59   Se achou o Item
60   Verifica se o item esta bloqueado e atribui variaveis de retorno
61 */
62 If lFindItem
63   /*Tratamento para Quantidade para itens que não tenham quantidade na tabela SLK*/
64   If STBGetQuant(2) > 0
65     STBSetQuant(STBGetQuant(2),1) //Retomo a Qtd de backup a Qtd oficial
66     STBSetQuant(0,2) //Zero o Qtd de backup
67   EndIf
68
69   aRet[ITEM_ENCONTRADO] := .T. // 01 - Encontrou Item?
70   aRet[ITEM_CODIGO] := SB1->B1_COD // 02 - Codigo do Item encontrado
71   aRet[ITEM_CODBAR] := SB1->B1_CODBAR // 03 - Codigo de barras do Item encontrado
72
73   If SB1->B1_MSBLQL=="1"
74     aRet[ITEM_BLOQUEADO]:= .T. // 04 - 0 item esta bloqueado para venda
75   EndIf
76
77   cLOCPAD := RetFldProd(SB1->B1_COD,"B1_LOCPAD")
78   cORIGEM := RetFldProd(SB1->B1_COD,"B1_ORIGEM")
79   cCODISS := RetFldProd(SB1->B1_COD,"B1_CODISS")
80   cTS := RetFldProd(SB1->B1_COD,"B1_TS")
81
82   aRet[ITEM_TES] := Iif(Empty(cTS), SB1->B1_TS, cTS) // 06 - TES do item
83   aRet[ITEM_BALANCA] := SB1->B1_BALANCA // 07 - Tipo produto de balanca
84   aRet[ITEM_DESCRICA0] := SB1->B1_DESC // 08 - Descrição do item
85   aRet[ITEM_UNID_MEDIDA] := SB1->B1_UM // 09 - Unidade de Medida do item
86   aRet[ITEM_ARMAZEMPAD] := Iif(Empty(cLOCPAD), SB1->B1_LOCPAD, cLOCPAD) // 10 - Armazem Padrão do item
87   aRet[ITEM_TIPO] := SB1->B1_TIPO // 11 - Tipo do Produto
88   aRet[ITEM_ORIGEM] := Iif(Empty(cOrigem), SB1->B1_ORIGEM, cOrigem) // 11 - Código Origem
89   aRet[ITEM_CODISS] := Iif(Empty(cCodISS), SB1->B1_CODISS, cCodISS) // 12 - Código ISS
90   aRet[ITEM_POSIPI] := SB1->B1_POSIPI // 13 - POSIPI
91   aRet[ITEM_INT_ICM] := SB1->B1_INT_ICM // 14 - INT_ICM
92   aRet[ITEM_EX_NCM] := SB1->B1_EX_NCM // 20 - Extensao NCM
93   aRet[ITEM_RECNO] := SB1->(RECNO()) // 24 - Recno do Produto na tabela SB1
94
95   If lPAFECF
96     STDVlItPAF(@aRet,lHomolPAF)
97   EndIf
98
99 EndIf
100
101 //Restaura area anterior
102 RestArea(aArea)
103
104 Return aRet

```

```

105
106 //-----
107 /*{Protheus.doc} STDFItemCB
108 Busca Item na tabela de codigos de barras
109
110 @param   cCodeReceived      Codigo recebido/digitado
111 @return  aRet               Array com informações do Item
112 @obs
113 @sample
114 */
115 //-----
116 Function STDFItemCB( cCodeReceived, lPAFECF,lHomolPaf )
117 Local aArea      := GetArea()           // Guarda area atual
118 Local lFindItem  := .F.                 // Encontrou Item?
119 Local aRet       := {}                 // Retorno
120 Local nTamCodBar := TamSx3("LK_CODBAR")[1] // Tamanho do Codigo de barras
121 Local lCpoTotImp := SB1->(ColumnPos("B1_IMPNCM")) > 0 // Lei dos impostos
122 Local nVlrImpNCM := 0                 // Valor do imposto vindo a partir do codigo da NCM - Lei dos impostos
123 Local cLOCPAD    := ""
124 Local cORIGEM    := ""
125 Local cCODISS    := ""
126 Local nIMPNCM    := 0
127 Local cTS        := ""                // TES do item
128
129 Default cCodeReceived := ""           // Codigo recebido/digitado
130 Default lPAFECF       := StbIsPaf()
131 Default lHomolPaf     := StbHomolPaf()
132
133 //Inicializa variaveis de retorno
134 aRet := STDIniRet()
135
136 /*
137 Procura Pela SLK Indice 1 ( LK_FILIAL + LK_CODBAR )
138 */
139 DbSelectArea("SLK")
140 DbSetOrder(1) //LK_FILIAL + LK_CODBAR
141 If DbSeek( xFilial("SLK") + Padr(cCodeReceived,nTamCodBar))
142
143 LjGrvLog( "STDFItemCB", "Achou SLK ",SLK->LK_CODIGO)
144
145 /*
146 Se achou o Item na SLK Busca na tabela SB1
147 Procura Pelo SB1 Indice 1 ( B1_FILIAL + B1_COD )
148 */
149 DbSelectArea("SB1")
150 DbSetOrder(1) //B1_FILIAL+B1_COD
151 If DbSeek ( xFilial("SB1") + SLK->LK_CODIGO )
152 lFindItem := .T.
153 LjGrvLog( "STDFItemCB", "Achou SB1 ",SB1->B1_COD )
154 EndIf
155
156 EndIf

```

```

157
158 /*
159     Se achou o Item
160     Verifica se o item esta bloqueado e atribui variaveis de retorno
161 */
162 If  lFindItem
163
164     aRet[ITEM_ENCONTRADO] := .T.           // 01 - Encontrou Item?
165     aRet[ITEM_CODIGO]    := SB1->B1_COD    // 02 - Código do Item encontrado
166     aRet[ITEM_CODBAR]    := SLK->LK_CODBAR // 03 - Código de barras do Item encontrado
167
168     If SB1->B1_MSBLQL=="1"
169         aRet[ITEM_BLOQUEADO] := .T.       // 04 - 0 item esta bloqueado para venda
170     EndIf
171
172 /*
173     Se alterar a quantidade do item pela quantidade
174     encontrada na tabela de codigo de barras
175 */
176 If SLK->LK_QUANT > 1
177     STBSetQuant(1)
178     aRet[ITEM_QTDE] := ( Iif (STBGetQuant(2) == 0,1,STBGetQuant(2)) * SLK->LK_QUANT ) // 05 - Quantidade do item
179 Else
180     If (STBGetQuant(2) > 0 .And. STBGetQuant(2) < 1) .Or. STBGetQuant(2) > 1 //Se a quantidade for 1 indica que ja realizou o ajuste
181         STBSetQuant(STBGetQuant(2))
182         STBSetQuant(1,2)
183     EndIf
184 EndIf
185
186 cLOCPAD      := RetFldProd(SB1->B1_COD,"B1_LOCPAD")
187 cORIGEM      := RetFldProd(SB1->B1_COD,"B1_ORIGEM")
188 cCODISS      := RetFldProd(SB1->B1_COD,"B1_CODISS")
189 aRet[ITEM_BALANCA] := SB1->B1_BALANCA // 07 - Tipo produto de balanca
190 aRet[ITEM_DESCRICA0] := SB1->B1_DESC // 08 - Descrição do item
191 aRet[ITEM_UNID_MEDIDA] := SB1->B1_UM // 09 - Unidade de Medida do item
192 aRet[ITEM_ARMAZEMPAD] := Iif(Empty(cLOCPAD), SB1->B1_LOCPAD, cLOCPAD) // 10 - Armazem Padrão do item
193 aRet[ITEM_TIPO] := SB1->B1_TIPO // 11 - Tipo do Produto
194 aRet[ITEM_CODISS] := Iif(Empty(cCodISS), SB1->B1_CODISS, cCodISS) // 12 - Código ISS
195 aRet[ITEM_POSIPI] := SB1->B1_POSIPI // 13 - POSIPI
196 aRet[ITEM_INT_ICM] := SB1->B1_INT_ICM // 14 - INT_ICM
197 aRet[ITEM_ORIGEM] := Iif(Empty(cORIGEM), SB1->B1_ORIGEM, cORIGEM) // 16 - Código Origem
198 aRet[ITEM_EX_NCM] := SB1->B1_EX_NCM // 20 - Extensao NCM
199
200 If SB1->B1_MSBLQL=="1"
201     aRet[ITEM_BLOQUEADO] := .T. // 04 - 0 item esta bloqueado para venda
202 EndIf
203
204 cTS := RetFldProd(SB1->B1_COD,"B1_TS")
205 aRet[ITEM_TES] := Iif(Empty(cTS), SB1->B1_TS, cTS) // 06 - TES do item
206
207 If lPAFECE
208     STDVlItPAF(@aRet,lHomolPAF)

```

```

209 EndIf
210
211 //Lei dos impostos
212 If lCpoTotImp
213     nIMPNCM := RetFldProd(SB1->B1_COD, "B1_IMPNCM")
214     nVlrImpNCM := Iif(Empty(nIMPNCM), SB1->B1_IMPNCM, nIMPNCM)
215 EndIf
216
217 If nVlrImpNCM == 0 .AND. !Empty(SB1->B1_POSIPI)
218     cCodISS := Iif(Empty(cCodISS), SB1->B1_CODISS, cCodISS)
219     nVlrImpNCM := Lj7BuscaImp(SB1->B1_POSIPI, !Empty(cCodISS), .F.)
220 EndIf
221
222 aRet[ITEM_TOTIMP] := nVlrImpNCM
223 aRet[ITEM_RECNO] := SB1->(RECNO()) // 24 - Recno do Produto na tabela SB1
224 EndIf
225
226 //Restaura area anterior
227 RestArea(aArea)
228
229 Return aRet
230
231 //-----
232 /*{Protheus.doc} STDFItemBal
233 Busca se e etiqueta criada na balanca
234
235 @param cCodeReceived      Codigo recebido/digitado
236 @return aRet              Array com informações do Item
237 @obs
238 @sample
239 */
240 //-----
241 Function STDFItemBal( cCodeReceived )
242 Local aArea := GetArea() // Guarda area atual
243 Local lFindItem := .F. // Encontrou Item?
244 Local aRet := {} // Retorno
245 Local cCodProBal := "" //Codigo do Item da balanca
246 Local cInfProBal := "" // Peso/quantidade do Item da balanca
247 Local nItemQtde := 0 // Quantidade do Item lido na etiqueta
248 Local cB1_Balanca := ""
249
250 Default cCodeReceived := "" // Codigo recebido/digitado
251
252 //Inicializa variaveis de retorno
253 aRet := STDIniRet()
254
255 /*
256 Verifica se e etiqueta criada na balanca
257 */
258
259 //Separa o codigo recebido para fazer validacao da balanca
260 cCodProBal := Substr(cCodeReceived,2,6) //Codigo do Item da balanca

```

```

261 cInfProBal := SubStr(cCodeReceived,8,5) //Peso/quantidade do Item da balanca
262
263 /*
264 Procura Pelo SB1 Indice 1 ( B1_FILIAL + B1_COD ) ou Pelo SB1 Indice 5 ( B1_FILIAL + B1_CODBAR )
265 */
266 SB1->(DbSelectArea("SB1"))
267 SB1->(DbSetOrder(1))//B1_FILIAL+B1_COD
268 If !(SB1->(DbSeek(xFilial("SB1") + cCodProBal)))
269     SB1->(DbSetOrder(5))//B1_FILIAL+B1_CODBAR
270     SB1->(DbSeek(xFilial("SB1") + cCodProBal ))
271 EndIf
272
273 cB1_Balanca := AllTrim(SB1->B1_BALANCA)
274
275 If ( (AllTrim(cCodProBal) == Alltrim(SB1->B1_COD) .OR. AllTrim(cCodProBal) == Alltrim(SB1->B1_CODBAR) ) .AND. ( cB1_Balanca $ "123" ) )
276
277     lFindItem             := .T.
278
279     /*
280     SB1->B1_BALANCA
281     Indica se o produto e pesavel na balança e
282     quais as informacoes que virao na Etiqueta:
283     0 - Nao usa balanca
284     1 -Codigo + Preco
285     2 -Codigo + Peso
286     3 -Codigo + Unidade
287     */
288     Do Case
289
290         /* ATENCAO!
291         No Caso do B1_BALANCA == "1"
292         sera tratado apos pegar o preco do item
293         */
294
295         Case ( cB1_Balanca == "2" )             // 2 - Quantidade (Peso)
296
297             // Verifica se e item de balanca para pegar peso - ( Quantidade = Peso )
298             If AllTrim(SB1->B1_UM) $ "G|MG|KG"
299                 nItemQtde := ( Val(cInfProBal) / 1000 )
300             Else
301                 STBBalQuant()
302             EndIf
303
304         Case ( cB1_Balanca == "3" )             // 3 - Quantidade (Unidade)
305
306             nItemQtde := Val(cInfProBal)
307
308     Endcase
309
310 EndIf
311
312

```

```

313 /*
314     Se achou o Item
315     Verifica se o item esta bloqueado e atribui variaveis de retorno
316 */
317 If lFindItem
318
319     aRet[ITEM_ENCONTRADO] := .T.           // 01 - Encontrou Item?
320     aRet[ITEM_CODIGO]    := SB1->B1_COD    // 02 - Codigo do Item encontrado
321     aRet[ITEM_CODBAR]    := SB1->B1_CODBAR // 03 - Codigo de barras do Item encontrado
322
323     If SB1->B1_MSBLQL=="1"
324         aRet[ITEM_BLOQUEADO] := .T.       // 04 - 0 item esta bloqueado para venda
325     EndIf
326
327 /*
328     Se alterar a quantidade do item pela quantidade
329     encontrada na tabela de codigo de barras
330 */
331 If nItemQtde > 0
332     aRet[ITEM_QTDE]      := nItemQtde     // 05 - Quantidade do item
333 EndIf
334 cLOCPAD      := RetFldProd(SB1->B1_COD,"B1_LOCPAD")
335 cTS          := RetFldProd(SB1->B1_COD,"B1_TS")
336 cCODISS      := RetFldProd(SB1->B1_COD,"B1_CODISS")
337 cORIGEM      := RetFldProd(SB1->B1_COD,"B1_ORIGEM")
338
339 aRet[ITEM_TES]      := Iif(Empty(cTS), SB1->B1_TS, cTS) // 06 - TES do item
340 aRet[ITEM_BALANCA] := SB1->B1_BALANCA // 07 - Tipo produto de balanca
341 aRet[ITEM_DESCRICA0] := SB1->B1_DESC // 08 - Descrição do item
342 aRet[ITEM_UNID_MEDIDA] := SB1->B1_UM // 09 - Unidade de Medida do item
343 aRet[ITEM_ARMAZEMPAD] := Iif(Empty(cLOCPAD), SB1->B1_LOCPAD, cLOCPAD) // 10 - Armazem Padrão do item
344 aRet[ITEM_TIPO]      := SB1->B1_TIPO // 11 - Tipo do Produto
345 aRet[ITEM_CODISS]    := Iif(Empty(cCODISS), SB1->B1_CODISS, cCODISS) // 12 - Codigo ISS
346 aRet[ITEM_POSIPI]    := SB1->B1_POSIPI // 13 - POSIPI
347 aRet[ITEM_INT_ICM]   := SB1->B1_INT_ICM // 14 - INT_ICM
348 aRet[ITEM_ORIGEM]    := Iif(Empty(cORIGEM), SB1->B1_ORIGEM, cORIGEM) // 16 - Código Origem
349 aRet[ITEM_EX_NCM]    := SB1->B1_EX_NCM // 20 - Extensao NCM
350
351 EndIf
352
353 //Restaura area anterior
354 RestArea(aArea)
355
356 Return aRet
357
358 //-----
359 /*{Protheus.doc} STDIniRet
360 Inicializa Variaveis de retorno
361
362 @param    nenhum
363 @return   aRet           Array com informações do Item
364 @obs

```

```

365 @sample
366 /*/
367 //-----
368 Static Function STDIniRet()
369 Local aRet := Array(ITEM_TOTAL_ARRAY) // Retorno
370
371 //Inicializa variaveis de retorno
372 aRet[ITEM_ENCONTRADO] := .F. // 01 - Encontrou Item?
373 aRet[ITEM_CODIGO] := "" // 02 - Codigo do Item encontrado
374 aRet[ITEM_CODBAR] := "" // 03 - Codigo de barras do Item encontrado
375 aRet[ITEM_BLOQUEADO] := .F. // 04 - 0 item esta bloqueado para venda
376 aRet[ITEM_QTDE] := 0 // 05 - Quantidade do item
377 aRet[ITEM_TES] := "" // 06 - TES do item
378 aRet[ITEM_BALANCA] := "" // 07 - Tipo produto de balanca
379 aRet[ITEM_DESCRICAO] := "" // 08 - Descrição do item
380 aRet[ITEM_UNID_MEDIDA] := "" // 09 - Unidade de Medida do item
381 aRet[ITEM_ARMAZEMPAD] := "" // 10 - Armazem Padrão do item
382 aRet[ITEM_TIPO] := "" // 11 - Tipo do item
383 aRet[ITEM_SITTRIB] := "" // 12 - ST do item
384 aRet[ITEM_IAT] := "" // 13 - IAT do item
385 aRet[ITEM_IPPT] := "" // 14 - PPT do item
386 aRet[ITEM_TOTIMP] := "" // 15 - Impostos - Lei dos impostos
387 aRet[ITEM_ORIGEM] := "" // 16 - Código Origem
388 aRet[ITEM_CODISS] := "" // 17 - Código ISS
389 aRet[ITEM_POSIPI] := "" // 18 - POSIPI
390 aRet[ITEM_INT_ICM] := 0 // 19 - ICM
391 aRet[ITEM_EX_NCM] := "" // 20 - Extensao do codigo NCM
392 aRet[ITEM_TOTFED] := 0 // 21 - Impostos - Lei dos impostos Federal
393 aRet[ITEM_TOTEST] := 0 // 22 - Impostos - Lei dos impostos Estadual
394 aRet[ITEM_TOTMUN] := 0 // 23 - Impostos - Lei dos impostos Municipal
395 aRet[ITEM_RECNO] := 0 // 24 - Recno do Produto na tabela SB1
396
397 Return aRet
398
399 //-----
400 /*/{Protheus.doc} STBGetTESInfo
401 Pega informacoes da TES
402 @param cField - Informacao que quer recuperar da TES
403 @param cTES - Codigo da TES
404 @return Retorna informacoes da TES
405 @obs
406 @sample
407 /*/
408 //-----
409 Function STBGetTESInfo( cField , cTES )
410 Local aArea := GetArea() // Guarda area atual
411 Local uRet := Nil // Retorno
412
413 Default cField := ""
414 Default cTES := ""
415
416 If !Empty(cField) .AND. !Empty(cField)

```



```

417
418     SF4->(DbSeek(xFilial("SF4")+cTES))
419
420     DBSelectArea("SF4")
421     DBSetOrder(1)//F4_FILIAL+F4_CODIGO
422     If DbSeek( xFilial("SF4")+cTES )
423         uRet := SF4->(cField)
424     EndIf
425
426 EndIf
427
428 //Restaura area anterior
429 RestArea(aArea)
430
431 Return uRet
432
433 //-----
434 /*{Protheus.doc} STDFindProd
435 Realiza a busca de Produtos
436 @param cField - Informacao que quer recuperar da TES
437 @param cTES   -Codigo da TES
438 @return Retorna informacoes da TES
439 @obs
440 @sample
441 */
442 //-----
443 Function STDFindProd( cFilter, cCliCode, cCliLoja, nMoeda, ;
444                     nLimitRegs, lPAFECE, lHomolPAF, aRecno, cSearchFields)
445 Local aProdSort := {} // -- Array com os produtos ordenados
446 Local aProducts := {} // -- Array com os produtos
447 Local aArea := GetArea() // -- Area Atual
448 Local aAreaSB1 := SB1->(GetArea()) // -- Area da tabela SB1
449 Local cMoedaSimb := SuperGetMV("MV_SIMB" + Str(nMoeda,1)) // -- Moeda atual
450 Local nPrice := 0 // -- Variavel utilizada para armazenar o preço do produto
451 Local nMV_LJPEATU := SuperGetMV("MV_LJPEATU",,0) // -- Define aonde será buscado o cliente (0=Pesquisa Local, 1=Pesquisa preferencialmente Local se falar
    pesquisa na retaguarda, 2=Pesquisa somente na retaguarda)
452 Local aRet := {} // -- Variavel de retorno
453 Local aProdSeek := {} // -- Variavel com os produtos buscados
454 Local nProd := 0 // -- Variavel de controle "For"
455 Local uResult := Nil // -- Resultado da função remote execute
456 Local lRet := .F. // -- Retorno da execução da função remoe execute
457 Local lLocalSearch := .F. // -- Indica se a busca foi feita localmente ou se foi feita na retaguarda.
458 Local aAux := Array(ITEM_TOTAL_ARRAY) // -- Array Auxiliar utilizado em ambientes PAF-EC
459 Local cSitTrib := "" // -- Situação tributaria do produto
460 Local lSTDPrcKit := ExistFunc("STDPrcKit") // -- Verifica a existencia da função STDPrcKit
461
462 Default cFilter := "" // -- Campo a ser buscado
463 Default cCliCode := SuperGetMV("MV_CLIPAD",, "") // -- Cliente padrao
464 Default cCliLoja := SuperGetMV("MV_LOJAPAD",, "") // -- Loja Cliente Padrão
465 Default nMoeda := STBGetCurrency() // -- Moeda corrente
466 Default nLimitRegs := SuperGetMV("MV_LJQTDPL",,20) // -- Limite de registro a serem buscados
467 Default lPAFECE := StbIsPaf() // -- Indica se é PAF ECF

```

```

468 Default lHomolPAF      := StbHomolPaf()           // -- Indica se é Homologação PAF
469 Default aRecno          := {}                     // -- REcno dos registros encontrados
470 Default cSearchFields   := ""                     // -- Campos que serão utilizados na busca.
471
472 cFilter := AllTrim(cFilter)
473
474 If "*" $ cFilter
475     cFilter := Replace(cFilter,"*", "")
476 EndIf
477
478 If nMV_LJPEATU == 0      // -- Pesquisa Somente local
479     LjGrvLog( ProcName(),"Parametro MV_LJPEATU = 0, as busca serão realizadas apenas no PDV")
480
481     STFMessage(ProcName(), "RUN", STR0001 ,{ || aRet := STDSearchP(cFilter,nLimitRegs,cSearchFields) }) // -- "Procurando produtos no PDV..."
482     STFSHOWMESSAGE(ProcName())
483
484     aProdSeek      := aRet
485     lLocalSearch    := .T.
486
487 ElseIf nMV_LJPEATU == 2 // -- Pesquisa Somente na retaguarda
488     LjGrvLog( ProcName(),"Parametro MV_LJPEATU = 2, as busca serão realizadas apenas na Retaguarda")
489
490     aParam := {cFilter,nLimitRegs,cSearchFields}
491
492     STFMessage(ProcName(), "RUN", STR0003 ,{ || lRet := STBRemoteExecute("STDSearchP", aParam,,, @uResult) }) // -- "Procurando produtos na retaguarda..."
493     STFSHOWMESSAGE(ProcName())
494
495     If lRet
496         aProdSeek      := uResult
497     Else
498         LjGrvLog( ProcName(),"Sem comunicação com o servidor.")
499         STFMessage(ProcName(),"STOP",STR0004) // -- "Sem comunicação com o servidor."
500         STFSHOWMESSAGE(ProcName())
501     EndIf
502 Else
503
504     LjGrvLog( ProcName(),"Parametro MV_LJPEATU = 1, as busca serão realizadas no PDV e caso nenhum produto seja encontrado será realizada uma nova busca na retaguarda")
505
506     STFMessage(ProcName(), "RUN", STR0001 ,{ || aRet := STDSearchP(cFilter,nLimitRegs,cSearchFields) }) // -- "Procurando produtos no PDV..."
507     STFSHOWMESSAGE(ProcName())
508
509     aProdSeek      := aRet
510
511     If Len(aProdSeek) == 0
512
513         // -- Se não encontrou nenhum produto realiza a busca na retaguarda.
514         STFMessage(ProcName(),"YESNO",STR0005) // -- produto não encontrado na base local. Deseja realizar a busca no servidor?
515
516         LjGrvLog( ProcName(),"Nenhum produto encontrado nas busca no PDV.")
517
518         If STFSHOWMESSAGE(ProcName())
519             LjGrvLog( ProcName(),"Usuario optou por realizar nova busca na retaguarda")

```

```

520
521     aParam := {cFilter,nLimitRegs,cSearchFields}
522
523     STFMessage(ProcName(), "RUN", STR0003 ,{ || lRet := STBRemoteExecute("STDSearchP", aParam,,, @uResult) }) // -- "Procurando produtos na retaguarda..."
524     STFShowMessage(ProcName())
525
526     If lRet
527         aProdSeek      := uResult
528     Else
529         LjGrvLog( ProcName(),"Sem comunicação com o servidor.")
530         STFMessage(ProcName(),"STOP",STR0004) // -- "Sem comunicação com o servidor."
531         STFShowMessage(ProcName())
532     EndIf
533     Else
534         LjGrvLog( ProcName(),"Nenhum produto encontrado nas busca no PDV e o usuario optou por não realizar buscas na retaguarda.")
535     EndIf
536 Else
537     // -- Achou o produto Localmente
538     lLocalSearch      := .T.
539 EndIf
540
541 EndIf
542
543 If Len(aProdSeek) == 0
544     LjGrvLog( ProcName(),"Nenhum produto encontrado.")
545     STFMessage(ProcName(),"STOP",STR0002) // -- Nenhum produto encontrado.
546     STFShowMessage(ProcName())
547 EndIf
548
549 DbSelectArea("SB1")
550 SB1->(DbSetOrder(1)) //B1_FILIAL+B1_COD
551
552 For nProd := 1 To Len(aProdSeek)
553
554     If lLocalSearch .Or. SB1->( DbSeek(xFilial("SB1") + aProdSeek[nProd][2]) )
555
556         If lSTDPrcKit .AND. aProdSeek[nProd][7] == "KT"
557             nPrice := STDPrcKit( aProdSeek[nProd][2], cCliCode, Nil, cCliLoja, nMoeda)
558         Else
559             nPrice := STWFormPr( aProdSeek[nProd][2], cCliCode, Nil, cCliLoja, nMoeda)
560         Endif
561
562         If nPrice > 0
563             If !lPAFECE
564                 aAdd(aProducts,{AllTrim(aProdSeek[nProd][1])                + " / " + ;
565                                     AllTrim(aProdSeek[nProd][2])            + " / " + ;
566                                     cMoedaSimb+AllTrim(Transform(nPrice,PesqPict("SL1","L1_VLRTOT")))+ " / " + ;
567                                     aProdSeek[nProd][1]                      + " / " + ;
568                                     Iif(lLocalSearch,aProdSeek[nProd][6],SB1->(Recno()))+ " / " + ;
569             Else
570                 STDVlItPAF( @aAux, lHomolPAF, .F. )
571                 cSitTrib := aAux[ITEM_SITTRIB]

```

```

572
573             aAdd(aProducts,{AllTrim(aProdSeek[nProd][1])           + " / " +;
574                     AllTrim(aProdSeek[nProd][2])           + " / " +;
575                     cMoedaSimb+AllTrim(Transform(nPrice,PesqPict("SL1","L1_VLRTOT")) + " / " +;
576                     AllTrim(aProdSeek[nProd][3])           + " / " +;
577                     AllTrim(cSitTrib)                       + " / " +;
578                     AllTrim(aProdSeek[nProd][4])           + " / " +;
579                     AllTrim(aProdSeek[nProd][5])           ,;
580                     aProdSeek[nProd][2]                     ,;
581                     Iif(lLocalSearch,aProdSeek[nProd][6],SB1->(Recno())) } )
582         EndIf
583     EndIf
584 Else
585         LjGrvLog( ProcName(),"o Produto:" + aProdSeek[nProd][2] + " Descrição: " + aProdSeek[nProd][1] + " Não Foi encontrado localmente, por esse motivo será ignorado." )
586     EndIf
587 Next
588
589 //Ordena pelo código
590 ASort(aProducts,, { |x, y| x[2] < y[2] } )
591
592 //Redimensiona o array
593 AEval( aProducts, { |a| aAdd( aProdSort, a[1] ) } )
594 AEval( aProducts, { |a| aAdd( aRecno , a[3] ) } )
595
596 RestArea(aAreaSB1)
597 RestArea(aArea)
598
599 Return aProdSort
600
601 //-----
602 /*{Protheus.doc} STDStartProd
603 Funcao utilizada para fazer o posicionamento da SB1, com o intuito de evitar lentidoes na pesquisa de produtos
604 @param lIncre - Define se esta sendo chamado pela carga incremental e nesse caso
605             incrementa o Array (.T.), caso contrario e tratado pelo start
606 @param lAdic - //Define se na carga incremental, esta sendo criado
607             um registro no cadastro de produto ou excluindo o mesmo
608 @return Retorna informacoes da TES
609 @obs
610 @sample
611 */
612 //-----
613 Function STDStartProd(lIncre,lAdic)
614
615 Local aArea      := GetArea()           //Guarda area
616 Local aAreaSX3   := SX3->( GetArea() ) //Guarda area SX3
617 Local nSec       := 0                   //Controle de tempo
618 Local aProds     := {}                  //array de produtos
619 Local dDtAtuPrd  := cTod(" / / ") //data atualizacao
620 Local nDiasVld   := SuperGetMV("MV_LJDVLC",,1) //Parametro com a quantidade de dias que a variavel Global fica ativa
621 Local nPos       := 0                   //Variavel usada como ponteiro do array
622 Local cPesq      := ""                  //Chave da pesquisa contendo todos os campos
623

```

```

624 //Define se esta sendo chamado pela carga incremental e nesse caso incrementa o Array (.T.), caso contrario e tratado pelo start
625 Default lIncre      := .F.
626
627 //Define se na carga incremental, esta sendo criado um registro no cadastro de produto ou excluindo o mesmo
628 Default lAdic       := .T.
629
630 //Verifica se a variavel global da consulta de produto será realizada por JOB 1=Sim e 2=Nao
631 If (SuperGetMV("MV_LJAJCP",,"2") == "1")
632
633     //Verifica se ja existe conteudo na variavel Global
634     If !GetGlbVars("TotvsPDV",@aProds,@dDtAtuPrd)
635         //Funcao que concatena todos os campos para criar uma descricao unica para pesquisa
636         cPesq := STDConcPesq()
637
638         aProds := {}
639
640         nSec := Seconds()
641
642         DbSelectArea("SB1")
643         SB1->(DbSetOrder(3)) // B1_FILIAL + B1_DESC
644         SB1->(DbSeek(xFilial("SB1"))) //Posiciona no primeiro registro
645
646         If SB1->(!EOF())
647             SB1->(DbGoTo(SB1->(LastRec())+1))
648             SB1->(DbSkip(-1))
649         EndIf
650
651         While SB1->(!BOF())
652             If SB1->B1_MSBLQL <> '1'
653                 Add(aProds,{&(cPesq), SB1->(Recno())})
654             EndIf
655             SB1->(DbSkip(-1))
656         End
657
658         dDtAtuPrd := Date()
659
660         //Atualiza a variavel Global
661         PutGlbVars("TotvsPDV",aProds,dDtAtuPrd)
662
663         nSec := Seconds() - nSec
664
665     Else
666         //Verifica se a data de criacao do parametro eh superior ao parametrizado
667         If Date() >= (dDtAtuPrd + nDiasVld)
668             //Zero a variavel global
669             ClearGlbValue("TotvsPDV")
670
671             //Chamo a rotina novamente, mas com as variaveis globais zeradas para que crie novamente os registros
672             STDStartProd()
673         ElseIf lIncre
674             If lAdic
675

```

```

676         cPesq := STDConcPesq()
677
678         //Adiciona o novo registro posicionado no produto
679         If aScan(aProds,{|x| x[2] == SB1->(Recno()) }) == 0
680             Aadd(aProds,{&(cPesq), SB1->(Recno())})
681
682             dDtAtuPrd := Date()
683
684             //Atualiza a variavel Global
685             PutGlbVars("TotvsPDV",aProds,dDtAtuPrd)
686         EndIf
687     Else
688         //Exclui o registro posicionado no produto
689         If (nPos := aScan(aProds,{|x| x[2] == SB1->(Recno()) }))) <> 0
690             ADel( aProds, nPos )
691             ASize( aProds, Len(aProds) - 1 )
692
693             dDtAtuPrd := Date()
694
695             //Atualiza a variavel Global
696             PutGlbVars("TotvsPDV",aProds,dDtAtuPrd)
697         EndIf
698     EndIf
699 EndIf
700 EndIf
701
702 aStartProds := aClone(aProds)
703
704 Else
705     //Nao utiliza variavel global
706     If ValType(aStartProds) <> "A"
707         aStartProds := {}
708         cPesq := STDConcPesq()
709
710         nSec := Seconds()
711
712         DbSelectArea("SB1")
713         SB1->(DbSetOrder(3)) // B1_FILIAL + B1_DESC
714         SB1->(DbSeek(xFilial("SB1"))) //Posiciona no primeiro registro
715
716         If SB1->(!EOF())
717             SB1->(DbGoTo(SB1->(LastRec())+1))
718             SB1->(DbSkip(-1))
719         EndIf
720
721         While SB1->(!BOF())
722
723             If SB1->B1_MSBLQL <> '1'
724                 //Aadd(aStartProds,{SB1->B1_DESC, SB1->(Recno())})
725                 Aadd(aStartProds,{ &(cPesq) , SB1->(Recno()) })
726             EndIf
727

```

```

728         SB1->(DbSkip(-1))
729
730     End
731
732     EndIf
733
734
735 EndIf
736
737
738 ConOut(cValToChar(len(aStartProds)))
739 nSec := Seconds()
740
741 ASort(aStartProds,,, { |x, y| x[1] < y[1] } )
742 nSec := Seconds()-nSec
743
744 RestArea(aArea)
745 RestArea(aAreaSX3)
746
747 Return
748
749
750 //-----
751 /*{Protheus.doc} STDIsPrdLike
752 Realiza a pesquisa por apenas parte do codigo do produto.
753
754 @param cCodProd - Codigo do Produto
755 @return Retorna array com os produtos encontrados
756 @obs
757 @sample
758 */
759 //-----
760 Function STDIsPrdLike(cCodProd)
761
762 Local aArea := GetArea() // Guarda a area
763 Local aRet := {} // Retorno da funcao
764
765 DEFAULT cCodProd := ""
766
767 cCodProd := AllTrim( cCodProd )
768
769 DbSelectArea("SB1")
770 SB1->(DbSetOrder(1))
771
772 If DbSeek(xFilial("SB1") + cCodProd)
773
774     While !EOF() .And. (xFilial("SB1") + cCodProd) == (SB1->B1_FILIAL + SubStr(SB1->B1_COD, 1, Len(cCodProd)))
775         AAdd(aRet, SB1->(Recno()))
776         SB1->(DbSkip())
777     End
778
779 EndIf

```

```

780
781 RestArea( aArea )
782
783 Return aRet
784
785
786 //-----
787 /*{Protheus.doc} STDJobCPrd
788 Funcao para iniciar um JOB para que nele gere as informacoes da
789 funcao STDStartProd
790 @param cEmpAnt - Empresa
791 @param cFilAnt - Filial
792 @return Nil
793 @obs
794 @sample
795 */
796 //-----
797 Function STDJobCPrd(cEmpAnt, cFilAnt)
798
799 Local lJob := .F. //Usa Job
800
801 Default cEmpAnt := ""
802 Default cFilAnt := ""
803
804 If !Empty(cEmpAnt) .And. Type("cEmpAnt") == "U"
805     RpcSetType(3)
806     RpcSetEnv(cEmpAnt, cFilAnt)
807     lJob := .T.
808 EndIf
809
810 STDStartProd()
811
812 If lJob
813     Reset Environment
814 EndIf
815
816 Return Nil
817
818
819 //-----
820 /*{Protheus.doc} STDConcPesq
821 Funcao que busca todos os campos que serao usados para concatenar uma
822 unica descricao para pesquisa da consulta de produto
823 @return cPesq - campos Adicionais da pesquisa de produto
824 @obs
825 @sample
826 */
827 //-----
828 Static Function STDConcPesq( )
829
830 //Parametro contendo os campos que serao concatenados ao B1_DESC para pesquisa.
831 //O separador dos campos sera o "|" (Pipe)

```



```

832 Local cCmpBscP      := SuperGetMV("MV_LJCM",,,)
833 Local cPesq         := ""           //Campos que serao usados para concatenar a pesquisa
834 Local nI            := 0           //Contador para laço
835 Local aCmpBscP      := {}          //Array para conter os campos personalizados para pesquisa
836 Local nTot          := 0           //Limitador do laço
837 Local aArea         := GetArea()   // Guarda a area
838
839 cPesq := "Alltrim(SB1->B1_DESC) + '|' "
840
841 //Verifica se existe algum campo que sera acrescentado a regra de busca
842 If !Empty(cCmpBscP)
843
844     //monta em array os campos personalizados a busca, a partir do limitador "|"
845     aCmpBscP := StrToKarr(cCmpBscP,"|")
846
847     //Analiso o array com os campos personalizados da busca para saber a quantidade de campos
848     If Len(aCmpBscP) > 2
849         nTot := 2
850     Else
851         nTot := Len(aCmpBscP)
852     EndIf
853
854     dbSelectArea("SX3")
855     dbSetorder(2)
856     For nI := 1 to nTot
857         //Valida se os campos sao validos
858         If SX3->( dbSeek( Alltrim(aCmpBscP[nI]) ) )
859             //Valida se pertence a tabela de produtos e se o tipo do campo eh caracter
860             If SX3->X3_ARQUIVO == "SB1" .And. SX3->X3_TIPO == "C"
861                 cPesq += " + Alltrim(SB1->" + Alltrim(SX3->X3_CAMPO) + ") + '|' "
862             EndIf
863         EndIf
864     Next nI
865
866 EndIf
867
868 RestArea( aArea )
869
870 Return cPesq
871
872 //-----
873 /*{Protheus.doc} STDAIqRed
874 Retorna a aliquota reduzida do campo SB0
875 */
876 //-----
877 Function STDAIqRed()
878
879 Local nAliq := 0
880
881 DbSelectArea("SB0")
882 SB0->(DbSetOrder(1)) // Fil + COD
883 If SB0->(DbSeek(xFilial("SB0")+ SB1->B1_COD))

```

```

884     nAliq := SB0->B0_ALIQRED
885 EndIf
886
887 Return nAliq
888
889 //-----
890 /*{Protheus.doc} STDVlitPAF
891 Validação dos itens especifica para o PAF
892
893 @param  aRet, array , contendo os dados do item
894 @param  lHomolPAF, lógico , tratamento especial quando em Homol
895 @return NIL, nenhum
896 */
897 //-----
898 Function STDVlitPAF(aRet,lHomolPAF, lGravaSB1)
899 Local cMD5Sb1 := ""
900 Local nPICM   := 0
901 Local nALIQUISS := 0
902 Local nRecnoSB1:= 0
903 Local lAlqIssIs:= .F.
904
905 Default lGravaSB1 := .T. // Indica se grava os campos B1_IAT e B1_IPPT da SB1
906
907 aRet[ITEM_SITTRIB] := STBFMSitTrib(SB1->B1_COD, "", "SB1")
908 lAlqIssIs := (aRet[ITEM_SITTRIB] $ "FS|NS|IS")
909
910 // Situacao tributaria
911 If lHomolPAF
912     //deve mostrar caso seja feita alteração no Banco de Dados - TESTE BLOCO VII
913     If !lAlqIssIs
914         nPICM := RetFldProd(SB1->B1_COD,"B1_PICM")
915         nPICM := Iif(Empty(nPICM), SB1->B1_PICM, nPICM)
916         If nPICM> 0
917             aRet[ITEM_SITTRIB] := AllTrim(aRet[ITEM_SITTRIB] ) + StrZero(nPICM, 5,2)
918         Else
919             nALIQUISS := RetFldProd(SB1->B1_COD,"B1_ALIQUISS")
920             nALIQUISS := Iif(Empty(nALIQUISS), SB1->B1_ALIQUISS, nALIQUISS)
921             aRet[ITEM_SITTRIB] := AllTrim(aRet[ITEM_SITTRIB] ) + StrZero(nALIQUISS, 5,2)
922         EndIf
923     EndIf
924 EndIf
925
926 If !lAlqIssIs .And. !(SubStr(aRet[ITEM_SITTRIB] ,1,1) == "T" .OR. SubStr(aRet[ITEM_SITTRIB] ,1,1) == "S")
927     aRet[ITEM_SITTRIB] := SubStr(aRet[ITEM_SITTRIB],1,1)
928 EndIf
929
930 If ( Empty( AllTrim( SB1->B1_IPPT )) .Or. Empty( AllTrim( SB1->B1_IAT )) ) .AND. lGravaSB1
931     Conout(" Campo B1_IPPT/B1_IAT em branco, por ser obrigatório no PAF-ECF preenchido com conteúdo padrão ")
932     LjGrvLog( NIL ," Campo B1_IPPT/B1_IAT em branco, por ser obrigatório no PAF-ECF preenchido com conteúdo padrão")
933
934     nRecnoSB1 := SB1->(Recno())
935

```

```

936   RecLock("SB1",.F.)
937
938   If Empty( AllTrim( SB1->B1_IPPT ))
939       REPLACE SB1->B1_IPPT WITH "T"
940   EndIf
941
942   REPLACE SB1->B1_IAT WITH IIf( SuperGetMV("MV_ARREFAT",,"N") == "S", "A", "T" )
943   SB1->(MsUnlock())
944
945   cMD5Sb1 := STxPafMd5("SB1")
946
947   RecLock("SB1",.F.)
948   REPLACE SB1->B1_PAFMD5 WITH cMD5Sb1
949   SB1->(MsUnlock())
950
951   SB1->(DbGoTo(nRecnoSB1))
952 EndIf
953
954 If lGravaSB1
955     aRet[ITEM_IAT] := SB1->B1_IAT
956     aRet[ITEM_IPPT] := SB1->B1_IPPT
957 Else
958     aRet[ITEM_IAT] := IIf( SuperGetMV("MV_ARREFAT",,"N") == "S", "A", "T" )
959     aRet[ITEM_IPPT] := "T"
960 EndIf
961
962 Return NIL
963
964 //-----
965 /*/{Protheus.doc} STDSearchP
966 Função responsável por realizar busca de um determinado Produto através do conteúdo informado na variável cWhatSearch
967
968 @param      cWhatSearch, Caracter, Conteúdo a ser buscado
969 @param      nLimitRegs, Numérico, Limite de produtos buscado
970 @param      cSearchFields, Caracter, Campos a serem utilizados na query para busca
971 @return     Array, Retorna informações sobre os produtos buscado
972 */
973 //-----
974
975 Function STDSearchP(cWhatSearch,nLimitRegs,cSearchFields)
976 Local cSGBD          := ""           // -- Banco de dados atualizado (Para ambientes TOP)
977 Local cAliasQuery    := ""           // -- Alias utilizado para query
978 Local cStartQuery    := ""           // -- Variável para montagem de Query
979 Local cBodyQuery     := ""           // -- Variável para montagem de Query
980 Local cEndQuery      := ""           // -- Variável para montagem de Query
981 Local cFullQuery     := ""           // -- Variável para montagem de Query
982 Local aProduct       := {}           // -- Variável utilizada para retorno com as informações dos produtos
983 Local nSearch        := 0            // -- Variável utilizada no Seek
984 Local cFilter        := ""           // -- Variável utilizada no Filtro
985 Local aSearchFields  := {}           // -- Variável utilizada na montagem dos campos que serão buscados
986 Local aOrderSeek     := {{1,"SB1->B1_COD"},{3,"SB1->B1_DESC"},{5,"SB1->B1_CODBAR"}}
987 Local nSeek          := 0

```

```

988 Local lFind          := .F.
989
990 Default cWhatSearch    := ""           // -- Parametro com a informação que será buscada
991 Default nLimitRegs     := 25           // -- Limite da busca
992 Default cSearchFields  := ""           //-- Campos default que serão buscados caso o parametro seja false
993
994 #IFDEF TOP
995
996   If Empty(cSearchFields)
997     cSearchFields := "B1_COD|B1_DESC"
998   EndIf
999
1000   cSGBD          := AllTrim(Upper(TcGetDb()))
1001   cAliasQuery     := GetNextAlias()
1002
1003   cStartQuery += " SELECT "
1004
1005   If cSGBD      $ "MSSQL|SYBASE"
1006     cStartQuery += " TOP " + AllTrim(Str(nLimitRegs))
1007   ElseIf cSGBD $ "INFORMIX"
1008     cStartQuery += " FIRST " + AllTrim(Str(nLimitRegs))
1009   EndIf
1010
1011   cStartQuery += " B1_DESC,B1_COD,B1_UM,B1_IAT,B1_IPPT, R_E_C_N_O_ , B1_TIPO"
1012
1013   cBodyQuery += " FROM " + RetSQLName("SB1") + " SB1 "
1014   cBodyQuery += " WHERE SB1.B1_FILIAL = '" + xFilial("SB1") + "' "
1015
1016   If cSGBD      $ "MSSQL"
1017     cSearchFields := Replace(cSearchFields,"|","")
1018     cBodyQuery    += " AND CONCAT(" + cSearchFields + ") LIKE '%" + cWhatSearch + "%' "
1019   Else
1020     cSearchFields := Replace(cSearchFields,"|","||")
1021     cBodyQuery    += " AND " + cSearchFields + " LIKE '%" + cWhatSearch + "%' "
1022   EndIf
1023
1024   cBodyQuery += " AND SB1.B1_MSBLQL <> '1' "
1025   cBodyQuery += " AND SB1.D_E_L_E_T_ = ' ' "
1026
1027   If cSGBD      $ "ORACLE"
1028     cBodyQuery += " AND ROWNUM <= " + AllTrim(Str(nLimitRegs))
1029   ElseIf cSGBD $ "DB2"
1030     cEndQuery  += " FETCH FIRST " + AllTrim(Str(nLimitRegs)) + " ROWS ONLY"
1031   ElseIf cSGBD $ "POSTGRES|MYSQL|SQLITE"
1032     cEndQuery  += " LIMIT " + AllTrim(Str(nLimitRegs))
1033   EndIf
1034
1035   cFullQuery := ChangeQuery(cStartQuery + cBodyQuery + cEndQuery)
1036
1037   DbUseArea(.T.,"TOPCONN",TcGenQry(,cFullQuery),cAliasQuery,.T.,.T.)
1038
1039   While (cAliasQuery)->(!EOF())

```

```

1040      aAdd(aProduct, {(cAliasQuery)->B1_DESC,(cAliasQuery)->B1_COD,(cAliasQuery)->B1_UM,(cAliasQuery)->B1_IAT,(cAliasQuery)->B1_IPPT,(cAliasQuery)->R_E_C_N_0_,(cAliasQuery)
->B1_TIP0})
1041      (cAliasQuery)->(DbSkip())
1042      End
1043
1044 #ELSE
1045
1046      DbSelectArea("SB1")
1047
1048      If Empty(cSearchFields)
1049          For nSeek := 1 To len(aOrderSeek)
1050              SB1->(DbSetOrder(aOrderSeek[nSeek][1]))
1051              If SB1->(dbSeek(xFilial("SB1") + cWhatSearch))
1052                  lFind := .T.
1053                  Exit
1054              EndIf
1055          Next
1056      Else
1057          If Empty(cSearchFields)
1058              cSearchFields := "B1_COD|B1_DESC"
1059          EndIf
1060
1061          // -- Montagem do Filtro
1062          aSearchFields := StrToKarr(cSearchFields,"|")
1063          cFilter := " ("
1064
1065          For nSearch := 1 To len(aSearchFields)
1066              cFilter += "'" + cWhatSearch + "' $ " + aSearchFields[nSearch]
1067              If len(aSearchFields) <> nSearch
1068                  cFilter += " .OR. "
1069              EndIf
1070          Next
1071
1072          cFilter += ") .AND. SB1->B1_MSBLQL <> '1' "
1073
1074          // -- Execução do Filtro
1075          SB1->(DbSetFilter({ || &cFilter }, cFilter))
1076          SB1->(DbGoTop())
1077          lFind := .T.
1078      EndIf
1079
1080      If lFind
1081          While SB1->(!EOF()) .AND. Len(aProduct) < nLimitRegs .AND. IIf(nSeek > 0, cWhatSearch $ &(aOrderSeek[nSeek][2]),.T.)
1082              aAdd(aProduct, {SB1->B1_DESC,SB1->B1_COD,SB1->B1_UM,SB1->B1_IAT,SB1->B1_IPPT,Recno(), SB1->B1_TIP0})
1083              SB1->(DbSkip())
1084          End
1085      EndIf
1086
1087      SB1->(DBCclearFilter())
1088
1089 #ENDIF
1090

```

1091 Return aProduct

Componentes/Item/STDKitSales.prw

85 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STDKitSales
6 Gera a lista de códigos de produtos que fazem parte do Produto Kit passado por parametro.
7 @param  cCodProdKit      Código do produto que sera analisado para ofertar outros
8 @return Vetor de codigos de produtos a serem sugeridos
9 @obs
10 @sample
11 */
12 //-----
13 Function STDKitSales(cCodProdKit)
14
15 Local aArea      := GetArea()    // Gurada area
16 Local aRetorno   := {}          // Retorno da funcao, vetor de codigos de produtos a serem sugeridos
17 Local nDesconto  := 0           // Caso o desconto esteja preenchida no cabecalho, esta variavel sera alimentada.
18
19 Default cCodProdKit := ""
20
21 ParamType 0 Var cCodProdKit As Character Default ""
22
23
24 DbSelectArea("MEU")
25 MEU->(DbSetOrder(1)) // MEU_FILIAL+MEU_CODIGO
26 If MEU->(DbSeek(xFilial("MEU") + PadR(cCodProdKit,TamSX3("MEU_CODIGO")[1])))
27     If !Empty(MEU->MEU_DESCNT) // Caso o desconto esteja preenchido no cabecalho, esse desconto sera utilizado.
28         nDesconto := MEU->MEU_DESCNT
29     EndIf
30 EndIf
31
32 DbSelectArea("MEV")
33 MEV->(DbSetOrder(1)) // MEV_FILIAL+MEV_CODKIT+MEV_PRODUT
34
35 If MEV->(DbSeek(xFilial("MEV") + PadR(cCodProdKit,TamSX3("MEV_CODKIT")[1])))
36     While !MEV->(Eof()) .AND. (AllTrim(MEV->MEV_CODKIT) == AllTrim(cCodProdKit))
37         AAdd(aRetorno, {MEV->MEV_PRODUT,IIF(!Empty(nDesconto),nDesconto,MEV->MEV_DESCNT),MEV->MEV_QTD})
38         MEV->(DbSkip())
39     End Do
40 EndIf
41
42 RestArea(aArea)
43
44 Return aRetorno
45
46 //-----
47 /*/{Protheus.doc} STDPrcKit
48 Rotina para retornar preço do Kit para consulta de produto
49 @type function
50 @param  cCodProdKit,  caractere ,   Código do produto kit
51 @param  cCustomer   ,  caractere ,   Código do cliente
52 @param  cFil        ,  caractere ,   Código da Filial

```

```

53 @param  cStore      , caractere ,  Código da Loja
54 @param  nQtde       , numérico  ,  Quantidade do Item
55 @param  nMoeda      , numérico  ,  Moeda
56 @return          numérico  ,  Valor do Kit
57 */
58 //-----
59 Function STDPrcKit(cCodProdKit, cCustomer, cFil, cStore, nMoeda, nQtde )
60
61 Local aItensKit      := {}
62 Local nI             := 0
63 Local nPrecoKit      := 0
64 Local nPrecoItem     := 0
65
66 Default cCodProdKit := ""
67 Default cCustomer  := ""
68 Default cFil       := ""
69 Default cStore     := ""
70 Default nMoeda     := 0
71 Default nQtde      := 1
72
73 aItensKit := STDKitSales(cCodProdKit)
74
75 If Len(aItensKit) > 0
76
77     For nI := 1 To Len(aItensKit)
78         nPrecoItem := STWFormPr( aItensKit[nI][1], cCustomer, cFil, cStore, nMoeda, nQtde )
79         nPrecoItem := STBarred((nPrecoItem - (nPrecoItem * aItensKit[nI][2])/100) * aItensKit[nI][3])
80         nPrecoKit += nPrecoItem
81     Next nI
82
83 EndIf
84
85 Return nPrecoKit

```


Componentes/Item/STWCancelItem.prw

490 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #include "TOTVS.CH"
4 #INCLUDE "FWEVENTVIEWCONSTS.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE "FWMVCDEF.CH"
7 #INCLUDE "STWCANCELITEM.CH"
8
9 //-----
10 /* {Protheus.doc} STWChkCancel
11 Avalia qual a forma de cancelamento.
12 Sendo possivel excluir qualquer item ou apenas o ultimo registrado.
13 @obs
14 @sample
15 */
16 //-----
17 Function STWChkCancel()
18
19 Local lRet          := .F.
20 Local nOpc          := STBCnAllItem()      // Opcao de cancelamento
21 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMv("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
22 Local lDesligaRD    := .F.                // Desabilita a Regra de Desconto do Varejo
23
24 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
25     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
    STDGPBasket('SL1','L1_LOJA'))
26     lCRdesItTt := !lDesligaRD
27 EndIf
28
29 If lCRdesItTt
30     PshClearPromo()
31 EndIf
32
33 If nOpc == 1
34     //Quando apenas o ultimo item puder ser cancelado
35     //e ele ja estiver cancelado, retorna a funcao de registro de item.
36     STFMessage(ProcName(),"STOP","O último item já está cancelado!")
37     STFShowMessage(ProcName())
38 ElseIf nOpc == 2
39     //Cancela o ultimo item
40     lRet := STIExchangePanel( { || STILastItCancel() } )
41     STIChangeCssBtn('oBtnCancItem')
42 ElseIf nOpc == 3
43     //Cancela item por parametro
44     lRet := STIExchangePanel( { || STIPanItCancel() } )
45     STIChangeCssBtn('oBtnCancItem')
46 EndIf
47
48 If lRet
49     STIRegItemInterface()

```

```

50 EndIf
51
52 STIGridCupRefresh()
53
54 Return
55
56
57 //-----
58 /*{Protheus.doc} STWItemCancel
59 Realiza o cancelamento do Item
60
61 @param  cGetProd      ,caracter  ,   Produto
62 @param  oReasons      ,object    ,   Objeto motivo venda perdida
63 @param  lCancel       ,lógico    ,   Indica que eh cancelamento para a pesquisa de produto, assim a pesquisa de produto nao pedira peso para produto balanca
64 @param  nItExcluir    ,numérico ,   Valor do campo L2_ITEM do registro a ser excluido (somente para PDV online)
65 @param  cGetRsnCanIt  ,caractere ,   Justificativa de cancelamento
66 @return Nil
67 @obs
68 @sample
69 */
70 //-----
71 Function STWItemCancel(cGetProd, oReasons, lCancel, nItExcluir, cGetRsnCanIt)
72
73 Local nItem          := 0
74 Local oModelCesta    := STDGPBModel()           // Model da cesta
75 Local lCancelado     := .F.                     // Item Cancelado?
76 Local aProfile       := STFProFile(7)           // Array de permissoes de cancelamento
77 Local nX             := 1                       // Variavel de controle
78 Local aItensCan      := {}                      // Armazena o(s) item(s) a sere(m) cancelado(s)
79 Local lMotVen        := SuperGetMV( "MV_LJMVPE", Nil, .F. ) // Motivo de venda perdida
80 Local lIsSelect      := .T.                     // Variavel que controla se o motivo para cancelamento esta selecionado
81 Local lStCancIt      := ExistBlock("STCancIt") //Verifica se existe o ponto de entrada StCancIt no cancelamento do item
82 Local lRetPe         := .T. //Retorno do ponto de entrada STCancIt
83
84 DEFAULT cGetProd     := ""
85 DEFAULT oReasons     := Nil
86 DEFAULT lCancel      := .F. // indica que eh cancelamento para a pesquisa de produto, assim a pesquisa de produto nao pedira peso para produto balanca
87 DEFAULT nItExcluir   := 0 // parametro criado para quando for cancelar importação de orçamento com pdv online e com item auto serviço.
88 DEFAULT cGetRsnCanIt:= "" // justificativa de cancelamento
89
90 If lMotVen .AND. ValType(oReasons) == "0"
91   If ExistFunc("STWldRsnInf")
92     lIsSelect := STWldRsnInf(oReasons,cGetRsnCanIt )
93   Endif
94 Endif
95
96 If lIsSelect
97   If !Empty(cGetProd)
98     If Empty(nItExcluir)
99       nItem := STBCnFindItem( cGetProd , lCancel)
100     else
101       nItem := Val(nItExcluir)

```

```

102     Endif
103     If nItem > 0
104         oModelCesta := oModelCesta:GetModel("SL2DETAIL")
105
106         //Verifico se existe itens ralacionado ao item que sera cancelado
107         If SL2->(ColumnPos("L2_IDITREL")) > 0 .AND. !Empty(STDGPBasket("SL2","L2_IDITREL",nItem))
108             If MsgYesNo(STR0005)//"0 Item selecionado para o cancelamento possui itens relacionados a ele, caso opte pelo cancelamento todos os itens relacionados serão
cancelados."
109                 aItensCan := STWListaRel(nItem)
110             Else
111                 aItensCan := {}
112             EndIf
113         Else
114             AADD( aItensCan, nItem )
115         EndIf
116
117         If !Empty(aItensCan)
118             //Verifica Permissao para Cancelamento Item
119             If aProfile[1]
120
121                 If lStCancIt
122                     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Antes de executar o ponto de entrada STCancIt")
123                     lRetPe := ExecBlock("STCancIt",.F.,.F.,{oModelCesta,aItensCan})
124                     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Depois de executar o ponto de entrada STCancIt")
125                     If !(ValType(lRetPe) == "L")
126                         LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "0 ponto de entrada STCancIt nao retornou um valor logico e por conta disso o item nao sera
cancelado")
127                         lRetPe := .F.
128                     EndIf
129                 EndIf
130
131                 If lRetPe
132                     For nX := 1 to Len(aItensCan)
133                         nItem := aItensCan[nX]
134                         If nItem > 0
135
136                             lCancelado := STWCancelProcess(nItem , oReasons , aProfile[2]) // aProfile[2] = usuario supervisor
137
138                             If lCancelado
139                                 LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Número do item a ser cancelado", nItem ) //Grava Log
140                                 LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "ID usuário superior", aProfile[2] ) //Grava Log
141                             EndIf
142
143                             If lCancelado .AND. oModelCesta:GetValue( "L2_BONIFICADOR", nItem )
144                                 STWCancelProcess(nItem+1 , oReasons)
145                             EndIf
146
147                         Else
148                             STFMessage("STCancelItem","STOP",STR0001) //"Item não registrado na venda"
149                             STFShowMessage("STCancelItem")

```

```

150             EndIf
151
152             If lCancelado
153                 STFMessage("STCancelItem","STOP",STR0002) //"Item cancelado com sucesso"
154                 STFShowMessage("STCancelItem")
155             Else
156                 STFMessage("STCancelItem","STOP",STR0003) //"Não foi possível cancelar o item"
157                 STFShowMessage("STCancelItem")
158             EndIf
159         Next
160     Else
161         LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Item nao foi cancelado devido ao retorno do ponto de entrada STCancIt")
162     EndIf
163 Else
164     //Se nao tem permissao para cancelar o item add mensagem
165     STFMessage("STCancelItem","STOP",STR0004) //Usuario sem permissão para cancelar itens"
166 EndIf
167 STFShowMessage("STCancelItem")
168 EndIf
169 Else
170     STFMessage("STCancelItem","STOP",STR0001) //"Item não registrado na venda"
171     STFShowMessage("STCancelItem")
172 Endif
173 EndIf
174
175 STIGridCupRefresh()
176 Endif
177
178 Return lCancelado
179
180
181 //-----
182 /*{Protheus.doc} STWLastCancel
183 Realiza o cancelamento do ultimo item registrado apenas, conforme a configuracao da impressora.
184
185 @param  nItem          Numero do item na venda
186 @param  cItemCode      Codigo do Item
187 @return Nil
188 @obs
189 @sample
190 */
191 //-----
192 Function STWLastCancel(lImportSale)
193
194 Local oModelCesta      := STDGPBModel()      // Model da cesta
195 Local nItem            := 0                  // Numero do item
196 Local lCancelado       := .F.                // Item Foi cancelado
197
198 Default lImportSale    := .F.                // Controla se houve importação de orçamento
199
200 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
201 oModelCesta:GoLine(oModelCesta:Length())

```

```

202
203 If !oModelCesta:IsDeleted()
204     nItem := Val(oModelCesta:GetValue("L2_ITEM"))
205
206     If !Empty(nItem)
207
208         lCancelado := STWCancelProcess(nItem,,lImportSale)
209         STIGridCupRefresh()
210
211         If lCancelado
212             STFMessage("STCancelItem","STOP",STR0002) //"Item cancelado com sucesso"
213         Else
214             STFMessage("STCancelItem","STOP",STR0003) //"Não foi possível cancelar o item"
215         EndIf
216     EndIf
217 EndIf
218
219 STFSHOWMESSAGE("STCancelItem")
220
221 Return lCancelado
222
223
224 //-----
225 /*{Protheus.doc} STWCancelProcess
226 Processo de cancelamento, realizado tanto quando e permitido cancelar qualquer item
227 quanto quando e permitido cancelar apenas o ultimo.
228
229 @param    nItem          Numero do item na venda
230 @param    oReasons       Objeto Motivo de venda perdida
231 @return   lRet - Retorna se cancelou item
232 @obs
233 @sample
234 */
235 //-----
236 Function STWCancelProcess( nItem , oReasons , cSuper, lImportSale)
237
238 Local lRet           := .T.           // Continua rotina?
239 Local cSupervisor    := ""            // Supervisor
240 Local aRet           := {}            // Array retorno
241 Local oModelMaster   := STDGPBModel()
242 Local oModelCesta    := STDGPBModel()
243 Local lFinServ       := SuperGetMv("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
244 Local lServFin       := .F.
245 Local lEmitNfCe      := Iif(FindFunction("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
246 Local lItFiscNFi     := .F.          //Verifica se existe o item fiscal e não fiscal para cancelamento no ECF (FieldPos)
247 Local aEstrItNFisc   := {}           //Estrutura do Contador do Item Fiscal (FieldPos)
248 Local aEstrItSF       := {}           //Estrutura do Contador Servico Financeiro (FieldPos)
249 Local uItem          := ""            //Item a ser cancelado
250 Local cItemAtu       := ""            //Item do produto deletado
251 Local nI              := 0            //Contador
252 Local cPrdCobe       := ""            //Produto Cobertura
253 Local cItemCob       := ""            //Item do Produto Cobertura

```

```

254 Local lLjLsPre      := SuperGetMv("MV_LJLSPRE",, .F.) //Funcionalidade de Lista de Presente Ativa
255 Local lLisPres      := .F.
256 Local lSaveOrc       := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
257 Local lItemFiscal    := .T. //Valida se item fiscal
258
259 ParamType 0 Var      nItem          As Numeric      Default 0
260
261 Default nItem        := 0
262 Default oReasons     := Nil
263 Default cSuper       := ""
264 Default lImportSale := .F. //Controla se houve importação de orçamento.
265
266 If !(lImportSale .AND. STBIsPAF()) //Se for orçamento importado e ambiente PAF, não pede a autorização novamente.
267   If Empty(cSuper)
268     cSupervisor := STFProFile(7)[2] // Supervisor, aqui chamara a tela de supervisor
269   Else
270     cSupervisor := cSuper // Supervisor vindo da rotina STWItemCancel.Protecao para nao chamar a tela supervisor duas vezes
271   EndIf
272 EndIf
273
274 oModelMaster := oModelMaster:GetModel("SL1MASTER")
275
276 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
277 oModelCesta:GoLine(nItem)
278
279 /*
280 Valida na regra de negocio se pode realizar o cancelamento
281 */
282 If lRet
283   lRet := STBValCnItem( nItem, oModelCesta )
284 EndIf
285
286 //Valida item fiscal
287 If lRet .AND. Empty( oModelCesta:GetValue("L2_ITFISC") )
288   lItemFiscal := .F.
289 EndIf
290
291 /* Verifica se produto Servico Financeiro avulso */
292 If lFinServ
293   If STBIsFinService(oModelCesta:GetValue("L2_PRODUTO"))
294     lServFin := .T.
295   EndIf
296 EndIf
297
298 /* Verifica se produto é da lista de presente */
299 If lLjLsPre
300   If !Empty(oModelCesta:GetValue("L2_CODLPRE")) .AND. AllTrim(oModelCesta:GetValue("L2_ENTREGA")) == "3"
301     lLisPres := .T.
302   EndIf
303   /* Subtrai valor nao fiscal no totalizador */
304   STBSubNotFiscal( oModelCesta:GetValue("L2_VRUNIT") )
305 EndIf

```

```

306
307 EndIf
308
309 If !lItemFiscal .AND. !lFinServ .AND. !lLisPres .AND. ExistFunc("STBSubNotFiscal") //Servico financeira e lista d e presente ja retira de outra forma
310     // Subtrai valor nao fiscal no totalizador
311     STBSubNotFiscal( oModelCesta:GetValue("L2_VRUNIT") )
312 EndIf
313
314 /*
315 Valida se e possivel cancelar todos ou so o ultimo item
316 */
317 If lRet .AND. !lServFin .AND. !lEmitNFCe .AND. !lLisPres .AND. !lSaveOrc .AND. lItemFiscal
318
319     //verifica se existe o item fiscal e não fiscal para cancelamento de item no ECF
320     aEstrItNFisc := STDGetProperty( "L2_ITFISC" )
321
322     lItFiscNFfi := Len(aEstrItNFisc) > 0
323
324     //verifica se existe o item Servico Financeiro para cancelamento de item no ECF
325     aEstrItSF := STDGetProperty( "L2_ITEMREA" )
326
327     //Compatibilizado para cancelar item maior que 99
328     If lFinServ .And. Len(aEstrItSF) > 0
329         uItem := STBPegaIT(oModelCesta:GetValue("L2_ITEMREA"))
330     ElseIf !lItFiscNFfi
331         uItem := STBPegaIT(oModelCesta:GetValue("L2_ITEM"))
332     Else
333         uItem := STBPegaIT(oModelCesta:GetValue("L2_ITFISC"))
334     EndIf
335
336     If ValType(uItem) <> "N" .OR. uItem = 0
337         uItem := Val(oModelCesta:GetValue("L2_ITEM"))
338     EndIf
339
340     // Inicia Evento
341     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
342                          "STCancelItem" ,; // Nome do evento
343                          {AllTrim(Str(uItem)) ,; // 01 - Numero do Item
344                          AllTrim(oModelCesta:GetValue("L2_PRODUTO")) ,; // 02 - Codigo do Item
345                          AllTrim(oModelCesta:GetValue("L2_DESCRI")) ,; // 03 - Descricao do Item
346                          StrZero(oModelCesta:GetValue("L2_QUANT"),8,3) ,; // 04 - Quantidade do Item
347                          AllTrim(Str(oModelCesta:GetValue("L2_VRUNIT")) ,; // 05 - Valor do Item
348                          AllTrim(Str(oModelCesta:GetValue("L2_VALDESC")) ,; // 06 - Valor Desconto
349                          AllTrim(oModelCesta:GetValue("L2_SITTRIB")) ,; // 07 - Situacao tributaria do Item
350                          AllTrim(cSupervisor) ,; // 08 - Supervisor
351                          Nil })
352
353     lRet := ValType(aRet[1]) == "U" .OR. (ValType(aRet[1]) == "N" .AND. aRet[1] == 0)
354
355 EndIf
356
357 /*

```

```

358     Exclui o item nas funcoes fiscais e dependentes
359 */
360 If lRet
361
362     // Limpa Motivo de desconto caso exista
363     STDDelReason( nItem )
364
365     //STFLogCanc( "cSupervisor" , nItem ) // TODO: Log nao subira na 1 fase
366
367     oModelCesta:LoadValue("L2_SITUA","05")
368     oModelCesta:LoadValue("L2_VENDIDO","N")
369     oModelCesta>DeleteLine(Nil ,.T.)
370
371     // Deleta Item das funcoes fiscais
372     STBTaxDel( nItem , .T. )
373
374     // Chamada Motivo de Venda Perdida
375     STWRsnLtSl( nItem , oReasons )
376
377     // Salva a venda (tabelas SL1 e SL2)
378     STDSaveSale(nItem,.T.)
379
380     // Se um item for cancelado, o Caixa poderá alterar as parcelas, independentemente da permissão, já que o valor da venda foi alterado
381     If FindFunction("STISetPayR0")
382         STISetPayR0(-1) //permite editar os pagamentos
383     EndIf
384
385     //Gera SLX
386     STDLogCanc(.T., nItem, cSuper, lImportSale)
387 EndIf
388
389 */
390 Marca item como deletado em Servicos Financeiros
391 */
392 If lFinServ .AND. lRet
393     /* Verifica se Cliente Padrao */
394     If STWValidService( 3,, oModelMaster:GetValue("L1_CLIENTE"), oModelMaster:GetValue("L1_LOJA") )
395         /* Se item Servico Financeiro atualiza totalizador */
396         If lServFin
397             /* Subtrai valor nao fiscal no totalizador */
398             STBSubNotFiscal( oModelCesta:GetValue("L2_VRUNIT") )
399
400             /* Marca itens Servico Financeiro como deletados */
401             STBDelServFin(oModelCesta:GetValue("L2_PRODUTO"), nItem, lServFin)
402         Else
403             /* Deleta Servicos Financeiros vinculados se existirem*/
404             cItemAtu := oModelCesta:GetValue("L2_ITEM")
405             cPrdCobe := oModelCesta:GetValue("L2_PRODUTO")
406
407             For nI := 1 To oModelCesta:Length()
408                 If nI <> nItem
409                     oModelCesta:GoLine(nI)

```



```

410
411      cItemCob := Posicione("SL2", 1, xFilial("SL2") + oModelCesta:GetValue("L2_NUM") + oModelCesta:GetValue("L2_ITEM") + oModelCesta:GetValue("L2_PRODUTO"),
      "L2_ITEMCOB")
412
413      If cItemCob == cItemAtu .And. !oModelCesta:IsDeleted()
414          // Limpa Motivo de desconto caso exista
415          STDDelReason( nI )
416
417          oModelCesta:LoadValue("L2_SITUA","05")
418          oModelCesta:LoadValue("L2_VENDIDO","N")
419          oModelCesta:DeleteLine(Nil ,.T.)
420
421          // Deleta Item das funcoes fiscais
422          STBTaxDel( nI , .T. )
423
424          // Chamada Motivo de Venda Perdida
425          STWRsnLtSl( nI , oReasons )
426
427          // Salva a venda (tabelas SL1 e SL2)
428          STDSaveSale(nI,.T.)
429
430          /* Subtrai valor nao fiscal no totalizador */
431          STBSubNotFiscal( oModelCesta:GetValue("L2_VRUNIT") )
432
433          /* Marca itens Servico Financeiro como deletados */
434          STBDelServFin(cPrdCobe, nItem, lServFin)
435      EndIf
436  EndIf
437  Next nI
438
439      oModelCesta:GoLine(nItem)
440  EndIf
441 EndIf
442 EndIf
443
444 Return lRet
445
446
447 //-----
448 /*{Protheus.doc} STWListaRel
449 Esta função retorna os itens relacionados ao produto que será cancelado, exemplo o KIT, todos os itens do kit são relacionados entre si.
450
451 Exemplo pratico
452 Ex.: Uma venda de um kit que possui mais de um produto, e por vender um kit fechado aplico um desconto sobre esse kit.
453 Os itens da venda do Kit são relacionados através do campo L2_IDITREL, com isso caso o operador tente cancelar apenas um dos itens, os demais relacionados ao Kit também serão
    cancelados.
454
455 @param      nItem          Numero do item na venda
456 @return     aItensRel - Retorna um array com os itens relacionados ao produto que deseja cancelar
457 @obs
458 @sample
459 */

```

```

460 //-----
461 Static Function STWListaRel(nItem)
462
463 local nX           := 1           // Variavel de controle para For
464 local nItRelIni    := 0           // Variavel que armazena o primeiro item relacionado.
465 local aItensRel    := {}          // Array que armazena os itens que serão cancelados/retornados
466 Local oModelCesta := STDGPBModel() // Model da cesta
467
468 Default nItem := 0
469
470 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
471
472 //Localizo o primeiro item do conjunto relacionado
473 For nX := nItem to 1 Step -1
474     If STDGPBasket("SL2","L2_IDITREL",nX) == STDGPBasket("SL2","L2_IDITREL",nItem)
475         nItRelIni := Val(STDGPBasket("SL2","L2_ITEM",nX))
476     Else
477         Exit
478     EndIf
479 Next nX
480
481 //A partir do primeiro salvos os próximos itens relacionados
482 For nX := nItRelIni to oModelCesta:Length()
483     If STDGPBasket("SL2","L2_IDITREL",nX) == STDGPBasket("SL2","L2_IDITREL",nItem)
484         AADD( aItensRel, nX )
485     Else
486         Exit
487     EndIf
488 Next nX
489
490 Return aItensRel

```

Componentes/Item/STWFindItem.prw

175 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWFindItem.CH"
5
6 //-----
7 /*{Protheus.doc} STWFindItem
8 Function Busca Item
9
10 @param  cCodeReceived      Codigo do Item
11 @return  aRet              Array com informações do Item
12 @obs
13 @sample
14 */
15 //-----
16
17 Function STWFindItem( cCodeReceived , lPAFECF, lHomolPaf, lOnlyCodBar)
18
19 Local aRet          := {}           // Retorno
20 Local aRetStry      := {}           // Retorno Balanca
21 Local aRetDtBar     := {}           // Retorno do leitor Optico
22 Local dDtValid      := ""           // Data validade do produto
23 Local cCodProd       := ""           // Codigo Produto
24 Local lShowMsgBlock := SuperGetMV("MV_LJITMSG",,.F.) //Contra se mostra mensagem de item não encontrado ou bloqueado
25 Local nTamBIUM       := TamSX3("B1_UM")[1]
26 Local cUnidMed       := PadR("G", nTamBIUM) + "|" + PadR("MG", nTamBIUM) + "|" + PadR("KG", nTamBIUM)
27
28 Default cCodeReceived := ""           // Codigo recebido/digitado
29 Default lPAFECF       := StbIsPaf()
30 Default lHomolPaf     := STBHomolPaf()//Homologação
31 Default lOnlyCodBar   := .F.
32
33 ParamType 0 Var cCodeReceived AS Character Default ""
34
35 /*
36     Caso Trabalhe com seperador de digito:
37     Pega o Codigo recebido/digitado e separa em Codigo do Item
38     e quantidade do Item, conforme configuracao do parameto
39 */
40 cCodeReceived := STBDigItem(cCodeReceived)
41
42 //Para otimizar pesquisa, realiza primeira a pesquisa de codigo de barras
43 aRet := STDFItemCB( cCodeReceived,lPAFECF,lHomolPaf )
44
45 /*
46     Procura Item na tabela de produtos
47 */
48 If !aRet[ITEM_ENCONTRADO] .AND. !lOnlyCodBar
49     aRet := STDFindItem( cCodeReceived, lPAFECF, lHomolPaf)
50 EndIf
51
52 //Recno do Produto na tabela SBI

```

```

53 If aRet[ITEM_ENCONTRADO]
54     SB1->(DbGoto(aRet[ITEM_RECNO]))
55 EndIf
56
57 If !IsInCallStack("STBCONFPAY") .AND. aRet[ITEM_ENCONTRADO] .AND. aRet[ITEM_BALANCA] == "2" .AND. aRet[ITEM_UNID_MEDIDA] $ cUnidMed .And. !Empty(STFGetStation("BALANCA"))
    .And. Empty(STDGPBasket("SL1", "L1_NUMORIG")) // 07 - Tipo produto de balanca
58 /*
59     Verifica se usa balanca para pegar peso - ( Quantidade = Peso )
60 */
61 aRetStry := STFFireEvent( ProcName(0) ,; // Nome do processo
62     "STBalanceUse" ,; // Nome do evento
63     {.T.} ) // 06 - Define se usa a balaça
64
65 //Essa verificação é feita para considerar se deve pegar a quantidade da balança (quantidade = 1 é o default, ou seja, a quantidade deve ser retornada da balança)
66 If STBGetQuant() == 1
67     If ValType(aRetStry) == "A" .AND. Len(aRetStry) > 0 .OR. ValType(aRetStry[1]) == "L" .AND. aRetStry[1]
68         // Pega quantidade da balanca
69         STBBalQuant()
70     EndIf
71 EndIf
72 EndIf
73
74
75 /*
76     Procura Item na tabela de codigo de barras
77 */
78 If !aRet[ITEM_ENCONTRADO]
79     aRet := STDFItemCB( cCodeReceived,lPAFECEf, lHomolPaf )
80 EndIf
81
82 /*
83     Ve se e item de Etiqueta/Balanca
84 */
85 If !aRet[ITEM_ENCONTRADO]
86     aRet := STDFItemBal( cCodeReceived )
87 EndIf
88
89 If !aRet[ITEM_ENCONTRADO] .AND. SuperGetMv("MV_LJDTBAR",,.F.)
90     aRetDtBar := STBDataBar(cCodeReceived)
91     cCodProd := aRetDtBar[1][1]
92     dDtValid := aRetDtBar[2][1]
93     aRet := STDFItemCB( cCodProd )
94
95     If !aRet[ITEM_ENCONTRADO]
96         aRet := STDFindItem( cCodProd )
97     EndIf
98
99     If aRet[ITEM_ENCONTRADO]
100
101         If !Empty(dDtValid) .AND. dDtValid <= dDatabase
102             // Caso Retorne que o produto esta vencido apresenta uma mensagem informando que nao pode ser vendido
103             STFMessage("ItemRegistered","POPUP",STR0004) //"Venda não permitida. Data de validade do produto inválida"

```

```

104         STFS showMessage("ItemRegistered")
105         aRet[ITEM_ENCONTRADO] := .F.
106     Else
107         cInfProBal:= SubStr(cCodProd,8,5)
108     EndIf
109
110 EndIf
111
112 EndIf
113
114 /*
115     Se encontrou Item e encontrou Qtde > 1 Altera Qtde
116 */
117 If aRet[ITEM_ENCONTRADO] .AND. aRet[ITEM_QTDE] > 0
118
119     STBSetQuant(aRet[ITEM_QTDE])
120
121 EndIf
122
123 /*
124     Se nao achou o Item ou esta Bloqueado add mensagem
125 */
126
127 If !aRet[ITEM_ENCONTRADO]
128     STFMessage("FindItem","STOP",STR0001)    //"Item nao encontrado."
129     If lShowMsgBlock
130         If !(IsInCallStack( 'STISearchProd' ) .Or. IsInCallStack( 'STBSearchPrice' ))
131             STIBtnDeActivate()
132             STWMsgNoBut(STR0001)
133             STIBtnActivate()
134         EndIf
135     EndIf
136 EndIf
137
138 If aRet[ITEM_BLOQUEADO]
139     STFMessage("FindItem","STOP",STR0002 + aRet[ITEM_CODIGO] + STR0003)    //"Item: " //" Bloqueado."
140     If lShowMsgBlock
141         STIBtnDeActivate()
142         STWMsgNoBut(STR0002 + aRet[ITEM_CODIGO] + STR0003)
143         STIBtnActivate()
144     EndIf
145 EndIf
146
147 Return aRet
148
149
150 //-----
151 /*/{Protheus.doc} STWMSGNOBUT
152 Tela de mensagem do item não localizado
153
154 @param   cMsg           Mensagem mostrada
155 @return

```

```
156 @obs
157 @sample
158 /*/
159 //-----
160 Function STWMsgNoBut(cMsg)
161
162 Local oDlgIt := Nil //Obj tela
163 Local oEditIt := Nil //Objeto para texto apresentada na tela
164
165 DEFINE FONT oFontIt NAME "Courier New" SIZE 14,32
166
167 DEFINE MSDIALOG oDlgIt TITLE "" FROM 000, 000 TO 110, 500 COLORS 0, 16777215 PIXEL
168
169 oEditIt:= TSimpleEditor():New(005,005,oDlgIt,240,35,,.T.,,oFontIt,.T.)
170 oEditIt:TextFormat(2)
171 oEditIt:Load(Space(2)+cMsg+" (ESC)")
172
173 ACTIVATE MSDIALOG oDlgIt CENTERED
174
175 Return
```

Componentes/Item/STWItemClustered.prw

31 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4
5 //-----
6 /* {Protheus.doc} STWItemClustered
7 Efetua os processamentos para produtos agrupados. Chamado durante o processamento do STWItemReg.
8
9 @param cItemCode      Quantidade do Item
10 @return lRet - Executou corretamente
11 @obs
12 @sample
13 */
14 //-----
15
16 Function STWItemClustered(aInfoItem)
17
18 Local cItemCode      := aInfoItem[ITEM_CODIGO] // Codigo do produto
19 Local cTipoProd      := aInfoItem[ITEM_TIPO]   // Tipo do produto
20 Local lRet           := .T.                   // Retorno da Funcao
21
22 Default aInfoItem := {}
23
24 ParamType 0 Var aInfoItem As Array Default {}
25
26 If cTipoProd == "KT"
27     lRet := .F.
28     STWKitSales(cItemCode)
29 EndIf
30
31 Return lRet

```

Componentes/Item/STWItemRegistry.prw

967 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWITEMREGISTRY.CH"
5
6 Static lReceiptIsOpen := .F. // Define se foi realizada abertura de cupom fiscal
7
8 Static cImpCodBar := SuperGetMv("MV_CODBAR",,"N") // Indica se imprime codigo de barras no cupom ao inves do codigo do produto
9 Static lStValPro := ExistBlock("STVALPRO")
10 Static lLj7013 := ExistBlock("LJ7013") // Permite customizar informacoes na impressao do comprovante
11 Static lSumItFisc := ExistFunc("STBSumItem") // Função de Soma do Contador
12 Static lMobile := IIF(STFGetCfg("lMobile", .F.) == Nil,.F.,STFGetCfg("lMobile", .F.)) //Valida versao mobile
13 Static lFinServ := SuperGetMv("MV_LJCSF",,.F.) // Define se habilita o controle de servicos financeiros
14 Static lItemDel := .F. // Define o controle de item deletado da MatxFis
15 Static lMVLJPRDSV := SuperGetMv("MV_LJPRDSV",.F.,.F.) // Verifica se esta ativa a implementacao de venda com itens de "produto" e itens de "servico" em Notas
    Separadas (RPS)
16 Static cMvLjTGar := SuperGetMV("MV_LJTGAR",,"GE") // Define se é tipo GE
17 Static lStQuant := ExistBlock("STQUANT")
18 Static lFuncGarEst := ExistFunc("STBIsGarEst") .AND. ExistFunc("STDGEProdVin") .AND. ExistFunc("STWItemGarEst")
19 Static lFunGiftCar := ExistFunc("STBIsGiftCard")
20 Static lFLImpItem := ExistFunc("STBLImpItem")
21 Static l950SP100K := ExistFunc("Lj950SP100K")
22
23 //-----
24 /* {Protheus.doc} STWItemReg
25 Function Registra Item.
26
27 @param nItemLine Linha do Item na Venda
28 @param cItemCode Codigo do Item
29 @param cCliCode Codigo do Cliente
30 @param cCliLoja Codigo da loja do Cliente
31 @param nMoeda Moeda Corrente
32 @param nDiscount Desconto no Produto
33 @param cTypeDesc Tipo de Desconto
34 @param lAddItem Indica se o item a ser registrado eh um item adicional
35 @param cItemTES TES que sera usado no calculo do imposto do Item
36 @param cCliType Tipo do Cliente
37 @param lItemFiscal Indica se o Item eh do tipo fiscal. Registra no cupom fiscal?
38 @param nPrice Preço do Item
39 @param cTypeItem Temos 2 tipos: "IMP" - Indica que o item vem de importação de orçamento;
40 "KIT" - Indica que o item eh um dos filhos de um kit
41 @param lInfoCNPJ Informa se o CPF/CNPJ deve ser informado na nota fiscal
42 @param lRecovery Informa se esta sendo chamado pela recuperacao de venda.
43 @param nSecItem Intervalo entre itens.
44 @param lServFinal Indica finalização dos serviços na Venda.
45 @return Nil
46 @obs
47 @sample
48 */
49 //-----
50
51 Function STWItemReg( nItemLine , cItemCode , cCliCode , cCliLoja ,;

```



```

52      nMoeda      , nDiscount      , cTypeDesc      , lAddItem      ;
53      cItemTES    , cCliType      , lItemFiscal    , nPrice        ;
54      cTypeItem   , lInfoCNPJ      , lRecovery      , nSecItem      ;
55      lServFinal  , lProdBonif     , lListProd      , cCodList      ;
56      cCodListIt  , cCodMens       , cEntrega       , cCodItemGar   ;
57      cIdItRel    , cValePre       , cCodProdKit    )
58
59
60 Local lKitMaster := .F. // Utiliza kit master
61 Local aInfoItem  := {} // Array de retorno da busca de item
62 Local lRet       := .T. // Retorno
63 Local cCodIniIT  := cItemCode // Armazena valor original doCodigo do Item Recebido/Digitado
64 Local cCodeForPrint := "" // Armazena Codigo que sera usado para impressao do Item
65 Local nItemTotal := 0 // Armazena Preço * Quantidade
66 Local lRecShopCard := .F. // Indica que é um produto do tipo Recarga de Shop Card
67 Local lItemServFin := .F. // Define ao serviço financeiro item NAO fiscal caso for avulso
68 Local aAlqLeiTr := {0,0,0,0,} // array utilizado para pegar as aliquotas das lei dos impostos
69 Local oTotal := STFGetTot() // Totalizador
70 Local cMsgErro := STR0003 // "Nao foi possivel a abertura do Cupom Fiscal"
71 Local oModel := iIf(ExistFunc("STDGPBModel"),STDGPBModel(),Nil) // Utilizada na busca de produtos na cesta de venda
72 Local nVTotAfter := 0 // Valor total antes de passar pela rotina de calculo de imposto
73 Local cDscPrdPAF := "" // Descrição do produto segundo legislação do PAF CONVÊNIO ICMS 25, DE 8 DE ABRIL DE 2016
74 Local cITcEst := "" // cEst retornado da MatxFis
75 Local cPosIpi := "" // código NCM do produto
76 Local cL1Num := "" // Numero do orcamento da venda em lancamento
77 Local aSTQuant := {} // Array retorno do ponto de entrada STQUANT
78 Local nTamArray := 0
79 Local lItemGarEst := .F. // Define a Garantia Estendida item NAO fiscal caso for avulso
80 Local cProdVend := "" // Produto que será vinculado à Garantia Estendida
81 Local nPrProd := 0 // Preço do produto que será impresso no Cupom Gerencial de Garantia Estendida
82 Local cSerieProd := "" // Série do produto que será impresso no Cupom Gerencial de Garantia Estendida
83 Local aRetLj7013 := {} // Retorno do PE Lj7013 para ser adicionado a descricao do produto
84 Local lEmitNFCE := STBGetNFCE() //valida se é NFC-e ou não
85 Local lUseSat := STFGetCfg("lUseSAT", .F.) //Utiliza SAT
86
87 Local cQuant := ""
88 Local cVrUnit := ""
89 Local cDesconto := ""
90 Local cSitTrib := ""
91 Local cVlrItem := ""
92 Local lIsItemRPS := .F. // Indica se é item de serviço (RPS)
93 Local lCRdesItTt := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMv("MV_LJCRDPT",,"0") == "1" .AND.
FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
final da venda
94 Local cTabPad := SuperGetMv("MV_TABPAD") // Tabela de preço utilizada
95 Local lDesligaRD := .F. // Desabilita a Regra de Desconto do Varejo
96 Local lItImpRDPT := .F. // Item importado com desconto da Regra de Desconto Varejo por Produto no Total
97
98 Default nItemLine := 0 // Linha do Item na Venda
99 Default cItemCode := "" // Codigo do Item
100 Default cCliCode := "" // Codigo do Clientep
101 Default cCliLoja := "" // Codigo da loja do Cliente

```

```

102 Default nMoeda      := 0           // Moeda Corrente
103 Default nDiscount   := 0           // Desconto no Produto
104 Default cTypeDesc    := ""          // Tipo de Desconto
105 Default lAddItem     := .F.         // Indica se o item a ser registrado eh um item adicional
106 Default cItemTES     := ""          // TES que sera usado no calculo do imposto do Item
107 Default cCliType     := ""          // Tipo do Cliente
108 Default lItemFiscal  := .T.         // Indica se o Item eh do tipo fiscal. Registra no cupom fiscal?
109 Default nPrice       := 0           // Preco do item
110 Default cTypeItem    := ""          // IMP - Indica que o item vem de importação de orçamento; KIT - Indica que eh um dos
    filhos de um kit.
111 Default lInfoCNPJ    := .T.         // Imprime CNPJ no cupom Fiscal?
112 Default lRecovery    := .F.         // A tela do POS está sendo apresentada?
113 Default nSecItem     := 0           // Intervalo entre itens
114 Default lServFinal   := .F.         // Indica finalização dos serviços na Venda.
115 Default lProdBonif   := .F.         // Identificacao se o produto é bonificado
116 Default lListProd    := .F.         // Identifica se o item é lista
117 Default cCodList     := ""          // Código da Lista;
118 Default cCodListIt   := ""          // Código do Item de Lista
119 Default cCodMens      := ""          // Código da Mensagem
120 Default cEntrega     := ""          // Entrega
121 Default cCodItemGar   := ""          // Item de Produto com Garantia Vinculada
122 Default cIdItRel     := ""          // Id do item Relacionado
123 Default cValePre     := ""          // Vale Presente, se houver
124 Default cCodProdKit  := ""          // Código do kit de produto
125
126 //Todo: Implementar na importação do DAV para cancelamento o parâmetro como .f. //PAF: Sera implemetado na segunda fase
127 LjGrvLog("Registra_Item", "ID_INICIO")
128
129 cL1Num := "L1_Num:"+STDGPBasket('SL1','L1_NUM')
130
131 LjGrvLog(cL1Num,"Inicio - Workflow Registra Item.Codigo do Item:" + cItemCode + " Tipo:"+cTypeItem )
132
133 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
134     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
        STDGPBasket('SL1','L1_LOJA'))
135 EndIf
136
137 If !lRecovery
138     STFMessage("ItemRegistered","STOP",STR0007) //Registrando Item...
139     STFShowMessage("ItemRegistered")
140 EndIf
141
142 //verifica se o item Fiscal/Não Fiscal Existe na Cesta
143 lSumItFisc := lSumItFisc .AND. Len(STDGetProperty( "L2_ITFISC" )) > 0
144
145 /*/
146     Busca item na base de dados
147 /*/
148 aInfoItem := STWFindItem( cItemCode , STBIsPaf() , STBHomolPaf() )
149
150 // Encontrou o item?
151 If aInfoItem[ITEM_ENCONTRADO] .AND. !aInfoItem[ITEM_BLOQUEADO]

```

```

152
153 LjGrvLog( cL1Num, "Registra Item - Item encontrado" ) //Gera LOG
154 //?P.E. Para Modificar a Quantidade e Valor Unitario
155
156 // Arredondamento
157 nItemTotal := STBArred( nPrice * STBGetQuant() )
158
159 If lStQuant
160   LjGrvLog(cL1Num,"Antes da Chamada do Ponto de Entrada:STQUANT",{aInfoItem[ITEM_CODIGO], STBGetQuant() } )
161   aSTQuant := ExecBlock( "STQUANT",.F.,.F.,{STBGetQuant(),nPrice,nItemTotal,aInfoItem[ITEM_CODIGO] } )
162   LjGrvLog(cL1Num,"Apos a Chamada do Ponto de Entrada:STQUANT. Retorno:", aSTQuant )
163
164   If ValType(aSTQuant) == "A" .AND. Len(aSTQuant) >= 2
165     STBSetQuant(aSTQuant[1])
166     nPrice := aSTQuant[2]
167     If Len(aSTQuant) == 3
168       aInfoItem[ITEM_CODIGO] := aSTQuant[3]
169     Endif
170   Else
171     //?Caso o Retorno do lStQuant Nao Seja Array,      ?
172     //?Eh Interpretado Que Devera Abandonar a Validacao ?
173     //?Voltando Para o Get.                            ?
174     lRet := .F.
175   EndIf
176
177   If !lRet
178     LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Ponto de Entrada STQUANT não retornou array ")
179   EndIf
180
181 EndIf
182 //Valida quantidade
183 //Limite de qtde de 9999.99 somente para ECF
184 If lRet .AND. ((!(lEmitNFce .OR. lUseSat) .AND.STBGetQuant() > 9999.99 ) .OR. STBGetQuant() == 0)
185   LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Quantidade informada não atende o limite permitido (mínimo = 1, máximo = 9999.99). Quantidade
Informada:",STBGetQuant())
186   STFMessage("ItemRegistered","STOP",STR0008) //Quantidade inválida.
187   lRet := .F.
188 EndIf
189
190 If lRet .And. lStValPro .And. !lRecovery
191   LjGrvLog( Nil, "Ponto de Entrada STVALPRO Existe. Parametros:", {aInfoItem[ITEM_CODIGO], STBGetQuant() } )
192   lRet := ExecBlock( "STVALPRO",.F.,.F.,{aInfoItem[ITEM_CODIGO], STBGetQuant() } )
193   LjGrvLog( Nil, "Ponto de Entrada STVALPRO. Retorno <- ", lRet )
194 EndIf
195
196 //Numero maximo de itens por venda
197 If lRet
198   If nItemLine > 990
199     LjGrvLog("L1_NUM: "+STDGPBasket('SL1','L1_NUM'),'','',"Quantidade de itens lançados na venda ultrapassa o maximo de itens permitido no grid: " +
AllTrim(Str(nItemLine)))
200     STFMessage("ItemRegistered","STOP",STR0030) //"Atingido numero maximo de itens por venda. Efetue nova venda."
201     lRet := .F.

```

```

202     EndIf
203 EndIf
204
205 //Identifica se o item encontrado eh um Item Agrupador,
206 //onde somente seus filhos serao inclusos na venda.
207 If lRet
208     lRet := STWItemClustered(aInfoItem)
209     If !lRet
210         LjGrvLog(cL1Num,"Item de KIT (B1_TIPO = KT)")
211         lKitMaster := .T.
212     EndIf
213 EndIf
214
215 If lRet .AND. nPrice == 0
216     LjGrvLog(cL1Num,"Item sem Valor, sera verificado se item de Recarga (STWShopCardRecharge)")
217     lRecShopCard := STWShopCardRecharge(aInfoItem[ITEM_CODIGO])
218     LjGrvLog(cL1Num,"Retorno da verificacao se Item de Recarga",lRecShopCard)
219
220     If lRecShopCard
221         LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Item nao possui preco e nao e item de recarga")
222         lRet := .F.
223     EndIf
224 EndIf
225
226 //So deixa vender com caixa aberto
227 If lRet
228     lRet := STBCaixaVld()
229
230     If !lRet
231         LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Caixa Fechado")
232     EndIf
233 EndIf
234
235 /*/
236     Validações específicas para Garantia Estendida
237     Nunca serao impressos, apenas armazenados em array para serem apresentados no final da venda
238     Não será tratado a garantia quando estiver na finalização da venda.
239 /*/
240 If lRet .AND. !lKitMaster .AND. (aInfoItem[ITEM_TIPO] == cMvLjTGar) .AND. !lServFinal
241
242     LjGrvLog(cL1Num,"Realiza validacoes para Item de Garantia Estendida")
243
244     If !lFuncGarEst
245         //se tentou vender item de garantia, avisa que precisa atualizar fontes
246         LjGrvLog(cL1Num,STR0033+STR0032) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado! Motivo: "####Favor incluir os fontes
STBEXTWARRANTY/STDEXTWARRANTY/STWEXTWARRANT para validar a Garantia Estendida."
247         STFMessage("ItemRegistered","STOPPOPUP",STR0033+STR0032) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado! Motivo: "####Favor
incluir os fontes STBEXTWARRANTY/STDEXTWARRANTY/STWEXTWARRANT para validar a Garantia Estendida."
248         lRet := .F.
249     EndIf
250
251     If lRet .AND. cTypeItem <> "IMP"

```

```

252      LjGrvLog(cLI1Num,STR0033+STR0027) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado! Motivo: "###"Não é possível inserir código de
Garantia Estendida no TOTVS PDV."
253      STFMessage("ItemRegistered","STOPPOPUP",STR0033+STR0027) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado! Motivo: "###"Não é
possível inserir código de Garantia Estendida no TOTVS PDV."
254      lRet := .F.
255      EndIf
256
257      //Armazena Itens Garantia Estendida
258      If lRet
259          aRet := STDGEPProdVin(cItemCode,cCodItemGar)
260          //aRet deverá retornar qual produto comum foi utilizado para aquela garantia.
261          If Len(aRet) = 3
262              cProdVend := aRet[1]
263              nPrProd := aRet[2]
264              cSerieProd := aRet[3]
265              STWItemGarEst(1      ,; //Tipo do Processo (1=Set - 2=Get - 3=Clear)
266                          cItemCode ,; //Codigo Garantia Estendida
267                          IIF(cTypeItem == "IMP", nPrice-nDiscount, STWFormPr( PadR(cItemCode, TamSx3("B1_COD"))[1]) ),; //Valor da Garantia Estendida
268                          cProdVend ,; //Codigo Produto Vendido
269                          Val(cCodItemGar) ,; //Item Produto Vendido
270                          cTypeItem ,; //Tipo do Item - Usado para importacao de Orcamento
271                          nPrProd ,; //Preco do Produto Vendido, que será impresso no cupom gerencial
272                          cSerieProd ) //Serie do Produto Vendido
273          Else
274              LjGrvLog(cLI1Num,STR0033+STR0034 + cItemCode + " - " + cCodItemGar) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado!
Motivo: "###"Não encontrado o produto vinculado para o produto garantia "
275              STFMessage("ItemRegistered","STOPPOPUP",STR0033+STR0034) //"Garantia Estendida: Item do tipo Garantia(MV_LJTPGAR) não poderá ser registrado! Motivo: "###"Não
encontrado o produto vinculado para o produto garantia "
276              lRet := .F.
277          EndIf
278      EndIf
279      EndIf
280
281      /*/
282      Validações específicas para serviços financeiros
283      Nunca serao impressos, apenas armazenados em array para serem apresentados no final da venda
284      Não será tratado o serviço quando estiver na finalização da venda.
285      /*/
286      If lRet .And. lFinServ .AND. !lKitMaster
287          LjGrvLog(cLI1Num,"Realiza validacoes para Servicos Financeiros(MV_LJCSF = .T.)")
288
289          If STBIsFinService( cItemCode ) // Produto tipo Servico Financeiro
290              LjGrvLog(cLI1Num,"Produto tipo Servico Financeiro (B1_TIPO = MV_LJTSPF)")
291              If !(STWValidService(3, NIL, NIL, cCliCode, cCliLoja)) //Valida se não for Cliente Padrão
292                  LjGrvLog(cLI1Num,"Item não poderá ser registrado, motivo: "+STR0014) //"Venda de Serviço Financeiro não permitida para Cliente padrão."
293                  lRet := .F.
294                  STFMessage("ItemRegistered","STOP",STR0014) //"Venda de Serviço Financeiro não permitida para Cliente padrão."
295              ElseIf STBGetQuant() > 1 //Valida quantidade do Servico Financeiro
296                  LjGrvLog(cLI1Num,"Item não poderá ser registrado, motivo: " + STR0012) //"Não é permitido alterar quantidade para Serviço Financeiro."
297                  lRet := .F.
298                  STFMessage("ItemRegistered","STOP",STR0012) //"Não é permitido alterar quantidade para Serviço Financeiro."
299              Else // Flag Servico Financeiro

```

```

300         lItemServFin := .T.
301         lRet          := .T.
302
303         If !lServFinal
304             /*
305              * Verifica se o item é um Serviço Avulso.
306              */
307             If STWValidService(1,aInfoItem) .Or. cTypeItem == "IMP" //Valida se o produto serviço é Avulso (.T.) ou Vinculado (.F.) ou importado
308                 /* Valida Desconto no Total da Venda */
309                 If cTypeItem <> "IMP" .And. oTotal:GetValue("L1_DESCONT") > 0
310                     lRet := .F.
311                     STFMessage("ItemRegistered","STOP",STR0016) //#"Não é possível inserir Servicos Financeiros pois venda possui desconto"
312                 EndIf
313
314                 /*
315                  * Armazena Itens Servico Financeiro Avulso
316                  */
317                 If lRet
318                     STWItemFin(1
319                         ,; //Tipo do Processo (1=Set - 2=Get - 3=Clear)
320                         cItemCode ,; //Codigo Servico Financeiro
321                         IIF(cTypeItem == "IMP", nPrice, STWFormPr( PadR(cItemCode, TamSx3("B1_COD"))[1] ) ),; //Valor do Servico Financeiro
322                         "" ,; //Codigo Produto Vendido
323                         0 ,; //Item Produto Vendido
324                         cTypeItem ) //Tipo do Item - Usado para importacao de Orcamento
325                     EndIf
326                 Else
327                     LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: " + STR0013) //"Item de Serviço não é valido por ser Vinculado a outro produto"
328                     lRet := .F.
329                     STFMessage("ItemRegistered","STOP",STR0013) //"Item de Serviço não é valido por ser Vinculado a outro produto"
330                 EndIf
331             EndIf
332         Else
333             LjGrvLog(cL1Num,"Produto nao e tipo Servico Financeiro (B1_TIPO <> MV_LJTPSF)")
334
335             //Armazena Servicos Financeiros Vinculados ao produto se cliente nao for Padrao
336             If ( STWValidService( 3, NIL, NIL, cCliCode,;
337                 cCliLoja ) )
338                 STDServItens( PadR(cItemCode, TamSx3("B1_COD"))[1]), nItemLine, cTypeItem) //Busca Servicos Financeiros Vinculados ao produto e Armazena
339             EndIf
340
341             lRet := .T.
342         EndIf
343     EndIf
344 EndIf
345
346 If lRet .And. !lRecovery .AND. !IsInCallStack("STISelValePresente") .And. lFunGiftCar .And. STBIsGiftCard(aInfoItem[ITEM_CODIGO])
347     LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Tentou registrar item de Vale Presente. Para esse tipo de venda deve utilizar a opcao do Menu(F2)")
348     lRet := .F.
349     STFMessage("ItemRegistered","STOP", STR0015 ) //"Para a venda de 'Vale Presente', utilizar opção do 'Menu(F2)'."
350 EndIf
351

```

```

352
353   If lRet .AND. !lKitMaster
354
355       //Vale-Presente se Recovery
356       If lRecovery .And. !Empty(cValePre);
357           .AND. (STDGGiftCard(cItemCode) == "1")
358           cItemCode := cValePre
359           STBSetCodVP(cItemCode)
360           lItemFiscal := .F.
361       EndIf
362
363       /*/
364       Busca preco do item caso nao tenha sido informada por parametro
365       /*/
366       If nPrice <= 0
367           LjGrvLog(cL1Num,"Item sem preco(Preco=0), sera realizado pesquisa de preco(STWFormPr).")
368           nPrice := STWFormPr( aInfoItem[ITEM_CODIGO], cCliCode , Nil , cCliLoja , nMoeda , STBGetQuant() )
369           LjGrvLog(cL1Num,"Preco apos pesquisa:",nPrice)
370
371           If nPrice == -999 // Verifica se tabela de preço esta dentro da vigencia
372               LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Tabela de preço fora de vigência.")
373               lRet := .F.
374               STFMessage("ItemRegistered","STOP", STR0035 + CHR(13)+CHR(10) + STR0036 ) //"Tabela de preço fora de vigência." "Verifique o código da tabela contido no
parâmetro MV_TABPAD"
375
376           ElseIf nPrice <= 0 //Se nao achou preco
377               LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Nao possui preco")
378               lRet := .F.
379               STFMessage("ItemRegistered","STOP",STR0001) //"Preço não encontrado"
380
381           EndIf
382
383       EndIf
384
385
386       // Busca TES que sera usada para calcular imposto do item
387       // Caso nao tenha sido informada por parametro
388       If lRet .AND. Empty( cItemTES )
389
390           cItemTES := STBTaxTES( 2 , "01" , cCliCode , cCliLoja , ;
391                               "C" , aInfoItem[ITEM_CODIGO] , Nil , aInfoItem[ITEM_TES], ;
392                               lListProd )
393
394           LjGrvLog(cL1Num,"Nao recebeu a TES do item para impostos, entao foi realizado consulta(STBTaxTES/MaTesInt)", cItemTES)
395
396           If Empty( cItemTES )
397               LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: " + STR0002) //"Atenção. TES de Saída Inválida."
398               lRet := .F.
399               STFMessage("ItemRegistered","STOP",STR0002) //"Atenção. TES de Saída Inválida."
400           EndIf
401
402       EndIf

```

```

403
404 // Se ainda nao inicializou a venda,
405 // inicializa cabecalho do controle de impostos
406 If lRet
407     If !(STBTaxFoun())
408         LjGrvLog(cLI Num,"Inicializa o Calculo das operacoes Fiscais(STBTaxIni/MaFisIni)")
409         STBTaxIni( cCliCode      , cCliLoja      , "C"      , "S"      , ;
410                 .F.          , "SB1"          , "LOJA701" , .T.      , ;
411                 cCliType      )
412     EndIf
413
414 // Salva cabecalho da venda na cesta de vendas
415 If nItemLine == 1
416     LjGrvLog(cLI Num,"Registro do Primeiro item, salva informacoes do cabecalho da venda (STBSaveSaleBasket)")
417     STBSaveSaleBasket()
418 EndIf
419
420 // Enviado via paramentro a media (Total de Segundos / Itens de Venda)
421 If SL1->(ColumnPos("L1_TIMEITE")) > 0
422     STDSPBasket( "SL1" , "L1_TIMEITE" , nSecItem / nItemLine )
423 EndIf
424
425 //Apos encontrar o preco verifica se altera quantidade
426 //com os dados do codigo de barras e o preco
427 STBQtdEtq( cCodIniIT , nPrice , aInfoItem[ITEM_BALANCA] )
428 EndIf
429
430 // Arredondamento
431 nItemTotal := STBArred( nPrice * STBGetQuant() )
432
433 // Total da venda antes do calculo dos impostos
434 nVTotAfter := oTotal:GetValue("L1_VLRTOT")
435
436 /*Atualiza totalizadores da Matxfis para evitar erro de diferenca de valores entre sistema e impressora fiscal.
437 Necessário para quando registra um item com desconto e recebe negacao da permissão de superior ou quando
438 caixa faz alguma operação na impressora. Ex. Troca de papel, queda de luz, etc. durante a inclusão do item.*/
439 If lItemDel .And. STBTaxFoun("IT", nItemLine)
440     conout("STIWtemRegister - Ajustando valor do item na Matxfis!")
441     LjGrvLog(cLI Num,"Ira ajustar valor do item na Matxfis")
442     STBTaxDel(nItemLine, .F. )
443     STBTaxAlt("IT_PRCUNI",0,nItemLine)
444     STBTaxAlt("IT_VALMERC",0,nItemLine)
445     lItemDel := .F.
446 EndIf
447
448 // Add item para calculo do imposto
449 If lRet
450     STBTaxIniL( nItemLine      , .F.          , aInfoItem[ITEM_CODIGO] , cItemTES      , ;
451               STBGetQuant()    , nPrice      , 0                  , nItemTotal      )
452
453 // Atualiza o preco pois apos passar pelas funcoes fiscais
454 // o preco pode ter sido alterado, arredondado etc.. devido aos impostos

```



```

455      nPrice := STBTaxRet(nItemLine,"IT_PRCUNI"      )
456
457      //Verifica se é item de Servico (RPS)
458      If !MVLJPRDSV .And. STBTaxRet(nItemLine, "IT_VALISS") > 0
459          If cTypeItem == "IMP"
460              lIsItemRPS := !lItemFiscal
461              ElseIf !STFGetCfg("lUseECF") .And. ExistFunc("Lj7RPSNew") .And. Lj7RPSNew()
462                  lIsItemRPS := .T.
463          EndIf
464      EndIf
465
466      //valores de impostos por ente tributario da lei dos impostos
467      If lFLImpItem .And. (Len(aInfoItem) >= 23)
468          aAlqLeiTr := STBLImpItem(aInfoItem[ITEM_POSIPI] ,aInfoItem[ITEM_EX_NCM] ,aInfoItem[ITEM_CODISS], aInfoItem[ITEM_CODIGO] )
469          nTamArray := Len(aAlqLeiTr)
470          aInfoItem[ITEM_TOTIMP] := Iif(nTamArray >= 1,aAlqLeiTr[1],0)
471          aInfoItem[ITEM_TOTFED] := Iif(nTamArray >= 2,aAlqLeiTr[2],0)
472          aInfoItem[ITEM_TOTEST] := Iif(nTamArray >= 3,aAlqLeiTr[3],0)
473          aInfoItem[ITEM_TOTMUN] := Iif(nTamArray >= 4,aAlqLeiTr[4],0)
474      EndIf
475
476      // Inicia Evento | Abertura do Cupom
477      If (lItemGarEst .OR. lItemServFin) .OR. ; //Garantia Estendida Avulso e Servico Financeiro Avulso nao registra
478          lIsItemRPS //Item de Servico (RPS)
479          lItemFiscal := .F.
480      EndIf
481
482      If lItemFiscal .AND. STFGetCfg("lUseECF")
483          LjGrvLog(cL1Num,"Inicia Operacao de Registro Fiscal para ECF")
484
485          // Requisito para Alagoas
486          // Verifica data e hora para registro de cada item.
487          If LjAnalisaLeg(31)[1]
488
489              //Verifica se a data do sistema eh a mesma data da impressora fiscal.
490              If !STWCheckDate()
491                  LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Diferenca entre a Data/Hora do Sistema com a Impressora Fiscal.")
492                  lRet := .F.
493              EndIf
494          EndIf
495      EndIf
496
497      //Verifica se tem desconto no Item
498      If nDiscount > 0 .AND. (cTypeDesc $ "V|P")
499          LjGrvLog(cL1Num,"Item possui desconto de:"+cValToChar(nDiscount))
500          STWIDBefore( nDiscount , cTypeDesc )
501      EndIf
502
503      STWItemDiscount( nItemLine , nDiscount , cTypeDesc , cTypeItem , lItemFiscal)
504      //Busca desconto do usuario
505      aDiscount := STBGetItDiscount()
506

```

```

507      // Busca permissão de desconto
508      If aDiscount[1] > 0
509          If ExistFunc("STBValidDesc")
510              lRet := IIf(cTypeItem == "KIT",.T.,STBValidDesc())
511              If !lRet
512                  LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Usuário não possui permissão para desconto")
513              EndIf
514          Else
515              LjGrvLog(cL1Num,"Função STBValidDesc não compilada, necessário atualizar o fonte STBItemDiscount, permissão de desconto não foi verificado." )
516          EndIf
517      EndIf
518
519      //Valida valor total da venda caso o item seja aceito
520      If lRet .And. l950SP100K
521          If !Lj950SP100K( oTotal:GetValue("L1_VLRTOT") , 1, cCliCode , cCliLoja)
522              LjGrvLog(cL1Num,"O valor deste item lançado fará com que o valor total da venda ultrapasse o total permitido por
legislação.",oTotal:GetValue("L1_VLRTOT") + STBarred( nPrice * STBGetQuant() ))
523              lRet := .F.
524          EndIf
525      EndIf
526
527      //Abertura do Cupom
528      If lRet .AND. !lReceiptIsOpen
529          LjGrvLog(cL1Num,"Abertura do Cupom Fiscal - Inicio" )
530          lRet := STBOpenReceipt(cCliCode, cCliLoja, lInfoCNPJ)
531          LjGrvLog(cL1Num,"Abertura do Cupom Fiscal - Retorno:",lRet )
532
533          STWSetIsOpenReceipt( lRet )
534
535          If !lRet //Ocorreu problemas na abertura do CF
536
537              If ExistFunc("STWGetMsgOperante")
538                  cMsgErro := STWGetMsgOperante()
539              Else
540                  CONOUT(STR0017) //"Contate o seu suporte. Favor atualizar o fonte STWECFCONTROL.PRW e STWZReduction.PRW."
541              EndIf
542
543              LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Falha na Abertura de Cupom. Mensagem:"+cMsgErro)
544              STFMessage("ItemRegistered","STOP",cMsgErro) //"Nao foi possivel a abertura do Cupom Fiscal"
545              //Chama a função que deleta o item na MatxFis, e não somente o marca como deletado
546              STBTaxDel( nItemLine , .T. )
547          Else
548              //Se Conseguiu Abrir Cupom atualiza Cesta de Venda
549              LjGrvLog(cL1Num,"Abertura de Cupom Fiscal realizada com sucesso" )
550              STDSPBasket( "SL1" , "L1_SITUA" , "02" ) // "02" - Impresso a Abertura do Cupom
551          EndIf
552      ElseIf lReceiptIsOpen
553          LjGrvLog(cL1Num,"Abertura do Cupom Fiscal já realizada" )
554      EndIf
555
556
557      If lRet //Cupom Aberto com sucesso

```

```

558
559
560 // Indica se imprime codigo de barras no cupom ao inves do codigo do produto
561 // LjAnalisaLeg(39)[1] - A legislação exige que no cupom fiscal seja impresso o codigo EAN"
562 If ( cImpCodBar == "S" .AND. !Empty(aInfoItem[ITEM_CODBAR]) ) .OR. LjAnalisaLeg(39)[1]
563   cCodeForPrint := aInfoItem[ITEM_CODBAR]
564 Else
565   cCodeForPrint := aInfoItem[ITEM_CODIGO]
566 EndIf
567
568 If LjAnalisaLeg(9)[1] //- Codigo do produto preenchido com zeros(0) a esquerda
569   Right('000000000000'+Alltrim(cCodeForPrint),13)
570 EndIf
571
572 cDscPrdPAF := aInfoItem[ITEM_DESCRICA0]
573 /*CONVÊNIO ICMS 25, DE 8 DE ABRIL DE 2016
574   #código CEST#NCM/SH#descrição do item*/
575 If STBIsPAF()
576   cITcEst      := AllTrim(STBTaxRet(nItemLine,"IT_CEST"))
577   cPosIpi      := AllTrim(aInfoItem[ITEM_POSIPI])
578   If !Empty(cITcEst) .And. !Empty(cPosIpi)
579     cDscPrdPAF := "#" + cITcEst + "#" + cPosIpi + "#" + AllTrim(cDscPrdPAF)
580   EndIf
581 EndIf
582
583 cQuant      := StrZero(STBGetQuant(),8,3)
584 cVrUnit     := AllTrim(STR(nPrice))
585 cDesconto   := AllTrim(STR(STBTaxRet(nItemLine,"IT_DESCONTO" )))
586 cSitTrib    := STBTaxSit(nItemLine)
587 cVlrItem    := AllTrim(STR(STBTaxRet(nItemLine,"IT_TOTAL" )))
588
589 //Ponto de entrada para adicionar dados na impressao da descricao do produto
590 If llj7013
591   LjGrvLog(SL1->L1_NUM,"Antes da Chamada do Ponto de Entrada:LJ7013",{cCodeForPrint, cDscPrdPAF, cQuant, cVrUnit, cDesconto, cSitTrib, cVlrItem,
nItemLine})
592   aRetLj7013 := ExecBlock("LJ7013",.F.,.F.,{cCodeForPrint, cDscPrdPAF, cQuant, cVrUnit, cDesconto, cSitTrib, cVlrItem, nItemLine })
593   LjGrvLog(SL1->L1_NUM,"Apos a Chamada do Ponto de Entrada:LJ7013",aRetLj7013)
594
595   If ValType( aRetLj7013 ) == "A" .AND. Len( aRetLj7013 ) >= 7
596     cCodeForPrint := aRetLj7013[1]
597     cDscPrdPAF    := aRetLj7013[2]
598     cQuant        := aRetLj7013[3]
599     cVrUnit       := aRetLj7013[4]
600     cDesconto     := aRetLj7013[5]
601     cSitTrib      := aRetLj7013[6]
602     cVlrItem      := aRetLj7013[7]
603   EndIf
604 EndIf
605
606 LjGrvLog(cL1Num, "Registro do Item - Inicio"+
607   " Codigo:"+cCodeForPrint+;
608   ";Descricao:"+cDscPrdPAF+;

```

```

609         ";Qtde:"+cQuant++;
610         ";ValorUnit:"+cVrUnit++;
611         ";Desconto:"+cDesconto++;
612         ";Aliq/SitTrib:"+STBTaxSit(nItemLine)+;
613         ";Total:"+cVlrItem++;
614         ";Unid.Medida:"+aInfoItem[ITEM_UNID_MEDIDA])
615
616     // Inicia Evento
617     aRet := STFFireEvent( ProcName(0)           ,;      // Nome do processo
618                          "STItemReg"           ,;      // Nome do evento
619                          {cCodeForPrint        ,;      // 01 - Codigo do Item
620                          cDscPrdPAF            ,;      // 02 - Descricao
621                          cQuant               ,;      // 03 - Quantidade
622                          cVrUnit              ,;      // 04 - Valor Unitario
623                          cDesconto            ,;      // 05 - Desconto do Item
624                          cSitTrib             ,;      // 06 - Situacao tributaria
625                          cVlrItem             ,;      // 07 - Total do Item
626                          aInfoItem[ITEM_UNID_MEDIDA] ,;      // 08 - Unidade de Medida
627                          "2"                  } )      // 09 - TIPO TES ( 2 - Saida )
628
629     lRet := Len(aRet) == 0 .Or. (ValType(aRet[1]) == "N" .AND. aRet[1] == 0)
630     LjGrvLog(cL1Num, "Registro do Item - Retorno:",lRet)
631     If !lRet
632         LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Não realizou o registro fiscal. aRet",aRet)
633     EndIf
634
635     //Informa que a venda esta em andamento
636     STBSaleAct(.T.)
637
638 EndIf
639
640 Else
641     LjGrvLog(cL1Num,"Inicia Operacao de Registro para item nao fiscal ou nao usuario de ECF")
642
643     //Verifica se tem desconto no Item
644     If nDiscount > 0 .AND. (cTypeDesc $ "V|P")
645         STWIDBefore( nDiscount , cTypeDesc )
646     EndIf
647
648     STWItemDiscount( nItemLine , nDiscount , cTypeDesc , cTypeItem , lItemFiscal )
649
650     If STFGetCfg("lUseECF") .Or. !lItemFiscal
651         // Soma valor não fiscal no totalizador
652         STBSumNotFiscal( (nPrice * STBGetQuant()) - STBTaxRet(nItemLine,"IT_DESCONTO"), lIsItemRPS )
653     EndIf
654 EndIf
655
656 EndIf
657
658 // Salva Item na Cesta de vendas
659 If lRet
660

```

```

661      LjGrvLog(cL1Num, "Registra Item - Salva Item na Cesta de vendas - SL2" )
662
663      If lCRdesItTt .AND. !lDesligaRD .AND. cTypeItem == "IMP"
664          lItImpRDPT := .T.
665      EndIf
666
667      lRet := STBSaveItBasket( aInfoItem , nItemLine, cIdItRel, lItImpRDPT)
668      //seta o contador do item fiscal/não fiscal
669      If lRet .AND. lSumItFisc
670          STBSumItem(lItemFiscal .AND. STFGetCfg("lUseECF"), nItemLine )
671      Endif
672
673      If(lRet .AND. !Empty(cCodProdKit) , STDSPBasket("SL2", "L2_KIT", rtrim(cCodProdKit), nItemLine) , )
674
675      IIf(lRet , lRet := STDSPBasket( "SL2" , "L2_FISCAL" , lItemFiscal , nItemLine ) ,)
676
677      If lRet .AND. lListProd .and. cTypeItem <> "IMP" //Lista importada do orçamento já alimenta os campos
678
679          STDSPBasket( "SL2" , "L2_CODLPRE" ,cCodList , nItemLine )
680          STDSPBasket( "SL2" , "L2_ITLPRE" , cCodListIt , nItemLine )
681          STDSPBasket( "SL2" , "L2_MSMLPRE" ,cCodMens , nItemLine )
682          STDSPBasket( "SL2" , "L2_ENTREGA" ,cEntrega , nItemLine )
683
684          If !lItemFiscal .And. ; //Nao eh item fiscal
685              If(lMVLJPRDSV, !lIsTesISS( STDGPBasket('SL1','L1_NUM'), STDGPBasket("SL2","L2_TES",nItemLine) ), .T.) //Nao eh Item de Servico (ISS)
686
687                  STDSPBasket( "SL2" , "L2_RESERVA" , "S" , nItemLine )
688              EndIf
689
690          EndIf
691      EndIf
692      If lRet .AND. lProdBonif
693          STDSPBasket( "SL2" , "L2_BONIF" , .T., nItemLine )
694      Endif
695
696      //Tratamento de item Servico Financeiro
697      If lRet .AND. lFinServ
698          //Se nao for Servico Financeiro, verifica posicao real do item na Cesta de Produtos
699          If !STBIsFinService( cItemCode )
700              STDSPBasket( "SL2" , "L2_ITEMREA" , STWNumItem(), nItemLine )
701          Else
702              STDSPBasket( "SL2" , "L2_ITEMREA" , "00", nItemLine )
703          EndIf
704      EndIf
705  EndIf
706
707 Else
708
709      LjGrvLog(cL1Num,"Item não poderá ser registrado, motivo: Item nao localizado ou indisponivel no cadastro do PDV.")
710      LjGrvLog(cL1Num,"Status Codigo do Item:" + cItemCode + " - Localizado:"+IIF(aInfoItem[ITEM_ENCONTRADO],"Sim","Nao")+ " -
Bloqueado:"+IIF(aInfoItem[ITEM_BLOQUEADO],"Sim","Nao") + " - Tipo GE(B1_TIPO):"+ IIF(aInfoItem[ITEM_TIPO] == 'GE',"Sim","Nao") )
711      lRet := .F.          // Item não encontrado

```

```

712
713 EndIf
714
715 If !lRet .AND. !lKitMaster
716     //Se ocorreu problemas na Impressão do item
717     //Chama a função que deleta o item na MatxFis, e não somente o marca como deletado
718
719     If STBTaxFoun("IT", nItemLine)
720         STBTaxDel(nItemLine, .T.)
721     EndIf
722
723     // Seta controle de item deletado como True
724     lItemDel := .T.
725 EndIf
726
727 If lRet
728
729     STFMessage("ItemRegistered","STOP",STR0004) //"Item registrado"
730     LjGrvLog( cL1Num, STR0004 ) //"Item registrado"
731
732 ElseIf aInfoItem[ITEM_BLOQUEADO]
733
734     STFMessage("ItemRegistered","STOP",STR0009 + aInfoItem[ITEM_CODIGO] + STR0010) //"Item bloqueado"
735     LjGrvLog( cL1Num, STR0010 ) //"Item bloqueado"
736
737 ElseIf lKitMaster
738
739     STFMessage("ItemRegistered","STOP",STR0005) //"Item do tipo Kit registrado"
740     LjGrvLog( cL1Num, STR0005 ) //"Item do tipo Kit registrado"
741
742 ElseIf !lRecShopCard
743
744     STFMessage("ItemRegistered","STOP",STR0006) //"Nao foi possivel registrar o item"
745     LjGrvLog( cL1Num, STR0006 ) //"Nao foi possivel registrar o item"
746
747 EndIf
748
749 // Limpa Objetos variaveis e restaura padrao
750
751 STBSetDefQuant() // Seta quantidade padrao apos o registro de item
752 STBDefItDiscount() // Seta desconto padrao apos registro de item
753
754 //Mostra mensagens referente ao registro de itens
755 If !lRecovery
756     STFShowMessage("ItemRegistered")
757 EndIf
758
759 LjGrvLog( cL1Num, "Fim - Workflow Registra Item" ) //Gera LOG
760 If lRet
761     If !lAddItem .AND. MsFile("ACR")
762         STWBonusSales( nItemLine )
763     EndIf

```

```

764     LjGrvLog( cL1Num, "Faz a persistencia dos dados nas tabelas SL1, SL2 e SL4." ) //Gera LOG
765     STDSaveSale(nItemLine)
766     LjGrvLog("Registra_Item", "ID_FIM")
767 Else
768     LjGrvLog("Registra_Item", "ID_ALERT")
769 EndIf
770
771 Return lRet
772
773
774 //-----
775 /* {Protheus.doc} STWSetOpenedReceipt
776 Function Registra_Item
777
778 @param lSet          Seta se cupom está aberto ou fechado
779 @return Nil
780 @obs
781 @sample
782 */
783 //-----
784 Function STWSetIsOpenReceipt( lSet )
785
786 Default lSet := .F.    // Seta se cupom está aberto ou fechado
787
788 ParamType 0 var      lSet          As Logical      Default      .F.
789
790 lReceiptIsOpen := lSet
791
792 Return
793
794
795 //-----
796 /* {Protheus.doc} STWSetOpenedReceipt
797 Retorna se o cupom está aberto ou fechado
798
799 @return lReceiptIsOpen    Retorna se o cupom está aberto ou fechado
800 @obs
801 @sample
802 */
803 //-----
804 Function STWGetIsOpenReceipt()
805
806 Return lReceiptIsOpen
807
808 //-----
809 /* {Protheus.doc} STWSetOpenedReceipt
810 Retorna se o cupom está aberto ou fechado
811
812 @param lItemFiscal Indica se é cupom fiscal ou não
813 @return lRegistred    Retorna se o item foi registrado ou nao
814 @obs
815 @sample

```

```

816 */
817 //-----
818 Function STWSetRegIt(nItemLine,cCodItem,cTesPad,nTotItem,;
819                     lItemFiscal)
820
821 Local lRegistered := .T. //Variavel que verifica se o item foi registrado
822 Local cCliCode := STDGPBasket("SL1","L1_CLIENTE") //Codigo do Cliente na Cesta
823 Local cCliLoja := STDGPBasket("SL1","L1_LOJA") //Loja do Cliente na Cesta
824 Local cCliType := STDGPBasket("SL1","L1_TIPOCLI") //Tipo do Cliente na Cesta
825 Local nMoeda := 1 //Moeda Corrente
826 Local nPrice := STBSearchPrice(cCodItem, cCliCode, cCliLoja )
827
828 Default lItemFiscal := .T.
829
830 lRegistered := STWItemReg( nItemLine , ; // Item
831                            cCodItem , ; // Codigo Prod
832                            cCliCode , ; // Codigo Cli
833                            cCliLoja , ; // Loja Cli
834                            nMoeda , ; // Moeda
835                            Nil , ; // Valor desconto
836                            Nil , ; // Tipo desconto ( Percentual ou Valor )
837                            Nil , ; // Item adicional?
838                            cTesPad , ; // TES
839                            cCliType , ; // Tipo do cliente (A1_TIP0)
840                            lItemFiscal , ; // Registra item no cupom fiscal?
841                            nPrice , ; // Preço
842                            Nil , ; // Tipo do Item
843                            Nil , ; // Imprime CNPJ no cupom Fiscal
844                            Nil , ; // Tela do POS está sendo apresentada
845                            nTotItem ) // Total dos segundos entre itens
846
847 //Funcao que zera o digitado pelo usuario no GET de Produto
848 STISetPrd("")
849
850 If lRegistered
851     STIShowProdData(nItemLine)
852     STIGridCupRefresh(nItemLine,nItemLine) // Sincroniza a Cesta com a interface
853
854     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Código produto registrado", cCodItem )
855     LjGrvLog( "L1_NUM: " + STDGPBasket("SL1","L1_NUM"), "Item Fiscal", lItemFiscal)
856 Else
857     LjGrvLog( "L1_Num:" + STDGPBasket("SL1","L1_NUM"), ">>>NC<<< - Falha no Registro do Item -> Código produto não registrado:" + cCodItem + "Item:" + cValToChar(nItemLine) + "
- Item Fiscal:" + "Item Fiscal:" + IIF(lItemFiscal,"Sim","Nao") + " - TES:" + cTesPad)
858 EndIf
859
860 Return lRegistered
861
862 //-----
863 /* {Protheus.doc} STWNumItem
864 Retorna o Item real do produto na cesta
865
866 @param

```



```

867 @return nItemReal      Item real do produto
868 @obs
869 @sample
870 */
871 //-----
872 Static Function STWNumItem()
873
874 Local aArea      := GetArea()           //Salva area
875 Local aSaveLines := FWSaveRows()       //Array de linhas salvas
876 Local cItemReal  := "00"               //Numeracao real de item do produto
877 Local nI         := 0                  //Contador
878 Local oModel     := STDGPBModel()      //Model
879
880 //Verifica se produto é Servico Financeiro
881 For nI := 1 To oModel:GetModel("SL2DETAIL"):Length()
882     //Posiciona no item
883     oModel:GetModel("SL2DETAIL"):GoLine(nI)
884
885     //Verifica se item ativo e nao Servico Financeiro
886     If !oModel:GetModel("SL2DETAIL"):IsDeleted(nI) .And. !STBIsFinService(oModel:GetValue("SL2DETAIL" , "L2_PRODUTO"))
887         cItemReal := Soma1(cItemReal)
888     EndIf
889 Next nI
890
891 //Restaura areas
892 RestArea(aArea)
893 FWRestRows(aSaveLines)
894
895 // Limpa os vetores para melhor gerenciamento de memoria (Desaloca Memória)
896 aSize( aSaveLines, 0 )
897 aSaveLines := Nil
898
899 Return cItemReal
900
901 //-----
902 /* {Protheus.doc} STWBtFimVnd
903 Retorna o Item real do produto na cesta
904
905 @param
906 @return lRet - Ok ou não ok?
907 @obs
908 @sample
909 */
910 //-----
911 Function STWBtFimVnd()
912 Local lFinServ := AliasIndic("MG8") .AND. SuperGetMv("MV_LJCSF",,.F.)
913 Local lRet     := .T.
914
915 IF lFinServ
916     STWFindServ()
917 Else
918     STWNxtTelaIt()

```

```

919 EndIf
920
921 Return lRet
922
923 //-----
924 /* {Protheus.doc} STWNxtTelaIt
925 Valida qual a proxima tela a ser mostrada no lançamento do item
926
927 @return lRet    -   Ok ou não ok?
928 @obs
929 @sample0
930 */
931 //-----
932 Function STWNxtTelaIt()
933
934 Local lRet      := .T.
935 Local l0brigaLJ950 := ExistFunc("Lj950ImpCpf") .And. Lj950ImpCpf(STDGPBasket("SL1","L1_VLRTOT"))
936 Local lCRdesItTt := SuperGetMv("MV_LJRGDES",.F.) .AND. SuperGetMV("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
937 Local cTabPad := SuperGetMV("MV_TABPAD")
938 Local lDesligaRD := .F. // Desabilita a Regra de Desconto do Varejo
939 Local lCalRDImp := lCRdesItTt .And. SuperGetMv("MV_LJRDIMP",,"0") == "0" .And. IsInCallStack("STIFinishImp")
940
941 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
942     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
        STDGPBasket('SL1','L1_LOJA'))
943     lCRdesItTt := !lDesligaRD
944
945     If !lDesligaRD .And. lCalRDImp
946         lDesligaRD := .T.
947     ElseIf !lDesligaRD .And. !lCalRDImp
948         lDesligaRD := .F.
949     EndIf
950 EndIf
951
952 If !lDesligaRD .And. lCRdesItTt
953     totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal("TOTVSPDV", STDGPBasket("SL1","L1_CLIENTE"), STDGPBasket("SL1","L1_LOJA"), cTabPad)
954 EndIf
955
956 /*Caso exija CPF ou tenha paramterização para chamar tela de CPF no final da venda*/
957 If l0brigaLJ950 .Or. (ExistFunc("LjInfDocCli") .And. LjInfDocCli() > 1)
958     lDadosInf := .T.
959     If Empty(STDGPBasket("SL1","L1_CGCLI")) .And. Empty(AllTrim(STDGPBasket("SL1","L1_PFISICA")))
960         STI7InfCPF(.F.)
961     Endif
962     STWInfoCNPJ(, .T., l0brigaLJ950)
963 Else
964     STICallPayment()
965 EndIf
966
967 Return lRet

```

Componentes/Item/STWKitSales.prw

59 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWKITSALES.CH"
4
5 //-----
6 /*{Protheus.doc} STWKitSales
7 Gera a lista de códigos de produtos que compoem determinado Kit
8 @param cCodProdKit  Codigo do produto kit.
9 @return Nil
10 @obs
11 @sample
12 */
13 //-----
14
15 Function STWKitSales(cCodProdKit)
16
17 Local aItensKit      := {}           // Array com os itens do kit de produtos
18 Local nI             := 0           // Contador de laço
19 Local nItemLine      := 0           // Linha do item
20 Local cTypeDesc       := "P"        // Tipo de desconto (percentual)
21 Local oModelSale     := STDGPBModel() // Model de venda
22 Local nQuantKit      := STBGetQuant() // Quantidade do Kit
23 local cIdItRel       := ""          // Id do item relacionado.
24
25 Default cCodProdKit := ""
26
27 // Carrega lista de produtos a serem sugeridos
28 aItensKit := STDKitSales(cCodProdKit)
29
30 If Len(aItensKit) > 0
31
32     For nI := 1 To Len(aItensKit)
33
34         // Selecionados os produtos, seta a quantidade de cada produto que compoe o kit
35         STBSetQuant( aItensKit[nI][3] * nQuantKit )
36
37         If oModelSale:GetModel("SL2DETAIL"):Length() == 1 .AND. Empty(STDGPBasket("SL2","L2_NUM",1))
38             nItemLine := 1
39         Else
40             nItemLine := oModelSale:GetModel("SL2DETAIL"):Length()+1
41         EndIf
42
43         If nI == 1
44             cIdItRel := StrZero(nItemLine,TamSx3("L2_ITEM")[1])
45         EndIf
46
47         // Dispara o registro de item para cada produto que compoe o kit
48         If !STWItemReg(nItemLine, aItensKit[nI][1],,,,aItensKit[nI][2],cTypeDesc,, ,,,,"KIT",,,,,,,,,,cIdItRel,,cCodProdKit)
49
50             // Havendo falha no registro do item, gera nova mensagem e aborta a emissão dos demais
51             STFMessage("STBKitSales", "STOP", STR0001) //"Não foi possível registrar os itens que compoem o kit."
52             Exit

```

```
53      EndIf
54
55      Next nI
56
57 EndIf
58
59 Return Nil
```

Componentes/MultimediaRegister/STBMultimediaRegister.prw

88 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STBMRIsRegistered
6 Verifica se a Midia foi registrada na venda
7
8 @param
9 @return lRet          Retorna se registrou a midia
10 @obs
11 @sample
12 */
13 //-----
14 Function STBMRIsRegistered()
15
16 Local lRet := .F.          // Retorna se registrou a Midia na Venda
17
18 lRet := !(Empty( STDGPBasket("SL1","L1_MIDIA") ))
19
20 Return lRet
21
22
23 //-----
24 /*{Protheus.doc} STBMRSetMedia
25 Adiciona o Registro de Midia na venda
26
27 @param cCodMedia Codigo da midia
28 @return lRet          Retorna se registrou a midia
29 @obs
30 @sample
31 */
32 //-----
33 Function STBMRSetMedia( cCodMedia )
34
35 Local lRet          := .F.          // Retorna se Registrou a midia
36
37 Default cCodMedia   := ""
38
39 ParamType 0 Var cCodMedia AS Character Default ""
40
41 lRet := STDSPBasket("SL1","L1_MIDIA",cCodMedia)
42
43 Return lRet
44
45
46 //-----
47 /*{Protheus.doc} STBMR0brigat
48 Verifica se utiliza Registro de Mídia
49
50 @param
51 @return lUse          Retorna se utiliza Registro de Mídia
52 @obs

```

```

53 @sample
54 /*/
55 //-----
56 Function STBMRUse()
57
58 Local lUse      := .T. // Retorna se utiliza registro de midia
59
60 If AllTrim(Str(SuperGetMv("MV_LJRG MID",,0))) == "0"
61     lUse := .F.
62 EndIf
63
64 Return lUse
65
66
67 //-----
68 /*/{Protheus.doc} STBMR0brigat
69 Verifica se o Registro de mídia é obrigatório
70
71 @param
72 @return    l0brigat          Retorna se o Registro de mídia é obrigatório
73 @obs
74 @sample
75 /*/
76 //-----
77 Function STBMR0brigat()
78
79 Local l0brigat    := .F.      // Retorna se o registro de mídia é obrigatório
80 Local cMultimedia := ""      // Armazena conteúdo do parametro de midia
81
82 cMultimedia := AllTrim(Str(SuperGetMv("MV_LJRG MID",,0)))
83
84 If cMultimedia == "2"
85     l0brigat := .T.
86 EndIf
87
88 Return l0brigat

```

Componentes/MultimediaRegister/STDMultimediaRegister.prw

73 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STDMRValid
6 Valida se existe cadastros válidos de midia
7
8 @param
9 @return   lRet           Retorna se existem registros válidos de midia
10 @obs
11 @sample
12 */
13 //-----
14 Function STDMRValid()
15
16 Local lRet      := .F.           // Retorna se existem cadastros validos
17 Local aArea     := GetArea()     // Armazena alias corrente
18
19 DbSelectArea("SUH")
20 DbSetOrder(1) // UH_FILIAL + UH_MIDIA
21
22 If DbSeek(xFilial("SUH"),.T.)
23     While !EOF()
24         If SUH->UH_VALIDO=="1"
25             lRet := .T.
26             Exit
27         EndIf
28         DbSkip()
29     EndDo
30 Else
31     lRet := .F.
32 EndIf
33
34 RestArea(aArea)
35
36 Return lRet
37
38
39 //-----
40 /*{Protheus.doc} STDMRGet
41 Busca as midias cadastradas
42
43 @param
44 @return   aMidias        Retorna As midias cadastradas
45 @obs
46 @sample
47 */
48 //-----
49 Function STDMRGet()
50
51 Local aMidias   := {}           // Retorna as midias cadastradas
52 Local aArea     := GetArea()    // Armazena alias corrente

```

```
53
54 DbSelectArea("SUH")
55 DbSetOrder(1) // UH_FILIAL + UH_MIDIA
56
57 If DbSeek(xFilial("SUH"))
58     While !EOF() .AND. ( xFilial("SUH") == SUH->UH_FILIAL )
59         If SUH->UH_VALIDO == "1"
60             AADD( aMidias , { SUH->UH_MIDIA , SUH->UH_DESC } )
61         EndIf
62         SUH->(DbSkip())
63     EndDo
64 EndIf
65 RestArea(aArea)
66 Return aMidias
```


Componentes/MultimediaRegister/STWMultimediaRegister.prw

118 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWMULTIMEDIAREGISTER.CH"
4
5 //-----
6 /*{Protheus.doc} STBMultimediaRegister
7 Function registro Multi-Midia
8
9 @param cCodMedia  Código da midia
10 @param lShowScreen  Mostra tela
11 @return  lRet          Retorna se registrou a midia
12 @obs
13 @sample
14 */
15 //-----
16 Function STWMultimediaRegister( cCodMedia , lShowScreen )
17
18 Local lRet          := .F.          // Retorna se registrou a midia
19 Local cMultimidia   := ""           // Armazena consulta do parâmetro
20
21 Default cCodMedia    := ""
22 Default lShowScreen  := .T.
23
24 ParamType 0 Var cCodMedia  AS Character  Default ""
25 ParamType 1 Var lShowScreen AS Logical   Default .T.
26
27 /*/
28 0 - Nao utiliza
29 1 - Usa registro de Midia com preenchimento opcional.
30 2 - Usa registro de Midia com preenchimento obrigatorio.
31 /*/
32 cMultimidia := AllTrim(Str(SuperGetMv("MV_LJRG MID",,0)))
33
34 Do Case
35
36   Case cMultimidia == "0" // Nao utiliza
37
38     lRet := .T.
39
40     If lShowScreen
41       STFMessage("STWMultimediaRegister","ALERT",STR0001) //"Não utiliza Registro de Midia. para utilizar habilitar o parâmetro MV_LJRG MID"
42     EndIf
43
44   Case cMultimidia == "1" // Opcional
45
46     If !Empty(cCodMedia)
47       lRet := STBMRSetMedia(cCodMedia)
48     Else
49       lRet := .T.
50     EndIf
51
52   Case cMultimidia == "2" // Obrigatório

```

```

53
54     If !Empty(cCodMedia)
55         lRet := STBMRSetMedia(cCodMedia)
56     Else
57
58         /*/
59         Caso venha vazio e for obrigatorio, ver se ja foi registrado, senao chama tela
60         /*/
61
62         lIsRegistered := STBMRIIsRegistered()
63
64         If lShowScreen
65             cCodMedia := "000001" // STIMultimediaRegister() // TODO: Get da Tela de registro multimedia
66             lRet := STBMRSetMedia(cCodMedia)
67         EndIf
68
69         If !lIsRegistered
70
71             cCodMedia := "000001" // STIMultimediaRegister( lObrigatório ) // TODO: Get da Tela de registro multimedia, Obrigatório, nao sair da tela enquanto nao
informar
72             lRet := STBMRSetMedia(cCodMedia)
73
74             Else
75                 lRet := .T.
76             EndIf
77         EndIf
78     EndIf
79 EndCase
80 EndCase
81
82 STFShowMessage("STWMultimediaRegister")
83
84 Return lRet
85
86
87 //-----
88 /*/{Protheus.doc} STWMRIniVal
89 Verifica as configurações do Registro Multi-Media na inicialização do sistema
90
91 @param
92 @return lRet          Retorna se a configuracao de registro de midia é válida
93 @obs
94 @sample
95 /*/
96 //-----
97 Function STWMRIniVal()
98
99 Local lRet := .T.      // Retorno da funcao
100
101 If STBMR0brigat()      // Verifica se o registro de midia é obrigatório
102
103     /*/

```

```
104      Sendo obrigatorio, verifica se existem registros válidos
105      /*/
106      lRet := STDMRValid()
107
108      If !lRet
109          //"0 parametro MV_LJRG MID esta com o preenchimento obrigatorio
110          // porem nao existem midias cadastradas, favor regularizar."
111          STFMessage("STWMultimediaRegister","ALERT",STR0002)
112      EndIf
113
114 EndIf
115
116 STFSHOWMessage("STWMultimediaRegister")
117
118 Return lRet
```

Componentes/NCCs/STDNCCModel.prw

166 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWMBROWSE.CH"
4 #INCLUDE "FWMVCDEF.CH"
5 #INCLUDE "PROTHEUS.CH"
6 #INCLUDE "APWIZARD.CH"
7 #INCLUDE "FWADAPTEREAI.CH"
8
9 #DEFINE SEIRECNO 5 // TODO: nao esta sendo usado apagar????
10
11 Static aNCCs := {}
12 Static nTotalNCCs := 0
13
14 //-----
15 /*{Protheus.doc} STDFindNCCModel
16 Busca as NCCs em aberto do cliente selecionado e preenche o Model da SEI com essas NCCs.
17
18 @param cCliente - Codigo do cliente
19 @param cLoja - Codigo da loja no cliente
20 @param aSelReynos, array , valor do campo MDK_NUMREC
21 @return oModelNCC - Retorna estrutura de dados do Model
22 @obs
23 @sample
24 */
25 //-----
26
27 Function STDFindNCCModel(cCliente,cLoja,lImport,cOrc, aSelReynos)
28
29 Local cCliPad := SuperGetMv( "MV_CLIPAD" ) // Cliente Padrao
30 Local cLojaPad := SuperGetMV( "MV_LOJAPAD" ) // Loja Padrao
31 Local lSTFNDNCC := ExistBlock( "STFNDNCC" ) // PE para customização da consulta NCC.
32
33 DEFAULT cCliente := ""
34 DEFAULT cLoja := ""
35 DEFAULT lImport := .F.
36 DEFAULT cOrc := ""
37 DEFAULT aSelReynos := {}
38
39 ParamType 0 var cCliente As Character Default ""
40 ParamType 1 var cLoja As Character Default ""
41
42
43 //Entra no conceito de NCC se:
44 //1 - Parametro MV_USACRED habilitado
45 //2 - Cliente e loja da venda diferente de padrao
46 If Substr(SuperGetMV("MV_USACRED"),2,1) == "S" .AND. (cCliente + cLoja <> cCliPad + cLojaPad)
47
48     if lSTFNDNCC
49         LjGrvLog(cCliente, "chamada do PE STFNDNCC")
50         aNCCs := ExecBlock("STFNDNCC", .F., .F., {cCliente, cLoja, dDataBase, lImport, cOrc})
51     Else
52         aNCCs := FrtNCCLoad(cCliente,cLoja,/*cEnvelope*/, dDataBase,lImport,cOrc, aSelReynos)

```

```

53     Endif
54
55
56     If Len(aNCCs) > 0 //Ordena por E1_EMISSAO+E1_PREFIXO+E1_NUM+E1_PARCELA
57         aSort(aNCCs, ,, { |x,y| x[4]< y[4] .And. x[9]< y[9] .And. x[3]< y[3] .And. x[10]< y[10]})
58     EndIf
59 EndIf
60
61 Return aNCCs
62
63
64 //-----
65 /* {Protheus.doc} STDSetNCCs
66 Seta o Array das NCCs que foi buscado na retaguarda do cliente selecionado
67
68 @param   cType: 1 - Retorna o array com as NCCs. 2 - Retorna o valor total das NCCs selecionadas
69 @param   xValue Recebe valor
70 @return  aNCCs - retorna NCCs em aberto do cliente
71 @obs
72 @sample
73 */
74 //-----
75 Function STDSetNCCs(cType,xValue)
76
77 DEFAULT cType   := ""
78 DEFAULT xValue  := Nil
79
80 If cType == "1"
81     If xValue <> Nil
82         aNCCs := xValue
83     Else
84         aNCCs := {}
85     EndIf
86 ElseIf cType == "2"
87     If xValue <> Nil
88         nTotalNCCs += xValue
89     Else
90         nTotalNCCs := 0
91
92         //Retira a selecao das NCCs
93         aEval( aNCCs, { |x| x[1] := .F. } )
94     EndIf
95 EndIf
96
97 /*
98 Posicoes de aNCCs
99
100 aNCCs[x,1] = .F. // Caso a NCC seja selecionada, este campo recebe TRUE
101 aNCCs[x,2] = SE1->E1_SALDO
102 aNCCs[x,3] = SE1->E1_NUM
103 aNCCs[x,4] = SE1->E1_EMISSAO
104 aNCCs[x,5] = SE1->(Recno())

```

```

105 aNCCs[x,6] = SE1->E1_SALDO
106 aNCCs[x,7] = SuperGetMV("MV_MOEDA1")
107 aNCCs[x,8] = SE1->E1_MOEDA
108 aNCCs[x,9] = SE1->E1_PREFIXO
109 aNCCs[x,10] = SE1->E1_PARCELA
110 aNCCs[x,11] = SE1->E1_TIPO
111
112 */
113
114 Return aNCCs
115
116
117 //-----
118 /*{Protheus.doc} STDGetNCCs
119 Retorna o Array das NCCs do cliente preenchido.
120
121 @param cType: 1 - Retorna o array com as NCCs. 2 - Retorna o valor total das NCCs selecionadas
122 @return xRet - retorna arrays de nccs ou valor da ncc
123 @obs
124 @sample
125 */
126 //-----
127 Function STDGetNCCs(cType)
128
129 Local xRet := Nil // Retorno
130
131 DEFAULT cType := ""
132
133 If nTotalNCCs < 0
134     nTotalNCCs := 0
135 EndIf
136
137 If cType == "1"
138     /*
139     Posicoes de aNCCs
140
141     aNCCs[x,1] = .F. // Caso a NCC seja selecionada, este campo recebe TRUE
142     aNCCs[x,2] = SE1->E1_SALDO
143     aNCCs[x,3] = SE1->E1_NUM
144     aNCCs[x,4] = SE1->E1_EMISSAO
145     aNCCs[x,5] = SE1->(Recno())
146     aNCCs[x,6] = SE1->E1_SALDO
147     aNCCs[x,7] = SuperGetMV("MV_MOEDA1")
148     aNCCs[x,8] = SE1->E1_MOEDA
149     aNCCs[x,9] = SE1->E1_PREFIXO
150     aNCCs[x,10] = SE1->E1_PARCELA
151     aNCCs[x,11] = SE1->E1_TIPO
152     */
153
154     //Verifica aNCCs esta definido como Array
155     If ValType(aNCCs) == "A"
156         xRet := aNCCs

```

```
157     Else
158         LjGrvLog("Retorno NCC", "Variavel aNCCs nao esta definida como Array.aNCCs:",aNCCs)
159         xRet := {}
160     EndIf
161
162 ElseIf cType == "2"
163     xRet := nTotalNCCs
164 EndIf
165
166 Return xRet
```

Componentes/NCCs/STWBxNCC.prw

131 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /* {Protheus.doc} STWBxNCC
6 Realiza a baixa das NCCs selecionadas
7
8 @param
9 @return      lRet - Retorna se realizou a baixa
10 @obs
11 @sample
12 */
13 //-----
14
15 Function STWBxNCC()
16
17 Local aNCCs      := STDGetNCCs("1")           // NCC selecionadas
18 Local aParam     := {}                       // Array com os parametros que serao passados para o STBRemoteExecute
19 Local cL1Doc     := STDGPBasket("SL1","L1_DOC") // Valor do campo L1_DOC
20 Local cL1Serie   := STDGPBasket("SL1","L1_SERIE") // Valor do campo L1_SERIE
21 Local cSerEst    := ""                      // Serie da estacao, essa informação sera gravada no prefixo da E1
22 Local cMvLjPref  := SuperGetMV("MV_LJPREF")   // Regra para gravacao do E1_PREFIXO
23 Local cL1Oper    := STDGPBasket("SL1","L1_OPERADO") // Valor do campo L1_DOC
24 Local cL1Cliente := STDGPBasket("SL1","L1_CLIENTE") // Valor do campo L1_CLIENTE
25 Local cL1Loja    := STDGPBasket("SL1","L1_LOJA") // Valor do campo L1_LOJA
26 Local dL1EmisNf  := STDGPBasket("SL1","L1_EMISNF") // Valor do campo L1_EMISNF
27 Local oTotal     := STFGetTot()              // Recebe o objeto do model para recuperar os valores
28 Local nValorUsado := STDGetNCCs("2")         // Valor total das NCCs utilizadas
29 Local lOnLine    := .F.
30 Local nMVLjNCCBC := SuperGetMv("MV_LJNCCBC")
31 Local aProfile   := {}
32
33 /*
34      nNccGerada = Valor da NCC gerada ((Total da venda - Desconto total) - Valor total das NCCs utilizadas
35 */
36 Local nNccGerada := oTotal:GetValue("L1_VLRTOT") - nValorUsado
37 Local nL1Credit  := nValorUsado
38 Local uResult    := Nil
39 Local nTPCompNCC := SuperGetMV("MV_LJCPNCC",,1) // Tratamento para compensacao de NCC 1 - INCLUSAO DE NOVO TITULO |2 - ALTERACAO DO SALDO |3 - BAIXA TOTAL DA NCC |4 -
      SALDO DA NCC COM TROCO
40 Local aSelNum    := STWNumTit() //Retorna o que sera gravado no E1_NUM e E1_SERIE
41
42 //Tratamento para geração de NCC
43 If nTPCompNCC == 2
44     If nValorUsado > oTotal:GetValue("L1_VLRTOT")
45         nNccGerada := nValorUsado - oTotal:GetValue("L1_VLRTOT")
46     Else
47         nNccGerada := 0
48     EndIf
49 EndIf
50
51 cL1Doc := aSelNum[1]

```



```

52 cL1Serie := aSelNum[2]
53
54 //Como no PDV eu ainda não tenho a SF2 pego a serie da SL1
55 If AllTrim(Upper(cMvLjPref)) == "SF2->F2_SERIE"
56     cSerEst := cL1Serie
57 Else
58     cSerEst := &(cMvLjPref)
59 EndIf
60
61 LjGrvLog( "L1_NUM: " + cL1Doc, "Baixa NCC", /*xVar*/ ) //Grava Log =====
62
63 STBRemoteExecute( "STFCOMMUOK", NIL, NIL, .F., @lOnline )
64
65 LjGrvLog( "L1_NUM: " + cL1Doc, "NCC Gerada PDV", nNccGerada ) //Grava Log =====
66 LjGrvLog( "L1_NUM: " + cL1Doc, "MV_LJNCCBC", cMVLjNCCBC ) //Grava Log =====
67 LjGrvLog( "L1_NUM: " + cL1Doc, "Comunicação STFCOMMUOK ", lOnline ) //Grava Log =====
68
69 If !lOnline .AND. cMVLjNCCBC == "3" // MV_LJNCCBC = 1 - Online apenas
70
71     STFMessage(ProcName(),"STOP","Falha de comunicação. Não será possível utilizar a Nota de Crédito.") // "Falha de comunicação. Não será possível utilizar a Nota de
72     STFSHOWMessage(ProcName())
73
74 Else
75
76     If !lOnline .AND. cMVLjNCCBC == "2" // Solicita senha do supervisor para liberacao de NCC offline.
77
78         aProfile := STFProFile(36)
79
80     EndIf
81
82     If Len(aProfile) == 0 .OR. aProfile[1]
83
84         aParam := {aNCCs      , nValorUsado, nNccGerada, cL1Doc      ,;
85                   cSerEst    , cL1Serie   , cL10per    , dL1EmisNf,;
86                   cL1Cliente, cL1Loja    , nL1Credit , !lOnline}
87
88         STBRemoteExecute("STWBOBxNCC", aParam,,.T., @uResult)
89
90         LjGrvLog( "L1_NUM: " + cL1Doc, "Conseguiu dar baixa na NCC PDV ", uResult ) //Grava Log =====
91
92     EndIf
93
94
95 EndIf
96
97 Return
98
99
100 //-----
101 /*/ {Protheus.doc} STWNumTit
102 Funcao para decidir se na geracao da compensacao da NCC (SE1) sera

```

```

103 utilizado do L1_DOC ou L1_DOCPED para o E1_NUM
104
105 @param
106 @return      lRet - Retorna se realizou a baixa
107 @obs
108 @sample
109 /*/
110 //-----
111 Function STWNumTit()
112 Local cMvLjE1Num := SuperGetMV("MV_LJE1NUM",,"1") //Permite configurar como sera gerado o numero do tiulo da venda (campo E1_NUM)
113 Local cE1Num := "" //Numero para geracao do titulo na baixa da NCC
114 Local cE1Ser := "" //Serie para geracao do titulo na baixa da NCC
115
116 Do Case
117     Case cMvLjE1Num == "1" //Opcao 1: Default - Comportamento Padrao do Sistema (para manter o legado)
118
119         cE1Num := If(Empty(STDGPBasket("SL1","L1_DOCPED")),STDGPBasket("SL1","L1_DOC"),STDGPBasket("SL1","L1_DOCPED"))
120         cE1Num := If(Empty(cE1Num) .And. !Empty(STDGPBasket("SL1","L1_DOCRPS")),STDGPBasket("SL1","L1_DOCRPS"),cE1Num)
121         cE1Ser := If(Empty(STDGPBasket("SL1","L1_SERPED")),STDGPBasket("SL1","L1_SERIE"),STDGPBasket("SL1","L1_SERPED"))
122         cE1Ser := If(Empty(cE1Ser) .And. !Empty(STDGPBasket("SL1","L1_SERRPS")),STDGPBasket("SL1","L1_SERRPS"),cE1Ser)
123
124     Case cMvLjE1Num == "2" //Opcao 2: Considera o Numero do Orcamento ou Numero do Orcamento Pai quando existir Pedido (L1_NUM)
125
126         cE1Num := STDGPBasket("SL1","L1_NUM")           //Numero do orcamento
127         cE1Ser := STDGPBasket("SL1","L1_SERIE")
128
129 EndCase
130
131 Return {cE1Num,cE1Ser}

```

Componentes/NotFiscal/STBNotFiscal.prw

922 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBNOTFISCAL.CH"
4 #INCLUDE "AUTODEF.CH"
5
6 #DEFINE ARQLJINI GetClientDir()+"SIGALOJA.INI"      // caminho do ini do SIGALOJA
7
8 Static lMVLJPRDSV := SuperGetMv("MV_LJPRDSV",.F.,.F.) // Verifica se esta ativa a implementacao de venda com itens de "produto" e itens de "servico" em Notas Separadas (RPS)
9
10 //-----
11 /* {Protheus.doc} STBSumNotFiscal
12 Adiciona valor no totalizador não fiscal
13
14 @param  nValue      Valor
15 @return Nil
16 @obs
17 @sample
18 */
19 //-----
20 Function STBSumNotFiscal( nValue, lIsRPS )
21
22 Local oTotal := STFGetTot()      // Totalizador
23
24 Default nValue      := 0
25 Default lIsRPS      := .F.
26
27 ParamType 0 Var  nValue As Numeric  Default 0
28
29 oTotal:SetValue( "L1_NOTFISCAL" , nValue + oTotal:GetValue("L1_NOTFISCAL") )
30
31 If lIsRPS
32     //Adiciona valor do serviço (RPS) no totalizador não fiscal do RPS
33     STBSumVRPS( nValue )
34 EndIf
35
36 Return Nil
37
38
39 //-----
40 /* {Protheus.doc} STBFactor
41 Calcula fatores Fiscal e Não-Fiscal
42
43 @return aRet - Fatores fiscais e nao fiscais
44 @obs
45 @sample
46 */
47 //-----
48 Function STBFactor(lUseServ)
49
50 Local nFiscalFactor      := 0      // Fator Fiscal
51 Local nNotFiscalFactor   := 0      // Fator Não Fiscal
52 Local nTotFiscal         := 0      // Total Fiscal

```

```

53 Local oTotal          := STFGetTot()           // Totalizador
54 Local nTotNaoFisc     := oTotal:GetValue("L1_NOTFISCAL") // Totalizador Nao Fiscal
55 Local lItEntrega      := STBTemEntr()         // Indica se a venda tem item de entrega
56 Local nTotalVda       := 0                   // Total da venda
57 Local nTotServic      := 0                   // Total de Itens de serviço (RPS)
58 Local nPedFactor      := 1                   // Fator Não Fiscal (Pedido)
59 Local nRPSFactor      := 1                   // Fator Não Fiscal (RPS)
60
61 Default lUseServ       := .T.                 // Controla se considera servicos no totalizador nao fiscal
62
63 If lMVLJPRDSV
64     If oTotal:HasField("L1_NOTFISRP") //Verifica se o campo está na estrutura do Model
65         nTotServic := oTotal:GetValue("L1_NOTFISRP")
66     Else
67         nTotServic := STFTotalServ()
68     EndIf
69 EndIf
70
71 nTotalVda := oTotal:GetValue("L1_VLRTOT") + oTotal:GetValue("L1_DESCONT") - oTotal:GetValue("L1_ACRSFIN")
72
73 /*-----
74 Adiciona os valores de Frete + Seguro + Despesa no total não-fiscal quando existir
75 item de entrega na venda, para o fator fiscal e não-fiscal ficarem corretos.
76 -----*/
77 If lItEntrega
78     nTotNaoFisc := nTotNaoFisc + oTotal:GetValue("L1_FRETE") + oTotal:GetValue("L1_SEGURO") + oTotal:GetValue("L1_DESPESA")
79 EndIf
80
81 //Se nao considera servicos no totalizador nao fiscal, armazena os valores de servicos
82 If !lUseServ
83     nTotNaoFisc -= nTotServic
84 EndIf
85
86 nNotFiscalFactor := (nTotNaoFisc / nTotalVda)
87 If nTotServic > 0
88     nRPSFactor := (nTotServic / nTotalVda)
89 EndIf
90 If nTotNaoFisc > 0
91     nPedFactor := (nTotNaoFisc-nTotServic) / nTotalVda
92 else
93     nPedFactor := nNotFiscalFactor
94 EndIf
95 nFiscalFactor := (1 - nNotFiscalFactor)
96 nTotFiscal := (nTotalVda - nTotNaoFisc)
97
98 nFiscalFactor := IIf(nFiscalFactor == 0, 1, nFiscalFactor)
99
100 aRet := { nFiscalFactor, nNotFiscalFactor, nTotFiscal, nPedFactor, nRPSFactor }
101
102 Return aRet
103
104 //-----

```

```

105 /*/ {Protheus.doc} STBPrintNotFiscal
106 Imprime cupom nao fiscal
107
108 @param
109 @return aRet[1] - Retorna se executou corretamente
110 @return aRet[2] - Numero do cupom
111 @return aRet[3] - Numero do PDV
112 @obs
113 @sample
114 /*/
115 //-----
116 Function STBPrintNotFiscal()
117
118 Local aRet          := { .T. , "" , "" }           // Retrono Função
119 Local oTotal        := STFGetTot()                 // Totalizador
120 Local aPrinter      := {}                          // Armazena Retorno da impressora
121 Local cText         := ""                          // Texto para impressão
122 Local cValue        := ""                          // Valor para impressão
123 Local lContinue     := .T.                         // Fluxo funcao
124 Local aCupom        := {Space(6), Nil}             // Retorno Evento GetCupom
125 Local aPDV          := {""}                        // Retorno evento pegar PDV
126 Local cCupom        := ""                          // Retorno número do Cupom
127 Local cPDV          := ""                          // Armazena PDV
128 Local cUseTef       := ""                          // Indica se utiliza tef. "S" ou "N"
129 Local cPayTef       := ""                          // Condição de pagamento TEF
130 Local lGuil         := SuperGetMV("MV_FTTEFGU",, .T.) // Ativa guilhotina
131 Local lIsImpOrc      := If(FindFunction("STBIsImpOrc"),STBIsImpOrc(),!(Empty(STDGPBasket("SL1","L1_NUMORIG")))) //Verifica se é importação de orçamento
132 Local cSerieRPS     := SuperGetMv("MV_LOJARPS",,"RPS") // Serie da NF de Servico RPS configurada no parametro MV_LOJARPS
133 Local lPedido       := STBTemEntr()                // Verifica se tem item de "Entrega" ou "Retira Posterior".
134 Local lRPS          := lMVLJPRDSV .And. (lIsImpOrc .Or. (ExistFunc("Lj7RPSNew") .And. Lj7RPSNew() .And. !STFGetCfg("lUseECF"))) .And. STBTemServ() // Verifica se tem item de
    "Servico".
135 Local aNotas        := {}
136 Local lTemItProd    := .F.                         // item fiscal
137 Local lTemItServ    := .F.                         // item de servico
138 Local lIsValePres   := .F.                         //Verifica se é vale presente
139 Local lExistSer     := .F.                         // verifica se existe a Serie Não Fiscal
140 Local lSFinanc      := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) //Se .T. indica que o Serviço financeiro esta ativo
141 local lFindExist    := Iif(ExistFunc("LjFindServ"),Iif(LjFindServ(STDGPBasket("SL1","L1_NUM")),.T.,.F.),.F. ) //Verifico se a função LjFindServ existe e se existe verifico se
    todos os itens são serviço
142 Local lLjVpCnf      := SuperGetMV("MV_LJVPCNF",,.F.) //verifica se imprime vale presente no cupom fiscal
143 Local lOnlyServ     := .F.
144 Local nPrintNFis    := 0
145 Local cTxtCpvPED     := "" //Texto do comprovante nao fiscal "SCRPED"
146 Local cTxtCpvRPS     := "" //Texto do comprovante nao fiscal "SCRPRS"
147 Local aComprovan    := {}
148 Local nI            := 0
149
150 cSerieRPS := PadR(cSerieRPS , TamSX3("L1_SERRPS")[1])
151
152 If !STFGetCfg("lUseECF") .AND. STBExistNItemFiscal()
153
154     If lRPS //RPS (Recibo Provisorio de Servico)

```

```

155
156         cTxtCpvRPS := LJSCRPS( {.F., {}, STBFactor()[5]} ) //No array, elem. 2, fator do comprovante não-fiscal
157         cTxtCpvRPS := StrTran( cTxtCpvRPS, ',', '.' )
158
159         If !Empty(cTxtCpvRPS)
160             aAdd( aComprovan, cTxtCpvRPS )
161         EndIf
162
163     EndIf
164
165     If (!lRPS .Or. lPedido)
166
167         cTxtCpvPED := LJSCRPED( {"" , STBFactor()[4]} ) //No array, elem. 2, fator do comprovante não-fiscal
168         cTxtCpvPED := StrTran( cTxtCpvPED, ',', '.' )
169
170         If !Empty(cTxtCpvPED)
171             aAdd( aComprovan, cTxtCpvPED )
172         EndIf
173
174     EndIf
175
176     If lRPS .Or. !Empty(cTxtCpvPED)
177
178         //Verifica se é vale presente:
179         lIsValePres := iIf(FindFunction("STDExistVP"),STDExistVP(),.F.)
180
181         If lPedido .Or. lIsValePres //Pedido ou Vale presente
182             cSerie := STFGetStation("SERNFIS")
183
184             lExistSer:= LjxDNota( cSerie, 3, .F., 1, @aNotas,,,,,,,,, "DOCNF" ) // DOC/SERIE
185             If lExistSer
186                 cCupom := aNotas[1][2]
187
188                 STDSPBasket( "SL1" , "L1_DOCPED" , cCupom )
189                 STDSPBasket( "SL1" , "L1_SERPED" , cSerie )
190                 STDSPBasket( "SL1" , "L1_SITUA" , "00" )
191                 /* Se o campo L1_DOC estiver vazio, entao deve-se limpar o campo L1_SERIE
192                    melhoria: o campo L1_SERIE só deve ser preenchido quando a venda for fiscal */
193                 If lPedido .Or. (lIsValePres .And. !lLjVpCnf)
194                     If !STBExistItemFiscal()
195                         STDSPBasket( "SL1", "L1_DOC", "" )
196                         STDSPBasket( "SL1", "L1_SERIE", "" )
197                     EndIf
198                 EndIf
199             Else
200                 aRet := { .F. , "" , "" }
201             Endif
202         EndIf
203
204         If aRet[1] .And. lRPS //RPS (Recibo Provisorio de Servico)
205             cSerie := cSerieRPS // Serie da NF de Servico RPS configurada no parametro MV_LOJARPS
206

```

```

207      STDSPBasket( "SL1" , "L1_SERRPS"      , cSerie          ) // Numero de serie da nota fiscal de servico (RPS)
208      STDSPBasket( "SL1" , "L1_SITUA"      , "00"              )
209
210      LjCheckRPS(STDGPBasket("SL1","L1_NUM"),@lTemItProd,@lTemItServ)
211      lOnlyServ := lTemItServ .And. !lTemItProd .And. !Empty(cSerie)
212
213      If lRPS .And. !Empty(cTxtCpvPED) .And. !Empty(cSerie) .And. lTemItServ //Quando for entrega e serviço na mesma venda.
214          lOnlyServ := .T.
215          aNotas := {}
216      EndIf
217
218      If lOnlyServ //A venda possui apenas itens de Servico
219          lExistSer := LjxDNota( cSerie, 3, .F., 1, @aNotas ,,,,,,,,,,"RPS") // DOC/SERIE
220          If lExistSer
221              cCupom := aNotas[1][2]
222              STDSPBasket( "SL1" , "L1_DOC"      , ""              )
223              STDSPBasket( "SL1" , "L1_SERIE"    , ""              )
224              STDSPBasket( "SL1" , "L1_DOCRPS"   , cCupom          )
225          Else
226              aRet := { .F. , "" , "" }
227          EndIf
228      ElseIf Empty(STDGPBasket( "SL1" , "L1_DOC")) //Se DOC estiver vazio é porque na venda só tem produtos de servico (RPS)
229          STDSPBasket( "SL1" , "L1_DOC"      , cCupom          )
230          STDSPBasket( "SL1" , "L1_SERIE"    , cSerie          )
231          STDSPBasket( "SL1" , "L1_DOCRPS"   , cCupom          )
232      EndIf
233  EndIf
234
235  If aRet[1]
236      For nI := 1 To Len(aComprovan)
237          cText := aComprovan[nI]
238          nPrintNFis := STWPrintTextNotFiscal(cText)
239          If nPrintNFis <> 0
240              aRet := { .F. , "" , "" }
241              Exit
242          EndIf
243          If lGuil
244              cText := Replic(CHR(10)+CHR(13),6)
245              cText += TAG_GUIL_INI+TAG_GUIL_FIM //Corte de Papel
246              STWPrintTextNotFiscal(cText)
247          EndIf
248      Next nI
249  EndIf
250
251  If aRet[1]
252      aRet := { .T. , cCupom , STFGetStation("PDV") }
253  EndIf
254
255  EndIf
256
257  ElseIf STFUseFiscalPrinter() .AND. oTotal:GetValue("L1_NOTFISCAL") > 0 // Utiliza impressora fiscal e possui itens não fiscais
258

```

```

259 If STBValPrintNotFiscal(If(lPedido,"PED",If(lRPS,"RPS","")))
260
261     If lPedido //Pedido
262
263         cText := LJSCRPED( {"",STBFactor()[4]} ) //No array, elem. 2, fator do comprovante não-fiscal
264
265     ElseIf lRPS //RPS (Recibo Provisorio de Servico)
266
267         cText := LJSCRPS( {.F., {} , STBFactor()[5]} ) //No array, elem. 2, fator do comprovante não-fiscal
268
269     EndIf
270
271     cText := StrTran( cText, ',','.' )
272
273     // Se não tiver item fiscal na venda não multiplica total (L1_VLRLIQ) pelo fator fiscal/não-fiscal
274     If Empty(STDGPBasket( "SL1" , "L1_DOC"))
275         cValue := Str(( oTotal:GetValue("L1_VLRTOT") ) , 15 , 2 )
276     Else
277         cValue := Str(( oTotal:GetValue("L1_VLRTOT") * STBFactor()[2] ) , 15 , 2 )
278     EndIf
279
280     /*/
281     Imprime Não Fiscal
282     /*/
283     aPrinter := STFFireEvent( ProcName(0)
284                             "STSalesOrder"
285                             { cUseTef
286                             cText
287                             cValue
288                             cPayTef
289                             } )
290
291     If ValType(aPrinter) <> "A" .OR. Len(aPrinter) == 0 .OR. aPrinter[1] <> 0
292         lContinue := .F.
293     EndIf
294
295     /*/
296     Pega o numero do Comprovante Nao Fiscal
297     /*/
298     If lContinue
299         aPrinter := STFFireEvent( ProcName(0) , "STGetReceipt" , aCupom )
300         If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aCupom) < 1
301             If lPedido //Pedido
302                 MsgStop(STR0001) // "Erro ao pegar o número do cupom após a impressão do pedido. Verifique o ECF."
303             ElseIf lRPS //RPS (Recibo Provisorio de Servico)
304                 MsgStop(STR0009) // "Erro ao pegar o número do cupom após a impressão do comprovante RPS (Recibo Provisório de Serviço). Verifique o ECF."
305             EndIf
306             lContinue := .F.
307         Else
308             cCupom := aCupom[1]
309         EndIf
310     EndIf

```



```

311 //-----
312 cCupom := StrZero( Val(cCupom) - 1 , Len(AllTrim(cCupom)) , 0 )
313
314 //Se a impressão do comprovante não foi um relatório gerencial, então pode-se alterar
315 //a variável cNumCupom. Sem esta validação, poderá causar um erro de chave duplicada no Banco de Dados,
316 //pois o número título financeiro poderá se repetir.
317
318 // TODO: Verificar
319 /*
320 If !lRelGer .OR. Val(cValor) == 0
321     If STFGetCfg("lEcArg")
322         cNumCupom:=StrZero(Val(cNumCupom)-1,Len(AllTrim(cNumCupom)) ,0)
323     EndIf
324 EndIf
325 */
326 //-----
327
328 /*/
329     Get PDV
330 /*/
331 If lContinue
332     aPrinter := STFFireEvent( ProcName(0) , "STGetPDV" , aPDV )
333     If ValType(aPrinter) <> "A" .OR. Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aPDV) < 1
334         If lPedido //Pedido
335             MsgStop(STR0002) // "Erro ao pegar o número do PDV após a impressão do pedido. Verifique o ECF."
336         ElseIf lRPS //RPS (Recibo Provisorio de Serviço)
337             MsgStop(STR0010) // "Erro ao pegar o número do PDV após a impressão do comprovante RPS (Recibo Provisório de Serviço). Verifique o ECF."
338         EndIf
339         lContinue := .F.
340     Else
341         cPDV := aPDV[1]
342     EndIf
343 EndIf
344
345 If lContinue
346
347     aRet := { .T. , cCupom , cPDV }
348
349     //Verifica se é vale presente:
350     lIsValePres := iIf(FindFunction("STDExistVP"),STDExistVP(),.F.)
351
352     /*/
353     Atualiza Cesta
354     /*/
355     If lPedido .Or. lIsValePres //Pedido
356         cSerie := STFGetStat("SERIE")
357
358         STDSPBasket( "SL1" , "L1_DOCPED" , cCupom )
359         STDSPBasket( "SL1" , "L1_SERPED" , cSerie )
360         STDSPBasket( "SL1" , "L1_SITUA" , "00" )
361         /* Se o campo L1_DOC estiver vazio, então deve-se limpar o campo L1_SERIE
362            melhoria: o campo L1_SERIE só deve ser preenchido quando a venda for fiscal */

```

```

363         If lPedido .AND. !STBExistItemFiscal()
364             STDSPBasket( "SL1", "L1_DOC", "" )
365             STDSPBasket( "SL1", "L1_SERIE", "" )
366         EndIf
367     ElseIf lSfinanc .AND. lFindExist //Serviço Financeiro a avulso
368         cSerie := STFGetStat("SERIE")
369
370         STDSPBasket( "SL1" , "L1_DOCRPS", cCupom      )
371         STDSPBasket( "SL1" , "L1_SERRPS", cSerie       )
372         STDSPBasket( "SL1" , "L1_SITUA" , "00"         )
373
374     ElseIf lRPS //RPS (Recibo Provisorio de Servico)
375         cSerie := cSerieRPS // Serie da NF de Servico RPS configurada no parametro MV_LOJARPS
376
377         STDSPBasket( "SL1" , "L1_SERRPS" , cSerie      ) // Numero de serie da nota fiscal de servico (RPS)
378         STDSPBasket( "SL1" , "L1_SITUA" , "00"         )
379
380         LjCheckRPS(STDGPBasket("SL1","L1_NUM"),@lTemItProd,@lTemItServ)
381         lOnlyServ := lTemItServ .And. !lTemItProd .And. !Empty(cSerie)
382         If lOnlyServ //A venda possui apenas itens de Servico
383             STDSPBasket( "SL1" , "L1_DOC" , ""         )
384             STDSPBasket( "SL1" , "L1_SERIE" , ""        )
385             STDSPBasket( "SL1" , "L1_DOCRPS", cCupom    )
386         ElseIf Empty(STDGPBasket( "SL1" , "L1_DOC")) //Se DOC estiver vazio é porque na venda só tem produtos de servico (RPS)
387             STDSPBasket( "SL1" , "L1_DOC" , cCupom     )
388             STDSPBasket( "SL1" , "L1_SERIE" , cSerie    )
389             STDSPBasket( "SL1" , "L1_DOCRPS", cCupom    )
390         EndIf
391     EndIf
392
393     Else
394         aRet := { .F. , "" , "" }
395     EndIf
396
397     Else
398         aRet := { .F. , "" , "" }
399     EndIf
400
401 Else
402     aRet := { .T. , "" , "" }
403 EndIf
404
405 Return aRet
406
407
408 //-------------------------------------
409 /* {Protheus.doc} STBValPrintNotFiscal
410 Adiciona valor no totalizador não fiscal
411
412 @param
413 @return lRet          Retorna se pode imprimir o cupom nao fiscal
414 @obs

```

```

415 @sample
416 */
417 //-----
418 Function STBValPrintNotFiscal(cOpc)
419
420 Local lRet           := .T.           // Retorno da função
421 Local aRet           := {}           // Armazena Retorno da impressora
422 Local aIsOpenReceipt := { "5" , "" } // Armazena retorno se o cupom está aberto
423
424 Default cOpc         := ""
425
426
427 /*/
428   Valida Data/Hora
429 /*/
430 aRet := STFFireEvent(  ProcName(0)           ,; // Nome do processo
431                       "STCheckDate"         ,; // Nome do evento
432                       {}                     )
433
434 If Len(aRet) > 0 .AND. ValType(aRet[1]) == "L" .AND. aRet[1]
435   lRet := .T.
436 Else
437   lRet := .F.
438   MsgStop(STR0003) // TODO //"Impossível continuar impressão. Ajustar Data/Hora da impressora."
439 EndIf
440
441
442 /*/
443   Verifica cupom aberto
444 /*/
445 If lRet
446   aRet := STFFireEvent(  ProcName(0)           ,; // Nome do processo
447                       "STPrinterStatus"       ,; // Nome do evento
448                       aIsOpenReceipt           )
449
450   If Len(aIsOpenReceipt) >= 2 .AND. ValType(aIsOpenReceipt[2]) == "N"
451     If aIsOpenReceipt[2] == 7
452       nRet := IFCancCup( nHdlECF ) // TODO: Criar evento cancelamento
453       If Lj7VerCmd( nRet ) // TODO: Mudar para <> 0
454         /*/
455           Espera um tempo para a impressora fazer a impressao do cancelamento
456         /*/
457         Inkey(8)
458       EndIf
459     EndIf
460   EndIf
461 EndIf
462
463
464 /*/
465   Valida RdMake Impressao
466 /*/

```

```

467 If lRet
468     If Empty(c0pc) .Or. c0pc == "PED"
469         If !ExistBlock("SCRPED") .AND. !ExistFunc("LJSCRPED")
470             MsgStop(STR0004) //"0 rdmake SCRPED.PRW não está compilado e não será possível imprimir o comprovante de venda. Informe essa mensagem ao administrador do
                sistema."
471             lRet := .F.
472         EndIf
473     ElseIf c0pc == "RPS"
474         If !ExistBlock("SCRPPS") .AND. !ExistFunc("LJSCRPS")
475             MsgStop(STR0011) //"0 rdmake SCRPPS.PRW não está compilado e não será possível imprimir o comprovante RPS (Recibo Provisório de Serviço). Informe essa mensagem ao
                administrador do sistema."
476             lRet := .F.
477         EndIf
478     EndIf
479 EndIf
480
481 Return lRet
482
483
484 //-----
485 /*{Protheus.doc} STBCheckFiscalItens
486 Verifica se cupom fiscal está aberto e se existem itens fiscais para o cupom.
487 Se o cupom estiver aberto mas não existir itens, cancela o cupom.
488
489 @param
490 @return lRet - Successful Check
491 @obs
492 @sample
493 */
494 //-----
495 Function STBCheckFiscalItens()
496
497 Local oTotal      := STFGetTot() // Totalizador
498 Local cNumDoc     := ""         //Numero do CF
499 Local cSupervisor := ""
500 Local lRet        := .T.        //Execução com sucesso - realiza o prosseguimento da venda
501
502 If STWGetIsOpenReceipt() .AND. !( STBExistItemFiscal(.F.) ) .AND. ( oTotal:GetVaLue("L1_NOTFISCAL") > 0 )
503     /*/
504     Cancela o Cupom Fiscal, pois os itens registrados no cupom fiscal foram cancelados
505     /*/
506
507     //Envia o comando de cancelamento do cupom fiscal
508
509     cNumDoc := STDGPBasket( "SL1" , "L1_DOC")
510
511
512     lRet := STBSCancCupPrint( cSupervisor , cNumDoc )
513
514     If lRet
515         STWSetIsOpenReceipt( .F. )
516     EndIf

```

```

517 EndIf
518
519 Return lRet
520
521
522 //-----
523 /*{Protheus.doc} STBExistItemFiscal
524 Verifica se existe item fiscal na venda
525
526 @param      lDeleted - Não Verifica registros deletados (.t.)
527 @return      lExist      Retorna se existe item fiscal na venda
528 @obs
529 @sample
530 */
531 //-----
532 Function STBExistItemFiscal(lDeleted)
533
534 Local lExist      := .F.                // Retorno função
535 Local aLines      := FwSaveRows()
536 Local oModelCesta := STDGPBModel()      // Armazena model da cesta
537 Local oModelItens := oModelCesta:GetModel("SL2DETAIL") // Model Itens
538 Local nI          := 0                  // Contador
539
540 Default lDeleted := .T. //Considera registros deletados (default)
541
542 For nI := 1 To oModelItens:Length()
543
544     oModelItens:GoLine(nI)
545
546     If (!oModelItens:IsDeleted() .or. lDeleted) .AND. oModelItens:GetValue("L2_FISCAL")
547
548         lExist := .T.
549         Exit
550
551     EndIf
552
553 Next nI
554
555 FwRestRows(aLines)
556
557 Return lExist
558
559 //-----
560 /*{Protheus.doc} STBExistNItemFiscal
561 Verifica se existe item não-fiscal na venda
562
563 @param
564 @return      lExist      Retorna se existe item não-fiscal
565 @obs
566 @sample
567 */
568 //-----

```

```

569 Function STBExistNItemFiscal()
570
571 Local lExist          := .F.                // Retorno função
572 Local oModelCesta     := STDGPBModel()      // Armazena model da cesta
573 Local oModelItens     := oModelCesta:GetModel("SL2DETAIL") // Model Itens
574 Local nI              := 0                // Contador
575
576 For nI := 1 To oModelItens:Length()
577
578     If !oModelItens:IsDeleted(nI)
579         If !oModelItens:GetValue( "L2_FISCAL" , nI )
580             lExist := .T.
581             Exit
582         EndIf
583     EndIf
584
585 Next nI
586
587 Return lExist
588
589 //-----
590 /*{Protheus.doc} STBIANotFiscal
591 Imprime Cupom não-fiscal de Doação
592 @param nDoacao      Total da doação
593 @param
594 @return
595 @obs
596 @sample
597 */
598 */
599 //-----
600 Function STBIANotFiscal( nDoacao, cSerie, cDoc, cCliente,;
601                          cLojaCli, cCPFCLI, cFormPg )
602
603 Local cTickForm          // Texto do cupom não-fiscal
604 Local cFormaPgto        := ""          // Ler qual cond. pagto. no Sigaloja.ini
605 Local cTotalizNFis      := ""          // Ler qual totalizador no Sigaloja.ini
606 Local lRetorno          := .T.          // Retorna se a impressão foi efetuada com sucesso
607 Local lRecNaoFis := (nModulo == 5 .OR. nModulo == 6) .OR.; //Recebimento pelo Venda Direta
608                      ((nModulo == 12 .OR. nModulo == 23) .AND. ExistFunc("LjEmitNFCe") .AND. LjEmitNFCe()) //Recebimento pelo Loja/Front com NFC-e
609                      //Sinaliza recebimento por modulo não fiscal(sem ECF) *NFC-e, Venda Direta
610 Local cMsgComprovante   := ""          //Mensagem para Comprovante, se NFC-e ou SAT
611 Local lGuil             := ""          //Ativa guilhotina
612
613 Default nDoacao := 0                //Parâmetro do valor da doação
614 Default cSerie  := ""                //Serie
615 Default cDoc    := ""                //Documento
616 Default cCliente := ""                //Código do Cliente
617 Default cLojaCli := ""                //Loja do Cliente
618 Default cCPFCLI := ""                //CPF/CNPJ do Cliente
619 Default cFormPg := ""                //Forma de Pagamento
620

```

```

621 If lRecNaoFis      //Se NFC-e ou SAT
622
623     lGuil           := SuperGetMV("MV_FTTEFGU",, .T.) // Ativa guilhotina
624     cMsgComprovante := Lj950TxtIA(nDoacao, cSerie, cDoc, cCliente;;
625                                 cLojaCli, cCPFCLi, cFormPg)
626     nRet := STWPrintTextNotFiscal(cMsgComprovante)
627     lRetorno := (nRet = 0)
628
629     If lGuil .AND. lRetorno
630         STWPrintTextNotFiscal(TAG_GUIL_INI+TAG_GUIL_FIM)
631     EndIf
632
633 Else                //Se ECF, entrará no STBIAPrinReceipt(), com funções específicas somente para o SIGAFRT.
634
635     cFormaPgto      := GetPvProfString("Instituto Arredondar", "FormaPgto", " ", ARQLJINI ) // Ler qual cond. pagto. no Sigaloja.ini
636     cTotalizNFis    := GetPvProfString("Instituto Arredondar", "Totalizador", "01", ARQLJINI ) // Ler qual totalizador no Sigaloja.ini
637     // cTickform é a descrição do texto.
638     cTickForm := STR0012 + chr(10) + chr(13); // "DOAÇÃO - INSTITUTO ARREDONDAR"
639               + STR0013 + chr(10) + chr(13); // "CNPJ 14.416.996/0001-25"
640               + STR0014                      // "www.arredondar.org.br"
641
642     /*
643     A impressora deverá estar cadastrada como:
644     Forma de Pagamento: A VISTA ou RECEBER, ou o código cadastrado de um deles
645     Totalizador não-fiscal: DOACAO
646     */
647     lRetorno := STBIAPrinReceipt( cFormaPgto, nDoacao, cTotalizNFis, cTickform )
648
649 EndIf
650
651 Return lRetorno
652
653
654 //-----
655 /*{Protheus.doc} STBIAPrinReceipt
656 Imprime o Cabeçalho do Cupom não-fiscal de Doação
657 @param cFormaPagto      Forma de Pagamento
658 @param nValor           Valor da Doação
659 @param cTotalizador     Nome do Totalizador para cupom não-fiscal
660 @param cTexto           Texto informativo
661 @return
662 @obs
663 @sample
664 */
665
666 //-----
667 Function STBIAPrinReceipt(cFormaPgto, nValor, cTotalizador, cTexto)
668
669 Local lContinua := .F. //Laco de continuidade do processo
670 Local lReimprime := .F. //Indicação de reimpressão do comprovante
671 Local lImprimiu := .F. //Impressão do comprovante com sucesso?
672 Local aRet := {} //Retorno do ECF

```

```

673
674 Default cFormaPgto := ""           // Parâmetro 1: Forma de Pagamento cadastrado no ECF
675 Default nValor := 0               // Parâmetro 2: Valor de doação ao Instituto Arredondar
676 Default cTotalizador := ""       // Parâmetro 3: Totalizador cadastrado no ECF
677 Default cTexto := ""             // Parâmetro 4: Texto livre
678
679 lContinua := .T.
680 Do While lContinua
681
682     //Abre cupom fiscal não vinculado
683
684     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
685         "STOpenNotFiscalReceipt";
686         {cFormaPgto;
687             Alltrim(Str(nValor,14,2)); //totalizar nformas de pagamento
688             cTotalizador;
689             cTexto;
690             NIL})
691
692     If !STBIAVerifyPrint(aRet, @lReimprime )
693         //Erro de Impressao e pode reimprimir OU USUÁRIO NÃO TEM PERMISSÃO PARA CANCELAR //LjTEFAskImp(@nOpt,nRet) .OR. (nOpt == 0 .And. !LjSenSupTEF())
694
695         If lReimprime
696             Loop
697         Else
698             lContinua := .F.
699         EndIf
700     EndIf
701
702     If !lContinua
703         Loop
704     Endif
705
706     //bufferizar as linhas do comprovante e enviar conforme contador
707     aRet := STBIAPrinTxtFisc(@cTexto)
708
709     If !STBIAVerifyPrint(aRet, @lReimprime )
710         //Erro de Impressao e pode reimprimir OU USUÁRIO NÃO TEM PERMISSÃO PARA CANCELAR //LjTEFAskImp(@nOpt,nRet) .OR. (nOpt == 0 .And. !LjSenSupTEF())
711
712         If lReimprime
713             Loop
714         Else
715             lContinua := .F.
716         EndIf
717     EndIf
718
719     EndIf
720
721     If !lContinua
722         Loop
723     Endif
724     InKey(2)

```



```

725
726     aRet :=      STFFireEvent(  ProcName(0)                                ,;      // Nome do processo
727                               "STCloseNotFiscalReceipt",;
728                               {}))
729
730     If !STBIAVerifyPrint(aRet, @lReimprime )
731         //Erro de Impressao e pode reimprimir OU USUÁRIO NÃO TEM PERMISSÃO PARA CANCELAR //LjTEFAskImp(@nOpt,nRet) .OR. (nOpt == 0 .And. !LjSenSupTEF())
732         If lReimprime
733             Loop
734         Else
735             lContinua := .F.
736         EndIf
737     Else
738         lImprimiu := .T.
739         lContinua := .F.
740     EndIf
741
742     If !lContinua
743         Loop
744     EndIf
745
746 End
747
748 Return lImprimiu
749
750 //-----
751 /*{Protheus.doc} STBIAVerifyPrint
752 Verifica o status da impressão
753 @param aRet      Retorno do Array
754 @param lReimprime Permissão de reimpressão
755 return lImprime
756 @obs
757 @sample
758 */
759 //-----
760 Function STBIAVerifyPrint(aRet, lReimprime )
761
762 Local lImprime      := .T.      //Impressão com sucesso?
763 Local nOpt          := 2        //Opção selecionada pelo usuário
764 Local lTentativa    := .T.      //controle de tentativa
765
766 Default lReimprime := .F.      //Reimprime o cupom?
767
768 If Valtype(aRet) == "A" .AND. Len(aRet) > 0 .AND. aRet[1] <> 0 //Erro na impressão
769
770     //Erro na Impressão - Trata a mensagem
771     lImprime := .F.
772
773     While lTentativa
774
775         STFMessage("IA_Imprime", "YESNO", STR0005 + " " + STR0006) //"Impressora não responde."#"Deseja imprimir novamente?"
776         nOpt := If(STFShowMessage("IA_Imprime"),2,0)

```

```

777
778      //2=SIM
779      If n0pt == 2
780
781          Sleep(1000)
782
783          aRet := STFFireEvent( ProcName(0)                ,;      // Nome do processo
784                              "STCloseNotFiscalReceipt"    ,;      // Nome do evento
785                              {""} )
786
787          If Valtype(aRet) == "A" .AND. Len(aRet) > 0 .AND. aRet[1] <> 0 //Erro na impressão
788              lTentaTiva := .T.
789          Else
790              lTentaTiva := .F.
791              lReimprime := .F.
792          EndIf
793
794          Else
795              lTentaTiva := .F.
796          EndIf
797
798      EndDo
799
800      If n0pt == 2 // .OR. !Self:ReimpSenSup() //Superior não digitou senha para desfazer a transacao
801          lReimprime := .T.
802          STFMessage("IA", "RUN", STR0007) //"Aguarde a impressão do comprovante não fiscal...."
803          STFShowMessage("IA")
804      EndIf
805
806  Else
807      If ValType(aRet) <> "A"
808          STFMessage("IA_Imprime", "ALERT", STR0008) //"Problemas com a Impressora Fiscal"
809          STFShowMessage("IA_Imprime")
810      EndIf
811  EndIf
812
813  Return lImprime
814
815
816  //-----
817  /*{Protheus.doc} STBIAPrinTxtFisc
818  Retorna o texto do cupom não fiscal
819  @param cTexto      Texto Informativo
820  @return aRet
821  @obs
822  @sample
823  /*
824  //-----
825  Function STBIAPrinTxtFisc(cTexto)
826
827  Local aRet := {} //Array de Retorno
828

```

```

829 Default cTexto := ""           // Texto livre
830
831 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
832                      "STTxtNotFiscalReceipt",;
833                      {cTexto,;
834                      1})
835
836 Return aRet
837
838 //-----
839 /*{Protheus.doc} STBTemEntr
840 Verifica se tem item de "Entrega" ou "Retira Posterior".
841
842 @param
843 @return lExist           Retorna se tem item de "Entrega" ou "Retira Posterior".
844 @obs
845 @sample
846 */
847 //-----
848 Function STBTemEntr()
849 Local lExist             := .F.                // Retorno função
850 Local oModelCesta        := STDGPBModel()      // Armazena model da cesta
851 Local oModelItens        := oModelCesta:GetModel("SL2DETAIL") // Model Itens
852 Local nI                 := 0                  // Contador
853
854 For nI := 1 To oModelItens:Length()
855
856     If !oModelItens:IsDeleted(nI)
857         If (!Empty( oModelItens:GetValue("L2_RESERVA", nI) ) .And. oModelItens:GetValue("L2_ENTREGA", nI) <> "2") .OR. //2=Retira
858             (Empty( oModelItens:GetValue("L2_RESERVA", nI) ) .And. oModelItens:GetValue("L2_ENTREGA", nI) == "5") // 5= ENTREGA C/ PEDIDO S/ RESERVA
859             lExist := .T.
860             Exit
861
862     EndIf
863 EndIf
864
865 Next nI
866
867 Return lExist
868
869 //-----
870 /*{Protheus.doc} STBTemServ
871 Verifica se tem item de "Servico".
872
873 @param
874 @return lExist           Retorna se tem item de "Servico".
875 @obs
876 @sample
877 */
878 //-----
879 Function STBTemServ()
880 Local lExist             := .F.                // Retorno função

```

```

881 Local oModelCesta      := STDGPBModel()           // Armazena model da cesta
882 Local oModelItens      := oModelCesta:GetModel("SL2DETAIL") // Model Itens
883 Local nI                := 0                     // Contador
884
885 For nI := 1 To oModelItens:Length()
886
887     If !oModelItens:IsDeleted(nI)
888         If (Empty(oModelItens:GetValue("L2_ENTREGA", nI)) .Or. oModelItens:GetValue("L2_ENTREGA", nI) == "2") .And. ; //2=Retira
889             LjIsTesISS( oModelItens:GetValue("L2_NUM", nI), oModelItens:GetValue("L2_TES", nI) ) //Verifica se eh item de servico
890
891             lExist := .T.
892             Exit
893
894         EndIf
895     EndIf
896
897 Next nI
898
899 Return lExist
900
901 //-----
902 /* {Protheus.doc} STBSumVRPS
903 Adiciona valor do serviço (RPS) no totalizador não fiscal do RPS.
904
905
906 @param nValue, Numérico, Valor do Item de serviço (RPS)
907
908 @return Nil
909 */
910 //-----
911 Function STBSumVRPS( nValue )
912 Local oTotal := STFGetTot() // Totalizador
913
914 Default nValue := 0
915
916 If LMVLJPRDSV // Verifica se esta ativa a implementacao de venda com itens de "produto" e itens de "servico" em Notas Separadas (RPS)
917     If oTotal:HasField("L1_NOTFISRPS") //Verifica se o campo está na estrutura do Model
918         oTotal:SetValue( "L1_NOTFISRPS" , nValue + oTotal:GetValue("L1_NOTFISRPS") )
919     EndIf
920 EndIf
921
922 Return Nil

```

Componentes/NumberSale/STBNumberSaleEcf.prw

73 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} ValuePriceDefault
6 Funcao responsavel em solicitar e retornar o numero de C00 do CF
7
8 @param      Nil
9 @return     cCoo - Numero do C00
10 @obs
11 @sample
12 */
13 //-----
14 Function STBNumCoo()
15
16 Local cCoo := ""                // Numero do C00
17 Local aRet := {space(6)}       // Array de retorno
18
19 STFFireEvent(ProcName(0), "STGetReceipt", aRet)
20
21 If Len(aRet) > 0 .AND. ValType(aRet[1]) == "C"
22     cCoo := aRet[1]
23 EndIf
24
25 Return cCoo
26
27
28 //-----
29 /*{Protheus.doc} ValuePriceDefault
30 Funcao responsavel em solicitar e retornar o numero de fabricacao do ECF
31
32 @param      Nil
33 @return     cSerie - Numero de serie
34 @obs
35 @sample
36 */
37 //-----
38 Function STBNumFab()
39
40 Local aRet := { space(30)}       // Array de retorno
41 Local cSerie := ""              // numero de serie
42
43 STFFireEvent(ProcName(0), "STGetSerie", aRet)
44
45 If Len(aRet) > 0 .AND. ValType(aRet[1]) == "C"
46     cSerie := aRet[1]
47 EndIf
48
49 Return cSerie
50
51
52 //-----

```

```
53 /*/{Protheus.doc} STBQtdeInt
54 Funcao responsavel em solicitar e retornar a quantidade de intervencoes do ECF
55
56 @param      Nil
57 @return      cCRO - Intervencoes
58 @obs
59 @sample
60 */
61 //-----
62 Function STBQtdeInt()
63
64 Local cCRO := ""           // Intervencoes
65 Local aRet := {"23", space(6) } // Array de retorno
66
67 STFFireEvent(ProcName(0), "STPrinterStatus",aRet )
68
69 If Len(aRet) > 1 .AND. ValType(aRet[2]) == "C"
70     cCRO := aRet[2]
71 EndIf
72
73 Return cCRO
```

Componentes/NumberSale/STBNumberingDefault.prw

44 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cMvSerie      := STFGetStat("LG_SERIE") //Serie da nota
5
6 //-----
7 /*{Protheus.doc} STBNumSx5
8 Funcao que gera o numero a partir das tabelas: Sx5 ou Sxe/Sxf
9 A escolha das tabelas a serem utilizadas na numeracao e configurada a partir
10 do parametro MV_LJNRNFS.
11
12 @param      Nil
13 @return     cNumber - Retorno da numeracao pela Sx5 ou Sxe/Sxf
14 @obs
15 @sample
16 */
17 //-----
18 Function STBNumSx5()
19
20 Local cNumber      := "" //Armazena o numero da nota
21
22 cNumber := NxtSX5Nota( cMvSerie,, SuperGetMV("MV_LJNRNFS",.F., "2"))
23
24 Return cNumber
25
26
27 //-----
28 /*{Protheus.doc} STBNumSd9
29 A partir da rotina MA461NumNf gera o numero da tabela Sd9
30
31 @param      Nil
32 @return     cNumber - Retorno da numeracao pela Sd9
33 @obs
34 @sample
35 */
36 //-----
37 Function STBNumSd9()
38
39 Local cNumber      := "" //Armazena o numero da nota
40 Local nTamDoc      := TamSx3("L1_DOC")[1] //Tamanho do campo L1_DOC na Sx3
41
42 cNumber := PadR( MA461NumNf( .T., cMvSerie ) , nTamDoc )
43
44 Return cNumber

```

Componentes/NumberSale/STBSerieControl.PRW

200 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STBSERIECONTROL.CH"
3 //-----
4 /*{Protheus.doc} STBNSrAval
5 Verifica os numeros disponiveis para o controle de Numero de série
6 @obs
7 @sample
8 */
9 //-----
10 Function STBNSrAval()
11 Local nI      := 0 // Indica quando executar a rotina de serie x PDV
12 Local nMvNsrDisp := SuperGetMV("MV_NSRDISP",,0) // Indica se utiliza controle de serie automatica
13
14 If nMvNsrDisp > 0
15
16     DbSelectArea("SL6")
17     DbSetOrder(2)
18     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA ¿
19     //³Procura todas as series disponiveis para esta estacao³
20     //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA Û
21     If DbSeek( xFilial("SL6")+cEstacao+"1" )
22         While SL6->( !Eof() ) .AND. SL6->L6_ESTACAO == cEstacao;
23             .AND. SL6->L6_STATUS == "1"
24
25             nI++
26             SL6->( DbSkip() )
27         End
28     EndIf
29
30     If nI <= nMvNsrDisp
31         STFMessage("STBNSrAval", "ALERT", STR0001) //"Reserva de série para esta estação está chegando ao final. Favor rodar a rotina Série X PDV na retaguarda"
32         StfShowMessage("STBNSrAval")
33     EndIf
34 EndIf
35
36 Return (NIL)
37
38 //-----
39 /*{Protheus.doc} STBValcDoc
40 Realiza o incremento do número de série
41 @param cDoc Numero do Cupom Fiscal
42 @param cCRO Numero do CRO
43 @obs
44 @sample
45 */
46 //-----
47
48 Function STBSerieAlt( cDoc, cCRO)
49 Local aArea      := "" // Area
50 Local lAuto      := SuperGetMV("MV_LJSRAUT", ,.F.) // Indica se utiliza incremento automatico da serie
51 Local cMV_LJSTSER := SuperGetMV("MV_LJCTSER", , "0") //Controle para numero de serie 0 - C00, 1 - CRO, 2 - C00 + CRO
52 Local cC00       := "999999" // Numero maximo do C00

```



```

53 Local cDocAtu      := "" // Numero do documento com 6 digitos
54 Local cTamSer       := STFFGetStation("TAMSER") //Tamanho do numero de Série
55 Local cCR0Stat      := STFFGetStation("CR0") //Tamanho do CR0
56 Local cEstacao      := STFFGetStation("CODIGO") //Codigo da Estação
57 Local aAreaSL6      := SL6->(GetArea()) //WorkArea SL6
58 Local lAltC00       := .F. //Verifica se Alterou a série //Alterou o C00
59 Local cPDV          := STFFGetStation("PDV") //Numero do PDV
60 Local cSerieSt      := STFFGetStation("SERIE") //Serie da Estação
61
62
63 If lAuto
64     aArea := GetArea()
65
66     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
67     //³Compara se o numero do cupom e o ultimo numero antes de zerar o C00. ³
68     //³(l6_status == 1 -> serie disponivel pra uso. l6_status==2 -> serie utilizada. ³
69     //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
70     If cMV_LJSTSER == "2"
71         lAltC00 := STBAltC00(cCR0,cDoc, cCR0Stat, cPDV, cSerieSt)
72     EndIf
73     cDocAtu := STBValcDoc( cDoc )
74
75     If ( Val(cDocAtu) == Val(SubStr(cC00,1, Len(cTamSer) )) ) .OR. ;
76         (cMV_LJSTSER == "1" .AND. Val(cCR0) <> Val(cCR0Stat) ) .OR. ;
77         (lAltC00)
78
79
80     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
81     //³Inativa a serie atual do PDV na tabela SL6, para reservar a proxima ³
82     //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
83     DbSelectArea("SL6")
84     DbSetOrder(1)
85
86     If DbSeek( xFilial("SL6")+PadR(cSerieSt, TamSx3("L6_SERIE"))[1])+cEstacao )
87         RecLock( "SL6", .F. )
88             L6_STATUS := '2'
89             L6_CR0 := cCR0Stat
90         MsUnlock()
91     EndIf
92
93     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
94     //³Reserva a proxima serie disponivel para a Estacao. Caso nenhuma serie estiver disponivel³
95     //³para a estacao, o sistema devera informar ao usuario. ³
96     //³Status: 1-Disponivel para uso 2- utilizado. ³
97     //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
98     DbSetOrder(2)
99     If DbSeek( xFilial("SL6")+cEstacao+"1")
100         RecLock( "SL6", .F. )
101
102         L6_CR0 := cCR0
103         MsUnlock()
104

```

```

105         STFSetStat({ { "SERIE", SL6->L6_SERIE },;
106                     { "CRO", cCRO} })
107
108         STFMessage("STBSerieAlt2", "ALERT", STR0002)//"Número do C00 foi reinicializado."
109         STFShowMessage("STBSerieAlt1")
110     Else
111         STFMessage("STBSerieAlt2", "ALERT", STR0003) //"Reserva de série para esta estação está chegando ao final. Favor rodar a rotina Série X PDV na retaguarda"
112         STFShowMessage("STBSerieAlt2")
113     EndIf
114
115
116 EndIf
117
118     RestArea(aAreaSL6)
119     RestArea(aArea)
120
121 EndIf
122
123 Return (NIL)
124
125
126 //-----
127 /*{Protheus.doc} STBValcDoc
128 Ajusta o campo L1_DOC
129 @obs
130 @sample
131 */
132 //-----
133 Static Function STBValcDoc( cDoc )
134 Local cDocAtu := "" // Retorno da funcao
135
136 If TamSX3("L1_DOC")[1] == 9
137     cDocAtu := SubStr( cDoc, 4, Len( cDoc ) )
138 EndIf
139
140 Return( cDocAtu )
141
142 //-----
143 /*{Protheus.doc} STBAltC00
144 Verifica se alterou o C00
145 @param cCRO      Numero do CRO
146 @param cC00      Numero do C00
147 @param cCROStat  CRO da Estação
148 @param cPDV      Numero do PDV
149 @param cSerieST  Serie da Estacao
150 @return lAltC00  Houve alteração C00
151 @obs
152 @sample
153 */
154 //-----
155 Static Function STBAltC00(cCRO, cC00, cCROStat, cPDV, cSerieSt)
156     Local aAreaSL1 := SL1->(GetArea()) //WorkArea SL1

```

```

157 Local lAltC00 := .F. //Alterou C00
158 Local cUltDoc := "" //Ultimo documento fiscal
159
160
161 If Val(cCRO) <> Val(cCROStat)
162     lAltC00 := .T. //Alterou o CRO
163     //Busca o último SL1
164
165     //Considera registros Deletados para subir cancelamentos
166     SET DELETED OFF
167
168     SL1->(DbSetOrder(4) ) //L1_FILIAL + L1_EMISSAO
169
170     SL1->(DbSetFilter({ || L1_FILIAL == xFilial() .AND. ;
171                         AllTrim(L1_PDV) == AllTrim(cPDV) .AND. ;
172                         (!Empty(L1_DOC) .OR. !Empty(L1_DOCPED)) .AND. ;
173                         (AllTrim(L1_SERIE) == AllTrim(cSerieSt) .OR. ;
174                         AllTrim(L1_SERPED) == AllTrim(cSerieSt ) )}, ;
175                     " L1_FILIAL == xFilial() .AND. AllTrim(L1_PDV) == AllTrim(cPDV) .AND. (!Empty(L1_DOC) .OR. !Empty(L1_DOCPED)) .AND. (AllTrim(L1_SERIE) == AllTrim(cSerieSt)
176                     .OR. AllTrim(L1_SERPED) == AllTrim(cSerieSt ) )"))
177
178     SL1->(DbGoBottom())
179     SL1->(DbSkip(-1))
180
181     If SL1->(!Eof()) //Verifica se não é final de arquivo
182         //Verifica se o número de série é igual ao cadastrado na estação
183         //Se for, verifica o último cupom
184         //Se for menor ou igual, mantém a numeracao
185         cUltDoc := SL1->L1_DOC
186         If Empty(cUltDoc)
187             cUltDoc := SL1->L1_DOCPED
188         EndIf
189
190         lAltC00 := Val(cUltDoc) > Val(cC00)
191
192     EndIf
193     SL1->(DbClearFilter())
194
195     //Considera registros Deletados para apurar cancelamentos
196     SET DELETED ON
197 EndIf
198
199 RestArea(aAreaSL1)
200 Return lAltC00

```

Componentes/NumberSale/STDControlForm.prw

358 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cEspFor := "1|2|3|4" //Especies de formulario
5
6 //-----
7 /*{Protheus.doc} STDEspeFor
8 Verificar especies de formlarios validas que podem ser utilizadas na venda.
9
10 @param      Nil
11 @return     Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STDEspeFor()
17
18 Local   aArea      := GetArea()    //Armazena alias corrente
19 Local   aSpecies   := {}          //Array com as especies reservadas para a estacao
20 Local   nRecnoFo1  := 0            //nRecnoFo1
21 Local   nRecnoFo2  := 0            //nRecnoFo2
22 Local   cEspecie   := ""          //Especie que o usuario escolheu
23 Local   cNumNota   := ""          //Numero da nota
24 Local   cSerie     := ""          //Serie
25
26
27 //Verificar na tabela SL6 se os tipos de documento Boleta,
28 //Boleta Exenta, Factura ou Factura Exenta estao reservados
29 //para a estacao
30 DbSelectArea("SL6")
31 DbSetOrder(3)//L6_FILIAL+L6_ESTACAO
32 If DbSeek(xFilial("SL6") + cEstacao)
33     Do While SL6->L6_ESTACAO == cEstacao
34         If SL6->L6_ESPFO $ cEspFor
35
36             //Se a reserva de serie estiver associada a um
37             //formulario, verificar se o mesmo e valido.
38             If ChkFolCHI(SL6->L6_FILFO , SL6->L6_SERIE, Nil, cEspFor,;
39                 @nRecnoFo1 , .T.)
40                 Aadd(aSpecies, {SL6->L6_ESPFO, SL6->L6_SERIE, nRecnoFo1})
41             EndIf
42         EndIf
43     EndIf
44     SL6->(DbSkip())
45 End
46 EndIf
47
48
49 If Len(aSpecies) >= 1
50
51     STDLastSale( aSpecies )
52

```

```

53 EndIf
54
55 RestArea(aArea)
56
57 Return Nil
58
59 //-----
60 /*{Protheus.doc} STDLastSale
61 Busca informacao da ultima venda
62
63 @param      aSpecie - Especies
64 @return     aDadosFo - Dados do formulario
65 @obs
66 @sample
67 */
68 //-----
69 Function STDLastSale(aSpecie)
70
71 Local   aArea      := GetArea()                //Armazena alias corrente
72 Local cSerieFo    := SPACE(TamSx3("FP_SERIE")[1]) //Serie que sera utilizada para o tipo de documento escolhido
73 Local cDocFo      := SPACE(TamSx3("F2_DOC")[1])   //Nota referente a serie escolhida
74 Local nRecnoFo    := 0                          //Recno do formulario
75 Local aAreaSL1    := SL1->(GetArea())             //Restaura a area da tabela SL1
76 Local nPosRec     := 0                          //Posicao para buscar a especie no array
77
78 Default aSpecie := {}
79
80 ParamType 0 Var   aSpecie      As Array      Default   {}
81
82 //Obter dados da ultima venda
83 DbSelectArea("SL1")
84 SL1->(DbGoBottom ())
85
86 If !Empty(SL1->L1_SERIE) .AND. !Empty(SL1->L1_DOC)
87
88     nRecnoFo := STDRecnoFo(SL1->L1_SERIE)
89
90     aDadosFo := STDDadosFo(nRecnoFo)
91
92     If nRecnoFo > 0 .AND. Len(aDadosFo) > 0 .AND. aDadosFo[1][2] $ cEspFor
93         //Retornar a especie/serie/numero+1 do documento da ultima
94         //venda se ela for: 1-FCT ; 2-FCX ; 3-BLT ; 4-BLX
95         If !Empty(SL1->L1_SERIE)
96             cSerieFo := SL1->L1_SERIE
97         ElseIf !Empty(SL1->L1_SERPED)
98             cSerieFo := SL1->L1_SERPED
99         EndIf
100
101         DbSelectArea("SX5")
102         DbSetOrder(1)
103         DbSeek(xFilial() + "01" + cSerieFo)
104         cDocFo := AllTrim(X5Descri())

```

```

105
106      //Verificar se a serie da ultima venda esta reservada para a estacao
107      DbSelectArea("SL6")
108      DbSetOrder (1)//L6_FILIAL+L6_SERIE+L6_ESTACAO
109      If !DbSeek(xFilial("SL6") + cSerieFo + cEstacao)
110          aDadosFo[2][2] := SPACE(TamSx3("X5_CHAVE")[1])
111          cSerieFo       := SPACE(TamSx3("FP_SERIE")[1])
112          cDocFo         := SPACE(TamSx3("F2_DOC")[1])
113      EndIf
114      SL6->(DbCloseArea())
115  Else
116      cSerieFo := SPACE(TamSx3("FP_SERIE")[1])
117      cDocFo  := SPACE(TamSx3("F2_DOC")[1])
118  EndIf
119
120 EndIf
121
122 SL1->(RestArea(aAreaSL1))
123
124 If Len(aDadosFo) == 0
125     nPosRec := aScan(aSpecie, { |x| AllTrim(x[02]) == aSpecie[1][2] })
126
127     If nPosRec > 0
128         Aadd(aDadosFo, {"Especie",aSpecie[nPosRec][3]})
129     Else
130         Aadd(aDadosFo, {"Especie",""})
131     EndIf
132
133     Aadd(aDadosFo, {"Sigla",STDSiglaFo(aSpecie[1][1])})
134     Aadd(aDadosFo, {"Descricao",""})
135     Aadd(aDadosFo, {"Serie",aSpecie[1][2]})
136     Aadd(aDadosFo, {"Doc. Formulário",STDVldNota(aSpecie[1][2])[2]})
137 EndIf
138
139 RestArea(aArea)
140
141 Return aDadosFo
142
143
144 //-----
145 /*{Protheus.doc} STDRecnoFo
146 Retorna o Recno do controle de formulario de acordo com a serie.
147
148 @param      cSerie - Numero de serie
149 @return     nRecno - Recno da tabela SFP
150 @obs
151 @sample
152 */
153 //-----
154 Function STDRecnoFo(cSerie)
155
156 Local aArea := GetArea()    //Salva a area

```

```

157 Local nRecno      := 0           //Recno do formulario
158
159 Default cSerie:= ""
160
161 ParamType 0 Var      cSerie As Character   Default      ""
162
163 DbSelectArea("SFP")
164 DbSetOrder(1)
165
166 If DbSeek (xFilial("SFP") + SL1->L1_FILIAL + cSerie)
167     nRecno := SFP->(RecNo())
168 EndIf
169
170 RestArea(aArea)
171
172 Return nRecno
173
174
175 //-----
176 /*{Protheus.doc} STDDadosFo
177 Retorna os dados do controle de formulario de acordo com o Recno passado por
178 parametro
179 @param      nRecNo - Recno
180 @return     aRet
181 @obs
182 @sample
183 */
184 //-----
185 Function STDDadosFo(nRecNo)
186
187 Local aArea      := GetArea()      //Guarda a area atual
188 Local cEspCod    := ""             //Codigo da especie
189 Local cEspSigla := ""             //Sigla
190 Local cEspDesc  := ""             //Descricao
191 Local cSerie    := ""             //Serie
192 Local aRet      := {}             //Array para retorno das informacoes
193
194 Default nRecNo := 0                //Recno do registro do formularios
195
196 ParamType 0 Var      nRecNo      As Numeric   Default      0
197
198 If !(nRecNo == 0)
199
200     DbSelectArea("SFP")
201     SFP->(DbGoto(nRecNo))
202
203     cEspCod := SFP->FP_ESPECIE
204     cEspSigla := STD̃SiglaFo(cEspCod)
205
206     DbSelectArea("SX5")
207     DbSetOrder(1)
208     If !DbSeek(xFilial() + "42" + cEspSigla)

```

```

209      Help(" ",1,"LJ010SERIE")
210      lRet := .F.
211  Else
212      cEspDesc := AllTrim(X5_DESCRI)
213  EndIf
214
215  cSerie:= SFP->FP_SERIE
216
217  aRet := {"Especie",cEspCod},{ "Sigla",cEspSigla},{ "Descricao",cEspDesc},{ "Serie",cSerie}}
218
219 EndIf
220
221 RestArea(aArea)
222
223 Return aRet
224
225
226 //-----
227 /*{Protheus.doc} STDSiglaFo
228 Retorna sigla do formulario
229 @param      cCodigo -Codigo do formulario
230 @return      cRet - sigla do formulario
231 @obs
232 @sample
233 */
234 //-----
235 Function STDSiglaFo(cCodigo)
236
237 Local  aArea      := GetArea()           // Armazena alias corrente
238 Local aSX3Box     := {}                 //Array de Opcoes do Combo Box
239 Local cRet        := ""                 //Retorno
240 Local nPos        := 0                  //Posicao do array
241
242 Default cCodigo := ""
243
244 ParamType 0 Var      cCodigo      As Numeric Default      0
245
246 DbSelectArea("SX3")
247 DbSetOrder(2)
248
249 If DbSeek("FP_ESPECIE")
250     aSX3Box := RetSx3Box( X3CBox(),, , 1 )
251
252     nPos := Ascan(aSX3BOX,{|x| x[2] == Rtrim(cCodigo)})
253
254     If nPos > 0
255         cRet := aSX3Box[nPos,3]
256     EndIf
257 EndIf
258
259 If Empty(cRet)
260     cRet := cCodigo

```



```

261 EndIf
262
263 RestArea(aArea)
264
265 Return Rtrim(cRet)
266
267
268 //-----
269 /*{Protoheus.doc} STDDadosFo
270 Validar se a especie do formulario pode ser utilizada junto com a determinada
271 serie que deve estar reservada para estacao.
272
273 @param      cEspecie - Especie
274 @param      cSerie   - Serie
275 @return     lRet - Retorna .T. ou .F. se validou o formulario
276 @obs
277 @sample
278 */
279 //-----
280 Function STDVldEsp(cEspecie,cSerie)
281
282 Local lRet := .T.           //Retorno da funcao
283 Local aArea := GetArea()   //Area
284 Local nPosIni := 0         //Guarda a posicao da string
285 Local cCombo := ""        //Armazena o valor do combo do campo X3_CBOXSPA
286
287 Default cEspecie := ""
288 Default cSerie := ""
289
290 ParamType 0 Var      cEspecie As Character Default ""
291 ParamType 1 Var      cSerie As Character Default ""
292
293 //Busca posicao da descricao da especie da nota no combo da
294 //tabela SFP (1=FCT;2=FCT;3=BLT;4=BLX;5=GDP;6=NDC;7=NDI;8=NCC;9=NCI;A=NCX)
295 SX3->(dbSetOrder(2))
296 SX3->(dbSeek("FP_ESPECIE"))
297 nPosIni:= At(AllTrim(cEspecie),AllTrim(SX3->X3_CBOXSPA))
298 cCombo := Substr(AllTrim(SX3->X3_CBOXSPA), nPosIni - 2, 1)
299
300 DbSelectArea("SL6")
301 DbSetOrder(4)
302 If DbSeek(xFilial() + cEstacao + cCombo)
303     cSerie := SL6->L6_SERIE
304     lRet := .T.
305 Else
306     lRet := .F.
307 EndIf
308
309 RestArea(aArea)
310
311 Return lRet
312

```

```

313
314 //-----
315 /*{Protheus.doc} STDVldNota
316 Validar numeracao do formulario
317
318 @param      cSerie - Serie
319 @param      cNumNota - Numero do documento fiscal
320 @return     aRet - Validacao de funcoes e nu8mero na nota
321 @obs
322 @sample
323 */
324 //-----
325 Function STDVldNota( cSerie )
326
327 Local aRet      := {.T.,""} //Retorno da funcao
328 Local aArea     := GetArea() //Area
329 Local lNotaManual := IIf( Type("lNFManual")#"U", lNFManual, .F.) //Nota manual
330 Local cNumNota  := "" //Numero da nota
331
332 Default cSerie := ""
333
334 ParamType 0 Var cSerie As Character Default ""
335
336 If !lNotaManual
337
338     DbSelectArea("SX5")
339     DbSetOrder(1)
340
341     If !DbSeek(xFilial() + "01" + cSerie)
342         Help(" ",1,"LJ010SERIE")
343         aRet := {.F.,cNumNota}
344     EndIf
345
346     cNumNota := AllTrim(X5Descri())
347
348     If !ChkFolCHI(xFilial("SFP"), cSerie, cNumNota, "1|2|3|4|8")
349         aRet := {.F.,cNumNota}
350     Else
351         aRet := {.T.,cNumNota}
352     EndIf
353
354 EndIf
355
356 RestArea(aArea)
357
358 Return aRet

```

Componentes/NumberSale/STWFormNumber.prw

41 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STWFormNumber
6 Workflow para geracao da numeracao de venda.
7
8 @param      Nil
9 @return     Nil
10 @obs
11 @sample
12 */
13 //-----
14 Function STWFormNumber()
15
16 Local nMvTpNrNfs := SuperGetMV( "MV_TPNRNFS" ) //Define o tipo de controle da numeracao dos documentos de saida ( 1-SX5 | 2-SXE/SXF | 3-SD9 )
17 Local lMvCtrlFol := SuperGetMV( "MV_CTRLFOL" ) //Define se utiliza ou nao o controle de formulario
18 Local cCOONumber := STBNumCoo()                //Recupera o numero do coo
19 Local cFabNumber := STBNumFab()                //Recupera o numero de fabricacao
20 Local cInterQtd := STBQtdeInt()                //Recupera a quantidade de intervencoes da impressora
21
22 If lMvCtrlFol
23     //Controle de formulario
24     STDEspeFor()
25 ElseIf !Empty(cCOONumber) .AND. !Empty(cFabNumber) .AND. !Empty(cInterQtd)
26     //Se a impressora conseguir retornar o numero de Coo + Numero de Fabricao e Qtd de intervencoes
27     //entao utiliza essas 3 combinacoes para formar a numeracao da venda.
28
29 ElseIf !Empty(cCOONumber)
30     //Se a impressora retornar o numero de fabricacao e Qtd de intervencoes, entao utiliza a
31     //combinacao de numero de coo + serie Lg_Serie
32
33 Else
34     If nMvTpNrNfs = 1 .OR. nMvTpNrNfs = 2
35         STBNumSx5()
36     Else
37         STBNumSd9()
38     EndIf
39 EndIf
40
41 Return Nil

```

Componentes/Payment/STBChangePay.prw

275 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #Include 'STBChangePay.ch'
3
4 //-----
5 /*{Protheus.doc} STBValidPay
6 Função para validar (permitir) as trocas de pagamentos
7 @param
8 @return      .T. ou .F.
9 */
10 //-----
11 Function STBValidPay()
12 Local lRet      := .T.
13 Local lChgPay    := .T.
14 Local nCont      := 0
15 Local nCont2     := 0
16 Local nParcela   := 1
17 Local nPosForma  := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_FORMA"} )
18 Local nPosValor  := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_VALOR"} )
19 Local nPosParcela := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_PARCELA"} )
20 Local nPosIdCard  := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_FORMAID"} )
21 Local nPosNvForma := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_NVFORM"} )
22 Local nPosQtdCard := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_QTDCART"} )
23 Local nPosNvParc  := aScan(aHeadPagto,{ |x| AllTrim(x[2]) == "L4_NVPARC"} )
24 Local cMsg        := ""
25 Local aCartoes    := {}
26 //Valida se pode ou não adicionar na grid do resumo
27 If !Empty(oGetResumo:aCols[1][1])
28     MsgInfo(STR0001,STR0002) //"Não é possível adicionar as formas de pagamento, pois já foram adicionadas, favor clicar no botão 'Limpar' para alterar as formas
        alteradas!!!"/"Atenção"
29     lRet := .F.
30 Else
31     For nCont := 1 To Len(oGetPagtos:aCols)
32         If Empty(oGetPagtos:aCols[nCont][nPosNvForma])
33             lChgPay := .F.
34         Else
35             lChgPay := .T.
36             Exit
37         EndIf
38     Next nCont
39     If lChgPay
40         For nCont := 1 To Len(oGetPagtos:aCols)
41             If SubStr(oGetPagtos:aCols[nCont][nPosForma],1,2) == "R$"
42                 nParcela := 1
43             Else
44                 nParcela := Val(oGetPagtos:aCols[nCont][nPosParcela])
45             EndIf
46             If SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) $ "CC|CD"
47                 If oGetPagtos:aCols[nCont][nPosQtdCard] > 1//Qtde de Cartoes - STIChangePay
48                     Aadd(aCartoes,{oGetPagtos:aCols[nCont][nPosIdCard],oGetPagtos:aCols[nCont][nPosNvForma], oGetPagtos:aCols[nCont][nPosValor],;
49                         oGetPagtos:aCols[nCont][nPosParcela], oGetPagtos:aCols[nCont][nPosQtdCard], oGetPagtos:aCols[nCont][nPosForma]}) //ID Cartão, Nova Forma de
50                     Pagamento, Valor, Parcela, Qtde de Cartões, Forma Original
51                 ElseIf oGetPagtos:aCols[nCont][nPosQtdCard] == 0

```

```

51      cMsg += STR0003 + AllTrim(oGetPagtos:aCols[nCont][nPosNvForma]) + STR0004 + AllTrim(Str(nCont)) + STR0005 + CHR(13) + CHR(10) //"Foi informado a forma "/"
na linha "/" e não foi informado o campo 'Qtde. Cartões'. "
52      ElseIf !Empty(oGetPagtos:aCols[nCont][nPosNvParc]) .And. Val(oGetPagtos:aCols[nCont][nPosNvParc]) > nParcela
53      cMsg += STR0006 + AllTrim(Str(nCont)) + " ." + CHR(13) + CHR(10) //"Foi informado quantidade de parcelas maior que a parcela original na linha "
54      EndIf
55      ElseIf SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) <> "R$" .And. !Empty(oGetPagtos:aCols[nCont][nPosNvForma]) .And.
(Empty(oGetPagtos:aCols[nCont][nPosNvParc]) .Or. Val(oGetPagtos:aCols[nCont][nPosNvParc]) == 0 )
56      cMsg += STR0007 + AllTrim(Str(nCont)) + " ." + CHR(13) + CHR(10) //
57      ElseIf !Empty(oGetPagtos:aCols[nCont][nPosNvParc]) .And. Val(oGetPagtos:aCols[nCont][nPosNvParc]) > nParcela
58      cMsg += STR0008 + AllTrim(Str(nCont)) + " ." + CHR(13) + CHR(10) //"Foi informado quantidade de parcelas maior que a parcela original na linha "
59      EndIf
60      Next nCont
61      If Empty(cMsg)
62      If Len(aCartoes) > 0
63      lRet := STIChgCard(aCartoes)
64      EndIf
65      If lRet
66      For nCont := 1 To Len(oGetPagtos:aCols)
67      If !Empty(oGetPagtos:aCols[nCont][nPosNvForma])
68      If SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) $ "CC|CD" .And. oGetPagtos:aCols[nCont][nPosQtdCard] > 1
69      For nCont2 := 1 To Len(oGetCards:aCols)
70      //verifica qual é o pagamento atual e verifico com o getdados dos cartões
71      If SubStr(oGetCards:aCols[nCont2][6],1,2) == SubStr(oGetPagtos:aCols[nCont][nPosForma],1,2) .And. SubStr(oGetCards:aCols[nCont2][1],1,2) ==
SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) .And. oGetCards:aCols[nCont2][4] == oGetPagtos:aCols[nCont][nPosIdCard]
72      Add(aColsResumo,{oGetCards:aCols[nCont2][1],oGetCards:aCols[nCont2][2],StrZero(oGetCards:aCols[nCont2][3],2),oGetCards:aCols[nCont2][5],.
F.})
73      EndIf
74      Next nCont2
75      ElseIf SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) $ "CC|CD"
76      //Pagamento com a forma CC ou CD e definiu um cartão
77      Add(aColsResumo,{oGetPagtos:aCols[nCont][nPosNvForma],oGetPagtos:aCols[nCont][nPosValor],PadL(AllTrim(oGetPagtos:aCols[nCont][nPosNvParc]),2,"0")
,oGetPagtos:aCols[nCont][nPosIdCard],.F.})
78      Else
79      //Pagamento diferente da forma CC ou CD
80      Add(aColsResumo,{oGetPagtos:aCols[nCont][nPosNvForma],oGetPagtos:aCols[nCont][nPosValor],PadL(AllTrim(oGetPagtos:aCols[nCont][nPosNvParc]),2,"0")
, "",.F.})
81      EndIf
82      Else
83      //Pagamento não teve alteração
84      Add(aColsResumo,{oGetPagtos:aCols[nCont][nPosForma],oGetPagtos:aCols[nCont][nPosValor],IIf(SubStr(oGetPagtos:aCols[nCont][nPosNvForma],1,2) == "R$",
"01",oGetPagtos:aCols[nCont][nPosParcela]),oGetPagtos:aCols[nCont][nPosIdCard],.F.})
85      EndIf
86      Next nCont
87      aDel(aColsResumo, 1) //exclui a primeira linha
88      aSize(aColsResumo, Len(aColsResumo) - 1) //seta novo tamanho do array
89      oGetResumo:SetArray(aColsResumo,.T.)
90      oGetResumo:Refresh()
91      EndIf
92      Else
93      MsgInfo(Stuff(cMsg,0,0,STR0009 + Chr(13) + Chr(10)),STR0002) //"Favor Verificar: "/"Atenção"
94      lRet := .F.
95      EndIf

```

```

96     Else
97         MsgInfo(STR0010,STR0002) //"Não foi alterada nenhuma forma, favor realizar as alterações!!!"/"Atenção"
98         lRet := .F.
99     EndIf
100 EndIf
101 Return lRet
102
103 //-----
104 /*{Protheus.doc} STBChangePay
105 Função para confirmar e armazenar na variável de pagamentos os novos pagamentos
106 @param      aPagtos
107 @return      Nil
108 */
109 //-----
110 Function STBChangePay(aPagtos)
111 Local lRet      := .F.
112 Local nCont     := 0
113 Local nCont2    := 0
114 Local nCampos   := 0
115 Local nSobra    := 0
116 Local nValorTot := 0
117 Local nValParc  := 0
118 Local nTotSobra := 0
119 Local nUltParc  := 0
120 Local nTamaCols := 0
121 Local nQtdDel   := 0
122 Local nPosForma := aScan(aHeadResumo,{ |x| AllTrim(x[2]) == "L4_FORMA"} )
123 Local nPosValor := aScan(aHeadResumo,{ |x| AllTrim(x[2]) == "L4_VALOR"} )
124 Local nPosFrmId := aScan(aHeadResumo,{ |x| AllTrim(x[2]) == "L4_FORMAID"} )
125 Local nPosParc  := aScan(aHeadResumo,{ |x| AllTrim(x[2]) == "L4_NVPARC"} )
126 Local cIdCardCD := "00"
127 Local cIdCardCC := "00"
128
129 If !Empty(oGetResumo:aCols[1][1])
130     lRet := .T.
131     //Refaz os Id's dos cartões
132     For nCont := 1 To Len(aColsResumo)
133         If SubStr(aColsResumo[nCont][nPosForma],1,2) $ "CC|CD"
134             If SubStr(aColsResumo[nCont][nPosForma],1,2) == "CD"
135                 cIdCardCD := Soma1(cIdCardCD)
136                 aColsResumo[nCont][nPosFrmId] := cIdCardCD
137             Else
138                 cIdCardCC := Soma1(cIdCardCC)
139                 aColsResumo[nCont][nPosFrmId] := cIdCardCC
140             EndIf
141         EndIf
142     Next nCont
143
144     //Refaz os pagamentos com parcelas maior que 1
145     For nCont := 1 To Len(aColsResumo)
146         If Val(aColsResumo[nCont][nPosParc]) > 1
147             nValorTot := aColsResumo[nCont][nPosValor]

```

```

148     nValParc      := nValorTot / Val(aColsResumo[nCont][nPosParc])
149     For nSobra := 1 To Val(aColsResumo[nCont][nPosParc])
150         nTotSobra      += nValParc - NoRound(nValParc,2)
151         If nSobra == Val(aColsResumo[nCont][nPosParc])
152             nUltParc := Round(NoRound(nValParc,2) + nTotSobra,2)
153         EndIf
154     Next
155     nTamaCols := Len(aColsResumo)
156     aSize(aColsResumo,nTamaCols+Val(aColsResumo[nCont][nPosParc]))
157     For nCont2 := nTamaCols + 1 To Len(aColsResumo)
158         aColsResumo[nCont2] := Array(Len(aColsResumo[nCont]))
159         aColsResumo[nCont2][nPosForma] := aColsResumo[nCont][nPosForma]
160         If nCont2 == Len(aColsResumo)
161             aColsResumo[nCont2][nPosValor] := nUltParc
162         Else
163             aColsResumo[nCont2][nPosValor] := NoRound(nValParc,2)
164         EndIf
165         aColsResumo[nCont2][nPosParc] := aColsResumo[nCont][nPosParc]
166         aColsResumo[nCont2][nPosFrmId] := aColsResumo[nCont][nPosFrmId]
167         aColsResumo[nCont2][5] := .F. //CONTROLE DA LINHA DELETADA
168     Next nCont2
169     aColsResumo[nCont][5] := .T.
170 EndIf
171 Next
172
173 //Atualiza aColsResumo removendo itens excluídos
174 For nCont:=1 To Len(aColsResumo)
175     If aColsResumo[nCont] <> Nil .And. aColsResumo[nCont][5]
176         ADel(aColsResumo,nCont)
177         nQtdDel += 1
178         nCont := 0
179     Loop
180 EndIf
181 Next nCont
182
183 If nQtdDel > 0
184     ASize(aColsResumo,Len(aColsResumo)-nQtdDel)
185 EndIf
186
187 aPagtos := Array(Len(aColsResumo),Len(aPagtos[1]),2) //Quantidade de pagamentos do resumo,quantidade de campos SL4,quantidade de colunas = estrutura do aSL4
188
189 SX3->(DbSetOrder(1))
190 For nCont := 1 To Len(aPagtos)
191     nCampos := 0
192     SX3->(DbSeek("SL4"))
193     While !SX3->(Eof()) .And. SX3->X3_ARQUIVO == "SL4"
194         nCampos++
195         aPagtos[nCont][nCampos][1] := AllTrim(SX3->X3_CAMP0)//nome do campo
196         aPagtos[nCont][nCampos][2] := CriaVar(aPagtos[nCont][nCampos][1],.T.)//conteúdo (valor)
197     EndWhile
198     SX3->(DbSkip())
199 EndFor

```

```

200     End
201     Next nCont
202
203     For nCont := 1 To Len(aColsResumo)
204         For nCampos := 1 To Len(aPagtos[nCont])
205             If aPagtos[nCont][nCampos][1] == "L4_FILIAL"
206                 aPagtos[nCont][nCampos][2] := xFilial("SL4")
207             ElseIf aPagtos[nCont][nCampos][1] == "L4_NUM"
208                 aPagtos[nCont][nCampos][2] := cNumOrc
209             ElseIf aPagtos[nCont][nCampos][1] == "L4_DATA"
210                 aPagtos[nCont][nCampos][2] := DtoS(dDataBase)
211             ElseIf aPagtos[nCont][nCampos][1] == "L4_VALOR"
212                 aPagtos[nCont][nCampos][2] := aColsResumo[nCont][nPosValor]
213             ElseIf aPagtos[nCont][nCampos][1] == "L4_FORMA"
214                 aPagtos[nCont][nCampos][2] := SubStr(aColsResumo[nCont][nPosForma],1,2)
215             ElseIf aPagtos[nCont][nCampos][1] == "L4_FORMAID"
216                 aPagtos[nCont][nCampos][2] := aColsResumo[nCont][nPosFrmId]
217             EndIf
218         Next nCampos
219     Next nCont
220
221 Else
222     MsgInfo(STR0011,STR0002) //"Não foi possível confirmar, pois não há pagamentos informados, favor realizar as alterações!!!"/
223 EndIf
224 Return lRet
225
226 //-----
227 /*{Protheus.doc} STBVldCard
228 Função para validar se todas as informações dos cartões foram preenchidas
229 @param      aHeadCard, aColsCard
230 @return      .T. ou .F.
231 */
232 //-----
233 Function STBVldCard(aHeadCard, aColsCard)
234 Local lRet      := .T.
235 Local nCont     := 0
236 Local nVlrTot   := 0
237 Local nPosValor := aScan(aHeadCard,{ |x| AllTrim(x[2]) == "L4_VALOR" } )
238 Local nPosNvParc:= aScan(aHeadCard,{ |x| AllTrim(x[2]) == "L4_NVPARC" } )
239 Local cMsg      := ""
240 Local nPosForOld:= aScan(aHeadCard,{ |x| AllTrim(x[2]) == "L4_OLDFRM" } )
241 Local nPosForma := aScan(aHeadCard,{ |x| AllTrim(x[2]) == "L4_FORMA" } )
242 Local nPosOldID := aScan(aHeadCard,{ |x| AllTrim(x[2]) == "L4_OLDID" } )
243 Local nPosVlrPgt:= aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_OLDVLR" } )
244 Local cFormaOri := ""
245 Local cFormaNew := ""
246 Local cIdCardOld:= " " //deixar com um espaço, pois caso a primeira linha seja pagamento original diferente de cartão, não existe ID Cartão
247
248 For nCont := 1 To Len(aColsCard)
249     If Empty(aColsCard[nCont][nPosValor]) .Or. aColsCard[nCont][nPosValor] == 0
250         cMsg += STR0012 + Str(nCont) + ". " + CHR(13) + CHR(10) //"Não foi informado valor ou o valor está ZERO na linha "
251     EndIf

```



```

252 If Empty(aColsCard[nCont][nPosNvParc]) .Or. aColsCard[nCont][nPosNvParc] == 0
253     cMsg += STR0013 + Str(nCont) + ". " + CHR(13) + CHR(10) //"Não foi informado parcela ou a parcela está ZERO na linha "
254 EndIf
255 If aColsCard[nCont][nPosForOld] <> cFormaOri .And. aColsCard[nCont][nPosForma] <> cFormaNew .And. aColsCard[nCont][nPosOldID] <> cIdCardOld
256     If nCont > 1 .And. nVlrTot < aColsCard[nCont-1][nPosVlrPgt]
257         cMsg += STR0014 + CHR(13) + CHR(10) //"Valor dos pagamentos menor que o valor original. "
258     EndIf
259     cFormaOri := aColsCard[nCont][nPosForOld]
260     cFormaNew := aColsCard[nCont][nPosForma]
261     cIdCardOld:= aColsCard[nCont][nPosOldID]
262     nVlrTot := 0
263     nVlrTot+= oGetCards:aCols[nCont][nPosValor]
264 Else
265     nVlrTot+= oGetCards:aCols[nCont][nPosValor]
266     If nCont == Len(aColsCard) .And. nVlrTot < aColsCard[nCont][nPosVlrPgt]
267         cMsg += STR0014 + CHR(13) + CHR(10) //"Valor dos pagamentos menor que o valor original. "
268     EndIf
269 EndIf
270 Next nCont
271 If !Empty(cMsg)
272     MsgInfo(Stuff(cMsg,0,0,STR0009 + Chr(13) + Chr(10)),STR0002) //"Favor Verificar: "/"Atenção"
273     lRet := .F.
274 EndIf
275 Return lRet

```

Componentes/Payment/STBCurrency.prw

57 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static nCurrency := 1          // Moeda Corrente
5
6 //-----
7 /*{Protheus.doc} STBGetCurrency
8 Retorna a Moeda Corrente
9 @param
10 @return nCurrency Retorna a Moeda Corrente
11 @obs
12 @sample
13 */
14 //-----
15 Function STBGetCurrency()
16 Return nCurrency
17
18
19 //-----
20 /*{Protheus.doc} STBSetCurrency
21 Define a Moeda Corrente
22 @param nSetCurrency Moeda Corrente
23 @return Nil
24 @obs
25 @sample
26 */
27 //-----
28 Function STBSetCurrency( nSetCurrency )
29
30 nCurrency := nSetCurrency
31
32 Return Nil
33
34
35 //-----
36 /*{Protheus.doc} STBCurrency
37 Retorna a Moeda Corrente Padrão
38 @param
39 @return nDefCurrency Retorna a Moeda Corrente Padrão
40 @obs
41 @sample
42 */
43 //-----
44 Function STBDefCurrency()
45
46 Local nDefCurrency := 1          // Retorna a Moeda Corrente padrão
47
48 /*
49 Define a moeda padrao a ser utilizada para a venda
50 */
51 If STFGetCfg("lMultCoin") // Se usa Multimoeda
52     nDefCurrency := SuperGetMV("MV_LJMDORC",,1)

```

```
53 Else
54     nDefCurrency := 1
55 EndIf
56
57 Return nDefCurrency
```

Componentes/Payment/STBPayCard.prw

422 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPAYCARD.CH"
4
5 Static oRetTef := Nil
6 Static aIDTEF := {'0','0','0','0'} // [1]ID Cartão Crédito(CC) [2]ID Cartão Débito(CD) [3]ID PIX(PX) [4]ID Pagamento Digital(PD)
7
8 //-----
9 /*{Protheus.doc} STBRetTef
10 Valida a transacao
11
12 @param oRetTran - Objeto de retorno da transacao TEF
13 @param oTEF20 - Objeto TEF
14 @return lRet - Retorna se validou a transacao
15 @obs
16 @sample
17 */
18 //-----
19
20 Function STBRetTef( oRetTran, oTEF20 )
21
22 Local nPosAdmin := 1 // Posicao da administradora
23 Local lRetAux := .F.
24 Local lRet := .T. // TODO: se .F., desfaz a transacao TEF
25 Local aAdmSel := {} // Administradora financeira selecionada manualmente
26 Local aAuxAdmin := {} // Administradoras financeiras filtradas pela forma de pagamento
27 Local aAdminDesc:= {} // Administradoras financeiras selecionadas com base em duas descricao
28 Local aAdmin := {}
29 Local lSelAdm := ExistFunc("STICrdSlAdm")
30 Local cFirst := STFGetCfg("cIntegration", "DEFAULT")
31 Local lFirst := .F.
32 Local nParcs := 1
33 Local nAdmRet := 0 //quantidade adm fin retornadas automaticamente
34 Local nI := 0 //contador
35 Local nTamAECOD := TamSX3("AE_COD")[1]
36 Local nTamL4ADM := TamSX3("L4_ADMINIS")[1]
37
38 Default oRetTran:= Nil
39 Default oTEF20 := Nil
40
41 LjGrvLog(Nil, "Transação TEF bem sucedida: ", oRetTran:oRetorno:lTransOk)
42
43 //se a transação TEF foi bem sucedida
44 If oRetTran:oRetorno:lTransOk
45
46 //Alimenta o atributo da Administradora Financeira, seja retornada automaticamente ou selecionada pelo usuário.
47 STBGetAdmF(oRetTran, oTEF20)
48
49 If ExistFunc("IsPD0rPix") .AND. IsPD0rPix(oRetTran:cFormaPgto)
50 oTEF20:Cupom():Inserir( "V" ,;
51 oRetTran:oRetorno:oViaCaixa,;
52 oRetTran:oRetorno:oViaCliente,;

```

```

53         "D",;
54         oTEF20:PgtoDigital():GetTotalizador(),;
55         oTEF20:PgtoDigital():GetFormaPgto(oRetTran, oTEF20:Formas()),;
56         oRetTran:nValor,;
57         1,;
58         0,;
59         oRetTran:oRetorno:nVlrSaque,;
60         oRetTran:oRetorno:nVlrVndcDesc,;
61         oRetTran:oRetorno:nVlrDescTEF )
62     Else
63
64         oTEF20:Cupom():Inserir( "V" ,;
65             oRetTran:oRetorno:oViaCaixa,;
66             oRetTran:oRetorno:oViaCliente,;
67             "C",;
68             oTEF20:Cartao():GetTotalizador(),;
69             oTEF20:Cartao():GetFormaPgto(oRetTran, oTEF20:Formas()),;
70             oRetTran:nValor,;
71             1,;
72             0,;
73             oRetTran:oRetorno:nVlrSaque,;
74             oRetTran:oRetorno:nVlrVndcDesc,;
75             oRetTran:oRetorno:nVlrDescTEF )
76     EndIf
77
78     lRet      := .T.
79     oRetTef   := oRetTran:oRetorno
80     If ExistFunc("STIBlqTlTef")
81         STIBlqTlTef()
82     EndIf
83
84 Else
85     lRet := .F.
86 EndIf
87
88 Return lRet
89
90
91 //-----
92 /*{Protheus.doc} STBGetRetTef
93 Faz o retorno das variaveis do TEF
94
95 @param
96 @return    oRetTef - Retorno do TEF com a transacao
97 @obs
98 @sample
99 */
100 //-----
101 Function STBGetRetTef()
102 Return oRetTef
103
104

```

```

105 //-----
106 /*/{Protheus.doc} STBSetRetTef
107 Limpa o array de retorno do TEF
108
109 @param      lCancTrs - Cancela a transacao TEF
110 @return  lRet - Retorna se executou corretamente
111 @obs
112 @sample
113 */
114 //-----
115 Function STBSetRetTef(lCancTrs)
116
117 Local oTEF20          := Nil          // Objeto TEF
118
119 Default lCancTrs := .F.
120
121 /*/
122     Se cancelar a transação, Desfaz Transação TEF
123 /*/
124 If lCancTrs
125     oTEF20 := STBGetTef()
126     If ValType(oTEF20) == '0'
127         oTEF20:Desfazer()
128     EndIf
129 EndIf
130
131 If oRetTef <> Nil
132     // -- Não utilizar FreeObj, o FreeObj limpar o conteudo da unidade de memoria, porem neste exemplo precisamos limpar apenas da variavel oRetTef removendo a referencia da
    variavel.
133     oRetTef := Nil
134 EndIf
135
136 Return .T.
137
138
139 //-----
140 /*/{Protheus.doc} STBSetIdTF
141 Controla os contadores dos IDs de cartões e TPD
142 alimentando o array estatico aIDTEF
143
144 @param      cTipoTEF - tipo do cartão (CC/CD/PX/PD)
145 @return      Nil
146 @obs        Se passar cTipoTEF em branco, ele reseta todos IDs
147 */
148 //-----
149 Function STBSetIdTF(cTipoTEF)
150
151 Default cTipoTEF := ""
152
153 If cTipoTEF == "CC"
154     aIDTEF[1] = SOMA1(cValToChar(aIDTEF[1]))
155 ElseIf cTipoTEF == "CD"

```

```

156     aIDTEF[2] = SOMA1(cValToChar(aIDTEF[2]))
157 ElseIf cTipoTEF == "PX"
158     aIDTEF[3] = SOMA1(cValToChar(aIDTEF[3]))
159 ElseIf cTipoTEF == "PD"
160     aIDTEF[4] = SOMA1(cValToChar(aIDTEF[4]))
161 Else
162     aIDTEF[1] := '0'
163     aIDTEF[2] := '0'
164     aIDTEF[3] := '0'
165     aIDTEF[4] := '0'
166 EndIf
167
168 Return Nil
169
170
171 //-----
172 /*{Protheus.doc} STBGetIdTF
173 Retorna o ID atual do tipo de cartão requisitado
174
175 @param      cTipoTEF - tipo do cartão (CC/CD/PX/PD)
176 @return     cIDTEF - ID atual do Cartão
177
178 */
179 //-----
180 Function STBGetIdTF(cTipoTEF)
181
182 Local cIDTEF := '0'
183
184 Default cTipoTEF := ""
185
186 If cTipoTEF == "CC"
187     cIDTEF := aIDTEF[1]
188 ElseIf cTipoTEF == "CD"
189     cIDTEF := aIDTEF[2]
190 ElseIf cTipoTEF == "PX"
191     cIDTEF := aIDTEF[3]
192 ElseIf cTipoTEF == "PD"
193     cIDTEF := aIDTEF[4]
194 EndIf
195
196 Return cIDTEF
197
198
199 //-----
200 /*{Protheus.doc} STBGetAdmF
201 Alimenta o atributo da Administradora Financeira, seja retornada automaticamente ou selecionada pelo usuário.
202
203
204 @param      oRetTran, Objeto, Objeto com informações do retorno da transação TEF.
205 @param      oTEF20 , Objeto, Objeto do TEF 2.0.
206
207 @return     Nil, Nulo

```

```

208 /*/
209 //-----
210 Function STBGetAdmF(oRetTran, oTEF20)
211 Local nAdmRet := 0
212 Local nI := 0
213 Local lRetAux := .F.
214 Local aAdmin := {}
215 Local aAuxAdmin := {} //Administradoras financeiras filtradas pela forma de pagamento
216 Local nParcs := 1
217 Local nPosAdmin := 1
218 Local aAdminDesc:= {} //Administradoras financeiras selecionadas com base em duas descricao
219 Local cFirst := STGetCfg("cIntegration", "DEFAULT")
220 Local lFirst := ValType(cFirst) = "C" .AND. cFirst == "FIRST"
221 Local lSelAdm := !lFirst //Integração com o FIRST não necessita de adm financeira
222 Local aAdmSel := {} //Administradora financeira selecionada manualmente
223 Local nTamL4ADM := TamSX3("L4_ADMINIS")[1]
224 Local aAdmsSelec:= {} //Lista com as Adm. Financeiras para apresentar na tela para seleção manual
225
226 //Quantidade de Administradoras Financeiras retornadas de forma automatica
227 nAdmRet := Len(oRetTran:oRetorno:aAdmin)
228
229 /*
230 Se foi retornada uma UNICA administradora, houve sucesso na obtenção automatica,
231 já se NENHUMA ou MAIS DE UMA administradora foi retornada, a administradora deve ser selecionada manualmente.
232 */
233 If nAdmRet == 1
234
235     oRetTran:oRetorno:cAdmFin := PadR( oRetTran:oRetorno:aAdmin[1][3], nTamL4ADM )
236     lRetAux := .T.
237
238     LjGrvLog(Nil, "Adm.Fin. retornada automaticamente: ", oRetTran:oRetorno:cAdmFin)
239
240 Else
241
242     If nAdmRet == 0
243
244         LjGrvLog(Nil, "Nenhuma Adm.Fin. foi retornada")
245
246         //obtemos todas as administradoras financeiras cadastradas
247         aAdmin := aClone( oTEF20:Administradoras() )
248
249         //filtramos as Administradora pela forma de pagamento da transação TEF
250         For nI := 1 to Len(aAdmin)
251             If AllTrim(aAdmin[nI][2]) == AllTrim(oRetTran:cFormaPgto)
252                 aAdd( aAuxAdmin, aAdmin[nI] )
253             EndIf
254         Next
255
256     ElseIf nAdmRet > 1
257         LjGrvLog(Nil, "Foi retornado mais de uma Adm.Fin.: ", oRetTran:oRetorno:aAdmin)
258
259         //obtemos somente as administradoras retornadas automaticamente

```



```

260     aAuxAdmin := oRetTran:oRetorno:aAdmin
261 EndIf
262
263 /*
264 Buscamos a Adm.Fin. comparando sua descrição [AE_DESC ou MDE_DESC(se AE_ADMCART estiver preenchida)]
265 com a descrição da bandeira obtida pelo HashTable do LOJA1926.
266 Caso nenhuma administradora seja encontrado, a administradora (L4_ADMINIS) terá como prefixo a string "TEF: ",
267 assim essa venda não será processada pelo LjGrvBatch, até que esse campo seja ajustado (AE_COD + ' - ' + AE_DESC')
268 */
269 If !lRetAux .AND. Len(aAuxAdmin) > 0
270
271     //
272     //Seleção da Administradora Financeira pela Descricao da Bandeira
273     //
274     LjGrvLog( Nil, "Busca pela Adm.Fin. usando a descrição da bandeira", oRetTran:oRetorno:cAdmFin)
275
276     If oRetTran:oRetorno:nParcs > 0
277         nParcs := oRetTran:oRetorno:nParcs
278     EndIf
279     LjGrvLog( Nil, "Quantidade de Parcelas: ", nParcs )
280
281     /*
282     Para os casos onde a busca pela administradora financeira nao foi feita pelo MDE_CODSIT,
283     a busca será feita comparando a descrição da bandeira retornada com MDE_DESC(se AE_ADMCART preenchida)/AE_DESC,
284     além de comparar se a parcela retornada esta no range de parcelas configurada da administrada
285     */
286     While nPosAdmin > 0
287         nPosAdmin := Ascan( aAuxAdmin, {|x| oRetTran:oRetorno:cAdmFin $ x[7] .AND. (nParcs >= x[4] .AND. nParcs <= x[5])}, nPosAdmin )
288         If nPosAdmin > 0
289             Aadd( aAdminDesc, aAuxAdmin[nPosAdmin] )
290             nPosAdmin++
291         EndIf
292     EndDo
293
294     If Len(aAdminDesc) == 1
295         oRetTran:oRetorno:cAdmFin := PadR( aAdminDesc[1][3], nTamL4ADM )
296         lRetAux := .T.
297
298         LjGrvLog( Nil, "Adm.Fin. retornada automaticamente pela descrição da bandeira: ", oRetTran:oRetorno:cAdmFin)
299     ElseIf Len(aAdminDesc) == 0
300         LjGrvLog( Nil, "Adm.Fin. nao encontrada ou as parcelas configuradas nao condizem com a retornada" )
301     Else
302         //filtramos pelas administradoras que tem a mesma descricao, para que o usuario possa selecionar uma delas
303         aAuxAdmin := aClone( aAdminDesc )
304
305         LjGrvLog( Nil, "Retornado mais de uma Adm. Fin com a mesma descrição (AE_DESC/MDE_DESC)", aAdminDesc )
306     EndIf
307
308
309     //
310     //Seleção Manual da Administradora Financeira
311     //

```

```

312     LjGrvLog( Nil, "Escolha manual da Adm.Fin.: ", lSelAdm )
313
314     If !lRetAux .AND. lSelAdm
315
316         //-----
317         // Abre a tela para Seleção da administradora Financeira, caso não tenha identificado automaticamente.
318         //-----
319         If nModulo == 23 //SIGAFRT (Totvs PDV)
320             aAdmSel := STICrdSlAdm( aAuxAdmin, oRetTran:nValor, oRetTran:cAdmFin )
321         Else
322             //SIGALOJA (Venda Assistida) / SIGAFAT (Venda Direta)
323             If ExistFunc("FRgTelaSAE")
324                 For nI:=1 To Len(aAuxAdmin)
325                     //Constroi o array no formato que a função FRgTelaSAE está esperando
326                     aAdd( aAdmsSelec, { .F., aAuxAdmin[nI][2], aAuxAdmin[nI][3] } )
327                 Next nI
328                 nPosAdmin := FRgTelaSAE(aAdmsSelec,oRetTran:cAdmFin,.T.,oRetTran:cIdPagto)
329                 aAdmSel := aClone( { aAuxAdmin[nPosAdmin] } )
330             EndIf
331         EndIf
332
333         If Len(aAdmSel) == 1
334             oRetTran:oRetorno:aAdmin := aClone(aAdmSel)
335             oRetTran:oRetorno:cAdmFin := PadR( aAdmSel[1][3], nTamL4ADM )
336
337             lRetAux := .T.
338
339             LjGrvLog( Nil, "Adm.Fin. escolhida: ", aAdmSel[1][3] )
340         Else
341             LjGrvLog( Nil, "Nenhuma Adm.Fin. escolhida" )
342         EndIf
343     EndIf
344 EndIf
345
346 If !lRetAux
347     oRetTran:oRetorno:cAdmFin := "TEF: " + oRetTran:oRetorno:cAdmFin
348
349     LjGrvLog( Nil, "Nenhuma Adm.Fin. foi utilizada, portanto sera gravado TEF: + descrição, para que a venda possa ser ajustada" )
350 EndIf
351
352 Else
353     oRetTran:oRetorno:cAdmFin := "TEF: " + oRetTran:oRetorno:cAdmFin
354
355     LjGrvLog( Nil, "Nenhuma Adm.Fin. disponivel para busca, portanto sera gravado TEF: + descrição, para que a venda possa ser ajustada" )
356 EndIf
357
358 EndIf
359
360 Return
361
362 /*{Protheus.doc} STBCalcJur
363 Atualiza valores de taxa da Adm Financiera caso exista

```

```

364
365 @type Function
366 @param nil
367 @return nil
368 */
369 Function STBCalcJur()
370 Local aJurosAdm := STIGetAJur() // Dados do juros da Adm Fin
371 Local lMVLJJURCC := SuperGetMV( "MV_LJJURCC",,.F. ) // calcula juros da Adm financeira
372
373 If lMVLJJURCC
374     STWAddIncrease(aJurosAdm[4],aJurosAdm[3])
375 Endif
376
377 Return
378
379 /*{Protheus.doc} STBCalcTax
380 Calcula valores de taxa da Adm Financiera caso exista
381
382 @type Function
383 @param cCodAdmin    , character , código da Adm Financeira
384         nParc        , numeric   , numero da parcela
385         nValCC        , numeric   , valor da forma de pagamento
386         nVlJuroAdm    , numeric   , valor dos juros, para será atualizado na tela
387 @return
388 */
389 Function STBCalcTax(cCodAdmin, nParc, nValCC, nVlJuroAdm )
390 Local aValTax      := {} // Valores das taxas
391 Local lMVLJJURCC    := SuperGetMV( "MV_LJJURCC",,.F. ) // calcula juros da Adm financeira
392
393 Default cCodAdmin   := ""
394 Default nParc       := 1
395 Default nValCC      := 0
396
397 /*
398 aValTax[1] -> Taxa de Juros
399 aValTax[2] -> Valor dos Juros COM Imposto nTxVal //Valor do Juros COM Imposto
400 aValTax[3] -> Taxa da Adm Fin
401 aValTax[4] -> Valor do Juros SEM Imposto, no caso de ( LOCALIZACA0 <> Brasil)
402 */
403
404 If lMVLJJURCC .AND. !Empty(nValCC) .AND. !Empty(cCodAdmin)
405
406     cCodAdmin := SubStr( cCodAdmin, 1, TamSX3("AE_COD")[1] )
407
408     nValCC:=StrTran(If(ValType(nValCC)<>"C", Str(nValCC),nValCC),".","")
409
410     aValTax := LJ7_TxAdm( cCodAdmin, nParc, nValCC )
411
412     nVlJuroAdm := aValTax[2]
413
414     If nVlJuroAdm > 0
415         STISetAJur({.T.,cCodAdmin,aValTax[1],aValTax[2]})

```

```
416     Else
417         STISetAJur({.F., "", 0, 0})
418     EndIf
419
420 EndIf
421
422 Return
```

Componentes/Payment/STBPayCdPg.prw

557 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPAYCDPG.CH"
4
5 Static nValAcres      := 0          //Valor do acrescimo
6 Static cAcres         := ""         //Tipo de Acrescimo financeiro
7 Static aParcelas      := {}         //Parcelas gerada pela funcao Condicao()
8 Static cCondicao       := ""         //Codigo da condicao de pagamento
9 Static cForma         := ""         //Forma de pagamento
10 Static aDiasPgChck    := {}         // Dias das parcelas da Condicao de Pagamento em Cheque
11 Static aAcresFin      := {0,0}      // Dados do acrescimo financeiro aAcresFin[1] -> Valor, aAcresFin[1] -> Aliquota
12
13
14 //-----
15 /*{Protheus.doc} STBChkIsVp
16 Verifica se o tipo do vale presente selecionado é do tipo VP
17
18 @param      oMdlGrd - Grid do Model
19 @return      lRet - tipo do vale presente selecionado é do tipo VP
20 @obs
21 @sample
22 */
23 //-----
24 Function STBChkIsVp( oMdlGrd )
25 Local lRet := .T. //Variavel de retorno
26 Local nI   := 0   //Variavel de loop
27
28 Default oMdlGrd := Nil
29
30 For nI := 1 To oMdlGrd:Length()
31     oMdlGrd:GoLine(nI)
32     If oMdlGrd:GetValue('L4_MARCAR') .AND. AllTrim(oMdlGrd:GetValue('L4_FORMA')) == 'VP'
33         lRet := .F.
34         STFMessage("STBPayCdPg", "Alert", STR0001) //"Para o caso de Vale-Presente, utilize Forma de Pagamento ao invés de Cond. Pagamento."
35         STFShowMessage("STBPayCdPg")
36         Exit
37     EndIf
38 Next nI
39
40
41 Return lRet
42
43
44 //-----
45 /*{Protheus.doc} STBChkTpCP
46 Verifica o tipo de condicao selecionada
47
48 @param      oMdlGrd - Grid do Model
49 @return      lRet - tipo e CP
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STBChkTpCP( oMdlGrd )
55
56 Local nI      := 0          //Variavel de loop
57 Local lRet    := .T.        //Variavel de retorno
58 Local oTotal:= STFGetTot()  //Totalizador
59
60 Default oMdlGrd := Nil
61
62 cForma := "" //Limpa variavel estatica para nao preencher pagamento automaticamente quando o MV_CONDPAD estiver vazio.
63
64 For nI := 1 To oMdlGrd:Length()
65     oMdlGrd:GoLine(nI)
66     If oMdlGrd:GetValue('L4_MARCAR')
67
68         cCondicao := oMdlGrd:GetValue('L4_CODIGO')
69         cForma     := oMdlGrd:GetValue('L4_FORMA')
70
71         If AllTrim(oMdlGrd:GetValue('L4_TIP0')) == 'A'
72             lRet := .F.
73             // Retornar a tela das formas de pagamento
74             STIPayCancel()
75             STFMessage("STBPayCdPg", "Alert",STR0002) // "A condição de pagamento do tipo A é de uso exclusivo dos ambientes de veículos e oficinas."
76             STFShowMessage("STBPayCdPg")
77         ElseIf AllTrim(oMdlGrd:GetValue('L4_TIP0')) == '9'
78             lRet := .F.
79             // Retornar a tela das formas de pagamento
80             STIPayCancel()
81             STFMessage("STBPayCdPg", "Alert", STR0003) //"A condição de pagamento do tipo 9 é de uso exclusivo do faturamento."
82             STFShowMessage("STBPayCdPg")
83         ElseIf Empty(AllTrim(cForma))
84             //Se na condicao de pagamento o E4_FORMA nao esta preenchido, assume como padrao o cheque.
85             cForma = 'CH'
86         EndIf
87         Exit
88     EndIf
89 Next nI
90
91 /* Verifica limites da condição de pagamento */
92 DbSelectArea("SE4")
93 SE4->(DbSetOrder(1))
94 If SE4->(DbSeek(xFilial("SE4")+ cCondicao))
95     If SE4->E4_SUPER <> 0 .and. oTotal:GetValue("L1_VLRTOT") > SE4->E4_SUPER
96         lRet := .F.
97         // Retornar a tela das formas de pagamento, necessário para não perder o foco da tela
98         STIPayCancel()
99         STFMessage("STBPayCdPg", "STOP",STR0004) // "A condição de pagamento possui limite de pagamento inferior ao valor total da venda."
100        STFShowMessage("STBPayCdPg")
101    ElseIf SE4->E4_INFER <> 0 .and. oTotal:GetValue("L1_VLRTOT") < SE4->E4_INFER
102        lRet := .F.
103        // Retornar a tela das formas de pagamento
104        STIPayCancel()

```

```

105      STFMessage("STBPayCdPg", "STOP",STR0005) // "A condição de pagamento possui limite de pagamento superior ao valor total da venda."
106      STFShowMessage("STBPayCdPg")
107      EndIf
108 EndIf
109
110 Return lRet
111
112 //-----
113 /*{Protheus.doc} STBApliDes
114 Aplicando desconto
115
116 @param      oMdlGrd - Grid do Model
117 @return      lRet - aplicou desconto
118 @obs
119 @sample
120 */
121 //-----
122 Function STBApliDes( oMdlGrd )
123 Local nI      := 0      //Variavel de loop
124 Local lDesTot := STBCDPGDes()
125
126 Default oMdlGrd := Nil
127
128 For nI := 1 to oMdlGrd:Length()
129     oMdlGrd:GoLine(nI)
130
131     If oMdlGrd:GetValue('L4_MARCAR') .And. oMdlGrd:GetValue('L4_DESCFIN') > 0
132         If lDesTot
133             STFSetTot("L1_DESCNF", oMdlGrd:GetValue('L4_DESCFIN'))
134         EndIf
135         STWTotDisc( oMdlGrd:GetValue('L4_DESCFIN') , 'P', 'CP', .T.)
136         Exit
137     EndIf
138 Next nI
139
140 Return .T.
141
142 //-----
143 /*{Protheus.doc} STBApliAcr
144 Aplicando acrescimo
145
146 @param      oMdlGrd - Grid do Model
147 @return      lRet - aplicou acrescimo
148 @obs
149 @sample
150 */
151 //-----
152 Function STBApliAcr( oMdlGrd )
153
154 Local nI      := 0      //Variavel de loop
155 Local aAcres  := {}     //Acrescimo em valor
156 Local oTotal  := STFGetTot() // Recebe o Objeto totalizador

```

```

157 Local lRecebTitle := STIGetRecTit() // Indica se eh recebimento de titulos
158
159 Default oMdlGrd := Nil
160
161 For nI := 1 To oMdlGrd:Length()
162     oMdlGrd:GoLine(nI)
163     If oMdlGrd:GetValue('L4_MARCAR') .AND. oMdlGrd:GetValue('L4_ACRSFIN') > 0
164
165         aAcres := STBDiscConvert( oMdlGrd:GetValue('L4_ACRSFIN') , 'P' )
166         STWAddIncrease( aAcres[1] , oMdlGrd:GetValue('L4_ACRSFIN') )
167
168         If Len(aAcres) > 0 .AND. aAcres[1] > 0
169             STBSetAcresFin(aAcres[1], aAcres[2])
170             LjGrvLog( "L1_NUM: " + STDGPBasket('SL1','L1_NUM'), "Acrécimo Financeiro incluso ", aAcres[1] )
171         EndIf
172
173         // Se for recebimento de títulos não traz o acrescimo da condição de pagamento
174         // Necessário para evitar erro na baixa do título.
175         If lRecebTitle
176             oTotal:SetValue("L1_VLRTOT",oTotal:GetValue( "L1_VLRTOT"))
177             STFMessage("STBPayCdPg", "ALERT", STR0006) //"Acrécimo financeiro só é permitido na seleção do título."
178             STFShowMessage("STBPayCdPg")
179         EndIf
180         Exit
181     Else
182         nValAcres := 0
183     EndIf
184 Next nI
185
186 Return .T.
187
188
189 //-----
190 /*{Protheus.doc} STBParcCp
191 Gera as parcelas da condicao de pagamento
192
193 @param
194 @return      lRet - Gerou parcelas
195 @obs
196 @sample
197 */
198 //-----
199 Function STBParcCp()
200
201 Local lCalcFin := (SuperGetMv("MV_CALCFIN",,"M")== "F") //Variavel para controle do parametro MV_CALCFIN = [F]iniciado
202 Local nI      := 0 //Variavel de loop
203 Local oTotal  := STFGetTot() //Recebe o Objeto totalizador
204 Local nTotal  := oTotal:GetValue("L1_VLRTOT") //Total da venda
205
206 aParcelas := {}
207
208 If !Empty(cCondicao)

```



```

209     If nValAcres > 0
210         If cAcres $ 'N '
211             aParcelas := Condicao( nTotal, cCondicao, , , , , nValorAcres )
212         Else
213             aParcelas := Condicao( nTotal - nValAcres, cCondicao )
214         EndIf
215     Else
216         aParcelas := Condicao( nTotal, cCondicao )
217     EndIf
218 EndIf
219
220 Return .T.
221
222
223 //-----
224 /*{Protheus.doc} STBTratPar
225 Tratamento das parcelas geradas
226
227 @param
228 @return      lRet - Tratou parcelas
229 @obs
230 @sample
231 */
232 //-----
233 Function STBTratPar()
234
235 Local nI := 0 //Variavel de loop
236
237 If nValAcres > 0
238     If cAcres $ 'JN '
239         If lCalcFin .AND. Len(aParcelas) > 0
240             For nI := 1 To Len(aParcelas)
241                 If aParcelas[nI][1] = dDatabase
242                     If cAcres <> "J"
243                         aParcelas[nI][2] -= nValorAcres / Len( aParcelas )
244                     EndIf
245                 EndIf
246             Next nI
247             If cAcres == "J"
248                 aParcelas[1][2] += nValorAcres
249             EndIf
250         EndIf
251     EndIf
252
253
254
255
256
257
258
259
260 EndIf

```

```

261
262     ElseIf cAcres == "S"
263
264         aEval( aParcelas,{ |ExpA1,nCntFor| nValRat += If( nCntFor == 1, 0, ExpA1[2] ) } )
265         aEval( aParcelas,{ |ExpA1,nCntFor| nValTot += ExpA1[2] } )
266
267         For nI := 1 to Len( aParcelas )
268             If nI == 1
269                 aParcelas[nI][2] := nValorAcres
270             Else
271                 aParcelas[nI][2] := NoRound( nValTot * ( aParcelas[nI][2] / nValRat ), nDecimais )
272             EndIf
273         Next nI
274
275     EndIf
276
277 EndIf
278
279 Return .T.
280
281
282 //-----
283 /*/{Protheus.doc} STBGetTpPay
284 Retorna o tipo da forma de pagamento
285
286 @param
287 @return      cForma - Forma de pagamento
288 @obs
289 @sample
290 */
291 //-----
292 Function STBGetTpPay()
293 Return AllTrim(cForma)
294
295
296 //-----
297 /*/{Protheus.doc} STBGetTpPay
298 Retorna o tipo da forma de pagamento
299
300 @param
301 @return      cForma - Forma de pagamento
302 @obs
303 @sample
304 */
305 //-----
306 Function STBGetCdPg()
307 Return AllTrim(cCondicao)
308
309
310 //-----
311 /*/{Protheus.doc} STBTrnsCard
312 Condicao de pagamento com transacao TEF

```

```

313
314 @param   cTypePay - Tipo de pagamento
315 @return   lRet - Retorna se executou corretamente
316 @obs
317 @sample
318 /*/
319 //-----
320 Function STBTrnsCard( cTypePay )
321
322 Local oTEF20   := STBGetTEF()
323 Local lRet     := .F.
324 Local oPanPayCrd := STIGetPan() // Gera uma copia do oPanPayment original
325
326 Default cTypePay := ""
327
328 ParamType 0 Var   cTypePay   As Character   Default   ""
329
330 If ValType(oTEF20) <> '0' .OR. oTEF20:LATIVO == .F. // Se não utilizar TEF chama tela de cartão manual
331
332     STIAddNewPan( oPanPayCrd, ,cTypePay )
333
334 Else
335
336     lRet := STWTypeTran( Nil, oTEF20, AllTrim(cTypePay), Len(aParcelas))
337
338 EndIf
339
340 Return lRet
341
342
343 //-----
344 /*/{Protheus.doc} STBTrnsCheck
345 Condicao de pagamento com transacao de Cheque
346
347 @param
348 @return   lRet - Retorna se executou corretamente
349 @obs
350 @sample
351 /*/
352 //-----
353 Function STBTrnsCheck(aCondPg)
354
355 Local oPainel      := Eval(STIGetBlCod())
356 Local bBlCod       := STIGetBlCod()
357 Local cCondPg      := "" // Condicao de pagamento
358 Local aDiasPgChck  := {} // Array com a condicao de pagamento Iindica para a funao STICHConfPay que a operação é via Condição de Pagamento
359 Local nX           := 0 // Contador do For
360
361 Default aCondPg    := {} // Array com as Condicoes de Pagamento da SE4
362
363 For nX := 1 to Len(aCondPg)
364     If aCondPg[nX][1][1][1]

```

```

365         cCondPg := aCondPg[nX][1][1][2]
366         aDiasPgChck := STDCondPad(cCondPg)
367         aDiasPgChck := aDiasPgChck[1][8]
368     EndIf
369 Next nX
370
371 STBSetDPgChck( aDiasPgChck )
372
373 STICHConfPay(Nil, Nil, Nil, bBlCod, oPainel)
374
375 Return .T.
376
377
378 //-----
379 /*{Protheus.doc} STBTrnsCp
380 Inclui a condicao de pagamento no model
381
382 @param
383 @return      lRet - Retorna se executou corretamente
384 @obs
385 @sample
386 */
387 //-----
388 Function STBTrnsCp()
389 Local oTotal      := STFGetTot()                //Recebe o Objeto totalizador
390 Local nTotal      := oTotal:GetValue("L1_VLRTOT") //Total da venda
391
392 STIAddPay(cForma, Nil, Len(aParcelas), .T., /*cCodVp*/, nTotal)
393
394 Return .T.
395
396 //-----
397 /*{Protheus.doc} STBGetParc()
398 Retorna a variavel de parcelas
399
400 @param
401 @return      aParcelas - Informacoes das parcelas
402 @obs
403 @sample
404 */
405 //-----
406 Function STBGetParc()
407 Return aParcelas
408
409 //-----
410 /*{Protheus.doc} STBSetParc()
411 Limpa o array a parcelas
412
413 @param
414 @return      .T.
415 @obs
416 @sample

```

```

417 /*/
418 //-----
419 Function STBSetParc()
420 aParcelas := {}
421 Return .T.
422
423 //-----
424 /*/{Protheus.doc} STBCDPGAtuBasket()
425 Atualiza campos referente a Condição de pagamento na Cesta.
426
427 @param
428 @return      .T.
429 @obs
430 @sample
431 /*/
432 //-----
433 Function STBCDPGAtuBasket(lLimpa)
434
435 Default lLimpa := .F.
436
437 If FindFunction("STDSCondPG")
438     STDSCondPG(Iif(lLimpa,"",STBGetCdPg()))
439 EndIf
440
441 Return Nil
442
443 //-----
444 /*/{Protheus.doc} STBCDPGDes()
445 Valida se os fontes tratam o desconto condição de pagamento
446 com desconto no total
447
448 @return      .T.
449 /*/
450 //-----
451 Function STBCDPGDes()
452 Local lRet           := .F.
453 Local dDtFontes      := Ctod("14/05/2018")
454 Local lSTBDiscountTotalSale := GetApoInfo("STBDiscountTotalSale.PRW")[4] >= dDtFontes
455 Local lSTBImportSale      := GetApoInfo("STBImportSale.PRW")[4] >= dDtFontes
456 Local lSTBPayCdPg         := GetApoInfo("STBPayCdPg.PRW")[4] >= dDtFontes
457 Local lSTBPayment         := GetApoInfo("STBPayment.PRW")[4] >= dDtFontes
458 Local lSTBValues          := GetApoInfo("STBValues.PRW")[4] >= dDtFontes
459 Local lSTDProductBasket   := GetApoInfo("STDProductBasket.PRW")[4] >= dDtFontes
460 Local lSTFTotalUpdate     := GetApoInfo("STFTotalUpdate.PRW")[4] >= dDtFontes
461 Local lSTIDiscountTotal   := GetApoInfo("STIDiscountTotal.PRW")[4] >= dDtFontes
462
463 lRet := lSTBDiscountTotalSale .And. lSTBImportSale .And. lSTBPayCdPg .And. ;
464       lSTBPayment .And. lSTBValues .And. lSTDProductBasket .And. ;
465       lSTFTotalUpdate .And. lSTIDiscountTotal
466
467 Return lRet
468

```

```

469 //-----
470 /*{Protheus.doc} STBGetParc()
471 Retorna o array aDiasPgChck
472
473 @param
474 @return      aDiasPgChck - Intervalo de dias da Condicao de Pagamento (SE4)
475 @obs
476 @sample
477 */
478 //-----
479 Function STBGetDPgChck()
480 Return aDiasPgChck
481
482 //-----
483 /*{Protheus.doc} STBSetParc()
484 Atribui valor ao array aDiasPgChck
485
486 @param
487 @return      .T.
488 @obs
489 @sample
490 */
491 //-----
492 Function STBSetDPgChck( aValue )
493 Default aValue := {}
494
495 aDiasPgChck := aValue
496
497 Return .T.
498
499 //-----
500 /*{Protheus.doc} STBSetAcresFin()
501 Atribui valor ao array aAcresFin
502
503 @param      nValor -> Valor do acrescimo, nAliq -> aliquota do acrescimo
504 @return      .T.
505 */
506 //-----
507 Function STBSetAcresFin( nValor, nAliq )
508
509 aAcresFin[1] := nValor
510 aAcresFin[2] := nAliq
511
512 Return .T.
513
514 //-----
515 /*{Protheus.doc} STBGetAcresFin()
516 Retorna valor do array aAcresFin
517
518 @param
519 @return      valor do array aAcresFin
520 */

```

```

521 //-----
522 Function STBGetAcresFin()
523
524 Return aAcresFin
525
526
527 //-----
528 /*{Protheus.doc} STBRstCdPg()
529 Reseta a variável static sCondicao
530
531 @param
532 @return      nil
533 */
534 //-----
535 Function STBRstCdPg()
536 cCondicao:=""
537 Return
538
539 /*{Protheus.doc} STBCondPad
540     rotina criada para resolver o problema quando o Totvs PDV é online e conflita com o MV_CONDPAD
541     da Retaguarda. Para isso criamos o PE para que o Totvs PDV tenha seu próprio parâmetro
542     @type      Function
543     @return    cCondPad, caracter, Condição de Pagamento
544     */
545 Function STBCondPad()
546 Local cCondPad := ""
547 Local lPECondPad := ExistBlock("STCondPad")
548
549 If lPECondPad
550     LjGrvLog(NIL, "Antes da execução do PE STCondPad",{STBIsImpOrc(), STIGetRecTit()})
551     cCondPad := ExecBlock( "STCondPad",.F.,.F. , {STBIsImpOrc(), STIGetRecTit()})
552     LjGrvLog(NIL, "Depois da execução do PE STCondPad",cCondPad )
553 Else
554     cCondPad:= SuperGetMv('MV_CONDPAD',.F.,'')
555 Endif
556
557 Return cCondPad

```

Componentes/Payment/STBPayCheck.prw

274 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPAYCHECK.CH"
4
5 Static lCheck      := .F. //Se utilizou cheque ou nao
6 Static nParcels    := 0    //Numero de parcelas
7 Static oTEF20      := Nil   //Objeto do TEF
8
9 //-----
10 /*{Protheus.doc} STBLECMC7
11 Faz a leitura do cheque
12
13 @param
14 @return aRet - array com dados do cheque
15 @obs
16 @sample
17 */
18 //-----
19 Function STBLECMC7()
20
21 Local oMdl      := STIGetMdl() //Get no model do cheque
22 Local oModel    := oMdl:GetModel("CHECKMASTER") //Model master
23 Local aRet      := {} //Retorno dos dados do cheque
24 Local aDados    := {} //Quantidade de parcelas
25 Local nI        := 0 //Variavel de Loop
26
27 For nI := 1 To oModel:GetValue("L4_PARCELAS")
28
29     STFMessage("STBPayCheck", "OK", STR0001) //"Posicione o cheque no CMC7 para leitura!"
30     STFShowMessage("STBPayCheck")
31
32     Aadd(aDados, nI)
33     Aadd(aRet, STFFireEvent(ProcName(0), "STReadCmc7", aDados ))
34     aDados := {}
35
36 Next nI
37
38 Return aRet
39
40
41 //-----
42 /*{Protheus.doc} STBConCheck
43 Faz a consulta do cheque
44
45 @param      aRetCheck - Array com informacoes do TEF
46 @return lRet - Informa se executou corretamente
47 @obs
48 @sample
49 */
50 //-----
51 Function STBConCheck( aRetCheck )
52 Local oMdl := STIGetMdl() //Get no model do cheque

```



```

53 Local oModel      := oMdl:GetModel("CHECKMASTER") //Model master
54 Local nValor       := oModel:GetValue("L4_VALOR") //Valor do pagamento em cheque
55 Local dDtVenc      := oModel:GetValue("L4_DATA") //Vencimento do cheque
56 Local cCupom       := STBRetCup() //Numero do cupom
57 Local cCgcCli      := STDGPBasket("SL1","L1_CGCCLI")
58 Local cTipoCli     := ""
59 Local nI           := 0 //Variavel de loop
60 Local nX           := 0
61
62 Default aRetCheck := {}
63
64 oTEF20 := STBGetTEF()
65 aRetCheck := STWGetCkRet()
66
67 If ValType(cCgcCli) <> "C"
68     cCgcCli := ""
69 EndIf
70
71 For nI := 1 To Len(aRetCheck)
72     If ValType(aRetCheck[1][1][1]) == "A"
73         nX := 1
74         While nX <= Len(aRetCheck[1]) .AND. Len(aRetCheck[1]) > 0
75             oDados := LJCDadosTransacaoCheque():New(nValor, Val(cCupom), Date(), Time(),;
76                 Val(aRetCheck[nI][1][nX][1]), Val(aRetCheck[nI][1][nX][3]), Val(aRetCheck[nI][1][nX][4]), Val(aRetCheck[nI][1][nX][2]),;
77                 0, 0, 0, dDtVenc, Val(aRetCheck[nI][1][nX][5]), cTipoCli, cCgcCli)
78
79             oRetTran := oTEF20:Cheque():Consultar(@oDados)
80
81             If oRetTran:retorno:lTransOk
82
83                 oTEF20:Cupom():Inserir("G", oRetTran:retorno:oViaCaixa, oRetTran:retorno:oViaCliente, "H",;
84                     oTEF20:Cheque():GetTotalizador(), oTEF20:Cheque():GetFormaPgto(oTEF20:Formas()), oRetTran:nValor, 1,;
85                     0)
86
87                 If nX == Len(aRetCheck[1])
88                     lCheck := .T.
89                     STBInsCheck()
90                 EndIf
91
92                 nX++
93             Else
94                 adel(aRetcheck[1], nX) //deleta a transacao
95                 aSize(aRetCheck[1], Len(aRetCheck[1]) -1) //deleta o array
96             EndIf
97         End
98     Else
99         oDados := LJCDadosTransacaoCheque():New(nValor, Val(cCupom), Date(), Time(),;
100             Val(aRetCheck[nI][1][1]), Val(aRetCheck[nI][1][3]), Val(aRetCheck[nI][1][4]), Val(aRetCheck[nI][1][2]),;
101             0, 0, 0, dDtVenc, Val(aRetCheck[nI][1][5]), Nil, Nil)
102
103         oRetTran := oTEF20:Cheque():Consultar(oDados)
104

```

```

105      If oRetTran:oRetorno:lTransOk
106
107          oTEF20:Cupom():Inserir("G", oRetTran:oRetorno:oViaCaixa,oRetTran:oRetorno:oViaCliente, "H",;
108              oTEF20:Cheque():GetTotalizador(),oTEF20:Cheque():GetFormaPgto(oTEF20:Formas()), oRetTran:nValor, 1,;
109              0)
110
111      If nI == Len(aRetCheck)
112          lCheck := .T.
113          STBInsCheck()
114      EndIf
115
116      Else
117          adel(aRetcheck, nI) //deleta a transacao
118          aSize(aRetCheck, Len(aRetCheck) -1) //deleta o array
119      EndIf
120  EndIf
121 Next nI
122
123 Return .T.
124
125
126 //-----
127 /*{Protheus.doc} STBPrintChk
128 Impressao do cheque
129
130 @param      aRetCheck - Array com informacoes do TEF
131 @return lRet - Informa se executou corretamente
132 @obs
133 @sample
134 */
135 //-----
136 Function STBPrintChk(aRetCheck)
137
138 Local oMdl := STIGetMdl() //Get no model do cheque
139 Local oModel := oMdl:GetModel("CHECKMASTER") //Model master
140 Local aDados := {} //Dados para impressao do cheque
141 Local nI := 0 //Variavel de Loop
142
143 Default aRetCheck := {}
144
145 ParamType 0 Var      aRetCheck  As Array      Default      {}
146
147 For nI := 1 To oModel:GetValue("L4_PARCELAS")
148     STFMessage("STBPayCheck", "OK", STR0002 + AllTrim(Str(nI)) + STR0003) //"º cheque para impressao!"
149     STFShowMessage("STBPayCheck")
150     aDados := STBPrepDat( aRetCheck, nI, oModel:GetValue("L4_PARCELAS"), oModel:GetValue("L4_VALOR") )
151     STFFireEvent(ProcName(0), "STCHPRINTS", aDados)
152 Next nI
153
154 Return .T.
155
156

```

```

157 //-----
158 /*{Protheus.doc} STBPrepDat()
159 Prepara os dados para impressao do cheque
160
161 @param      aRetCheck - Array com informacoes do TEF
162 @param      nI - Linha posicionada
163 @param      Numero das parcelas
164 @param      Valor
165 @return lRet
166 @obs
167 @sample
168 */
169 //-----
170 Function STBPrepDat(aRetCheck, nI, nParcels, nVal)
171
172 Local aRet          := {}          // Retorno
173
174 Default aRetCheck   := {}
175 Default nI          := 0
176 Default nParcels    := 0
177 Default nVal        := 0
178
179 ParamType 0 Var aRetCheck As Array Default {}
180 ParamType 1 Var nI As Numeric Default 0
181 ParamType 2 Var nParcels As Numeric Default 0
182 ParamType 3 Var nVal As Numeric Default 0
183
184 Aadd(aRet, aRetCheck[nI][1][1]) //Banco
185 Aadd(aRet, AllTrim(Str((nVal / nParcels)))) //Valor
186 Aadd(aRet, If(Empty(SM0->M0_NOMECON),SM0->M0_NOME,SM0->M0_NOMECON))//Favorecido
187 Aadd(aRet, Left(SM0->M0_CIDCOB,15)) //Cidade
188 Aadd(aRet, DToS(Date())) //Data
189 Aadd(aRet, "") //Mensagem
190 Aadd(aRet, "") //Verso
191 Aadd(aRet, "") //Extenso
192 Aadd(aRet, "") //Chancela
193
194 Return aRet
195
196
197 //-----
198 /*{Protheus.doc} STBInsCheck()
199 Chama a funcao para inserir o pagamento de cheque na grid
200
201 @param
202 @return lRet - Informa se executou corretamente
203 @obs
204 @sample
205 */
206 //-----
207 Function STBInsCheck()
208

```

```

209 Local oMdl := STIGetMdl() //Get no model do cheque
210 Local oModel := oMdl:GetModel("CHECKMASTER") //Model master
211
212 STIAddPay("CH", oModel, oModel:GetValue("L4_PARCELAS"))
213
214 Return .T.
215
216
217 //-----
218 /*{Protheus.doc} STBSetCheck
219 Set na variavel lCheck para .F.
220
221 @param
222 @return Nil
223 @obs
224 @sample
225 */
226 //-----
227 Function STBSetCheck()
228 lCheck := .F.
229 Return Nil
230
231
232 //-----
233 /*{Protheus.doc} STBGetCheck
234 Retorna o valor de lCheck
235
236 @param
237 @return lRet - Retorna o valor de lCheck
238 @obs
239 @sample
240 */
241 //-----
242 Function STBGetCheck()
243
244 Local lRet := lCheck
245
246 Return lRet
247
248
249 //-----
250 /*{Protheus.doc} STBGetPcls
251 Retorna o numero do cheque a ser preenchido pelo usuario
252
253 @param
254 @return nParcels - Retorna o numero do cheque
255 @obs
256 @sample
257 */
258 //-----
259 Function STBGetPcls()
260 Return nParcels

```

```
261
262
263 //-----
264 /*/{Protheus.doc} STBGetRetChk
265 Retorno das variaveis da consulta de cheque
266
267 @param
268 @return oTEF20 - Retorno das variaveis da consulta de cheque
269 @obs
270 @sample
271 */
272 //-----
273 Function STBGetChkRet()
274 Return oTEF20
```

Componentes/Payment/STBPayGiftV.prw

226 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPAYGIFTV.CH"
4
5 Static oModelPgto := Nil
6 //-----
7 /*{Protheus.doc} STBValidVP
8 Valida se existe alguma inconsistencia do vale presente informado
9
10 @param      cValPre - Valor vale presente
11 @return cMsg - Mensagem
12 @obs
13 @sample
14 */
15 //-----
16 Function STBValidVP( cValPre )
17
18 Local lContinue      := .F.      //Retorno da funcao RemoteExecute
19 Local uRet           := Nil      //Retorno da funcao LjVldVP
20 Local cMsg           := ""       //Mensagem caso haja inconsistencia do vale presente
21
22 Default cValPre := ""
23
24 ParamType 0 Var      cValPre      As Character      Default      ""
25
26 lContinue := STBRemoteExecute( "LjVldVP"
27                               { "" , cValPre , 0 , 2 } , ;
28                               Nil , ;
29                               .F. , ;
30                               @uRet )
31
32 If lContinue
33   cMsg := uRet
34 Else
35   STFMessage("STBPayGiftV","STOP",STR0001) //"Nao foi possivel estabelecer conexão."
36   STFShowMessage("STBPayGiftV")
37 EndIf
38
39 Return cMsg
40
41
42 //-----
43 /*{Protheus.doc} STBValorVP
44 Valor do vale presente
45
46 @param      cValPre - Valor vale presente
47 @return nValor - Valor do vale presente
48 @obs
49 @sample
50 */
51 //-----
52 Function STBValorVP(cValPre)

```

```

53
54 Local lContinue      := .F. //Retorno da funcao RemoteExecute
55 Local uRet           := Nil //Retorno da funcao LjVPValor
56 Local nValor         := 0   //Valor do vale presente
57
58 Default cValPre := ""
59
60 ParamType 0 Var      cValPre      As Character      Default      ""
61
62 lContinue := STBRemoteExecute( "LjVPValor" ,;
63                               { cValPre } ,;
64                               Nil           ,;
65                               .F.          ,;
66                               @uRet        )
67
68 If lContinue
69     If ValType(uRet) == 'N'
70         nValor := uRet
71     EndIf
72 Else
73     STFMessage("STBPayGiftV","STOP",STR0001) //"Nao foi possivel estabelecer conexão."
74     STFShowMessage("STBPayGiftV")
75 EndIf
76
77 Return nValor
78
79
80 //-----
81 /*{Protheus.doc} STBBaixaVP
82 Faz a baixa do vale presente na retaguarda
83
84 @param
85 @return      lRet - Retorna se executou corretamente
86 @obs
87 @sample
88 */
89 //-----
90 Function STBBaixaVP()
91
92 Local lContinue      := .F.          //Retorno da funcao RemoteExecute
93 Local uRet           := Nil          //Retorno da funcao LjGrRVP
94 Local nI             := 1           //Variável de loop
95 Local oMdlGrdVP      := STBGtMdlVP() // VARIÁVEL para receber o valor do oMdlGrd
96
97 For nI := 1 To oMdlGrdVP:Length()
98
99     oMdlGrdVP:GoLine(nI)
100
101     If AllTrim(oMdlGrdVP:GetValue('L4_FORMA')) == 'VP'
102
103         lContinue := STBRemoteExecute( "LjGrRVP",;
104                                         { oMdlGrdVP:GetValue("L4_CODVP") , STDGPBasket("SL1", "L1_OPERAD0") , STDGPBasket("SL1", "L1_DOC") , STDGPBasket("SL1",

```

```

        "L1_ESTACA0"),;
105          STDGPBasket("SL1", "L1_PDV")      , STDGPBasket("SL1", "L1_EMISNF")      , STDGPBasket("SL1", "L1_HORA") , STDGPBasket("SL1",
        "L1_CLIENTE"),;
106          STDGPBasket("SL1", "L1_LOJA")      , STDGPBasket("SL1", "L1_SERIE"), oMdlGrdVP:GetValue("L4_VALOR") };;
107          Nil      ;;
108          .T.      ;;
109          @uRet      )
110      EndIf
111
112 Next nI
113
114 STBStMdlVP(Nil)
115
116 Return lContinue
117
118
119 //-----
120 /*{Protheus.doc} STBVldVp
121 Valida se o vale presente já nao foi lançado na grid
122
123 @param      oMdlDtl - Model de detalhes
124 @param      cCodVp - Codigo Vale presente
125 @return
126 @obs
127 @sample
128 */
129 //-----
130 Function STBVldVp(oMdlDtl,cCodVp)
131
132 Local lRet := .F.                //Variavel de retorno
133 Local nCont := 0                //Contador
134 Local nI      := 0                //Variavel de loop
135
136 Default oMdlDtl      := Nil
137 Default cCodVp      := ""
138
139 ParamType 0 Var      oMdlDtl      As Object      Default Nil
140 ParamType 1 Var      cCodVp      As Character      Default      ""
141
142 For nI := 1 To oMdlDtl:Length()
143     oMdlDtl:GoLine(nI)
144     If Alltrim(oMdlDtl:GetValue('L4_COD')) == AllTrim(cCodVp)
145         nCont++
146     EndIf
147 Next nI
148
149 If nCont >= 1
150     lRet := .F.
151     STFMessage("STBPayGiftV","STOP",STR0002) //"Vale presente já foi informado"
152     STFShowMessage("STBPayGiftV")
153 Else
154     lRet := .T.

```



```

155 EndIf
156
157 Return lRet
158
159
160 //-----
161 /*{Protheus.doc} STBVldValVp
162 Valida o valor dos vales informados
163
164 @param      oMdlDtl - Model de detalhes
165 @return      lRet - Executou corretamente
166 @obs
167 @sample
168 */
169 //-----
170 Function STBVldValVp(oMdlDtl)
171
172 Local lRet      := .F.          //Variavel de retorno
173 Local nI        := 0           //Variavel de loop
174 Local nValVps   := 0           //Valor dos vales presentes
175 Local oTotal    := STFGetTot() //Recebe o objeto do model para recuperar os valores
176
177 For nI := 1 To oMdlDtl:Length()
178     oMdlDtl:GoLine(nI)
179     nValVps += oMdlDtl:GetValue('L4_VALOR')
180 Next nI
181
182 If !SuperGetMv("MV_LJBXPAR",,.F.)
183
184     If nValVps > oTotal:GetValue("L1_VLRTOT")
185
186         lRet := .F.
187         STFMessage("STBPayGiftV","STOP",STR0003 + AllTrim(Str(nValVps, 10, 2)); //"0 valor dos vales presentes informados ("
188             + STR0004 + AllTrim(Str(oTotal:GetValue("L1_VLRTOT"), 10, 2)); //") é diferente do valor informado na forma de pagamento ("
189             + STR0005) //"). Corrija as informações antes de prosseguir."
190         STFShowMessage("STBPayGiftV")
191     Else
192         lRet := .T.
193     EndIf
194
195 Else
196     lRet := .T.
197 EndIf
198
199 Return lRet
200
201 //-----
202 /*{Protheus.doc} STBStMdlVP
203 SET do objeto oMdlGrd na variável estática oModelPgto
204
205 @param      oMdlGRd - Model da Grid de Pagamento
206 @return      sem retorno

```

```
207 @obs
208 @sample
209 /*/
210 //-----
211 Function STBStMdlVP(oMdlGrdPg)
212 oModelPgto:= oMdlGrdPg
213 Return
214
215 //-----
216 /*/{Protheus.doc} STBGtMdlVP
217 GET do objeto oModelPgto (Model da Grid de Pagamento)
218
219 @param      sem parametro
220 @return      oModelPgto (Model da Grid de Pagamento)
221 @obs
222 @sample
223 /*/
224 //-----
225 Function STBGtMdlVP()
226 Return oModelPgto
```